

>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

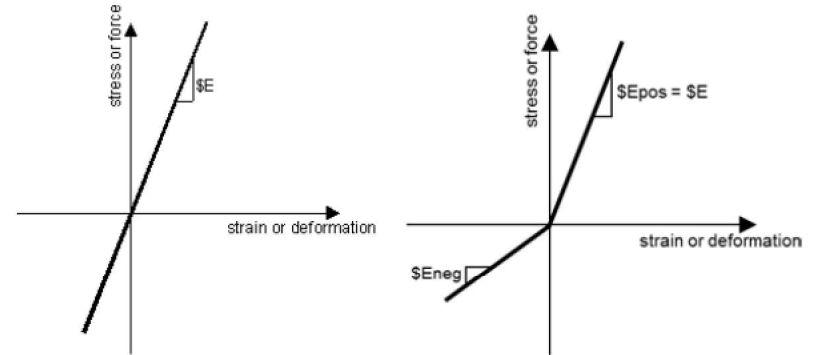
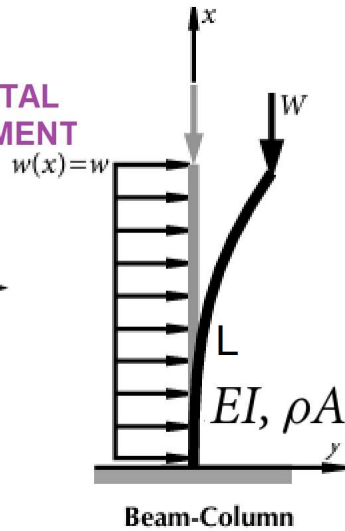
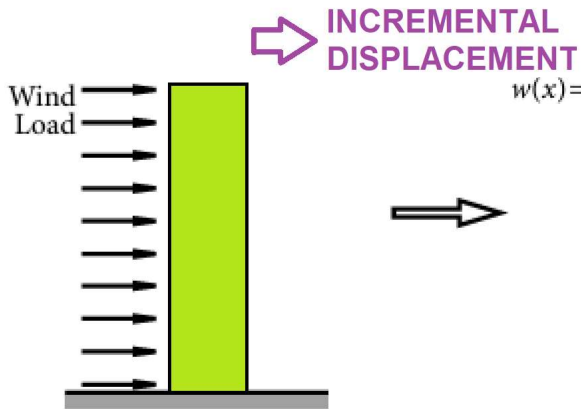
COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A CONCRETE COLUMN: AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS

THIS PROGRAM IS WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)

20
21
19
m
7

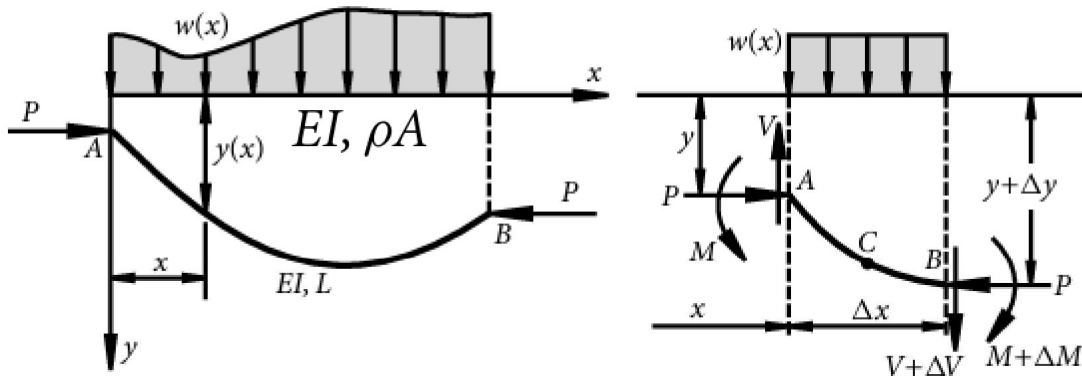
2
3
1
m
1

$$W(x, t) = P(t) = W_0 e^{-0.05 \bar{\omega} t} \sin(\bar{\omega} t)$$

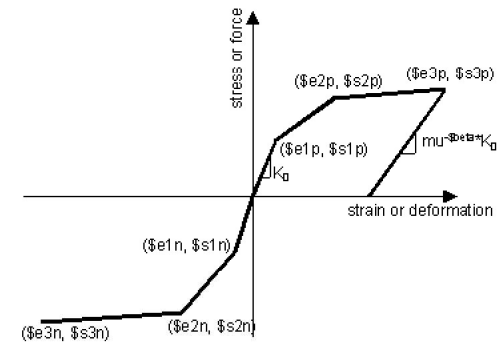


ELEMENT ELASTIC MATERIAL

$$W(x, t) = P(t) = W_0 e^{-0.05 \bar{\omega} t} \sin(\bar{\omega} t)$$



Beam-column.



ELEMENT INELASTIC MATERIAL

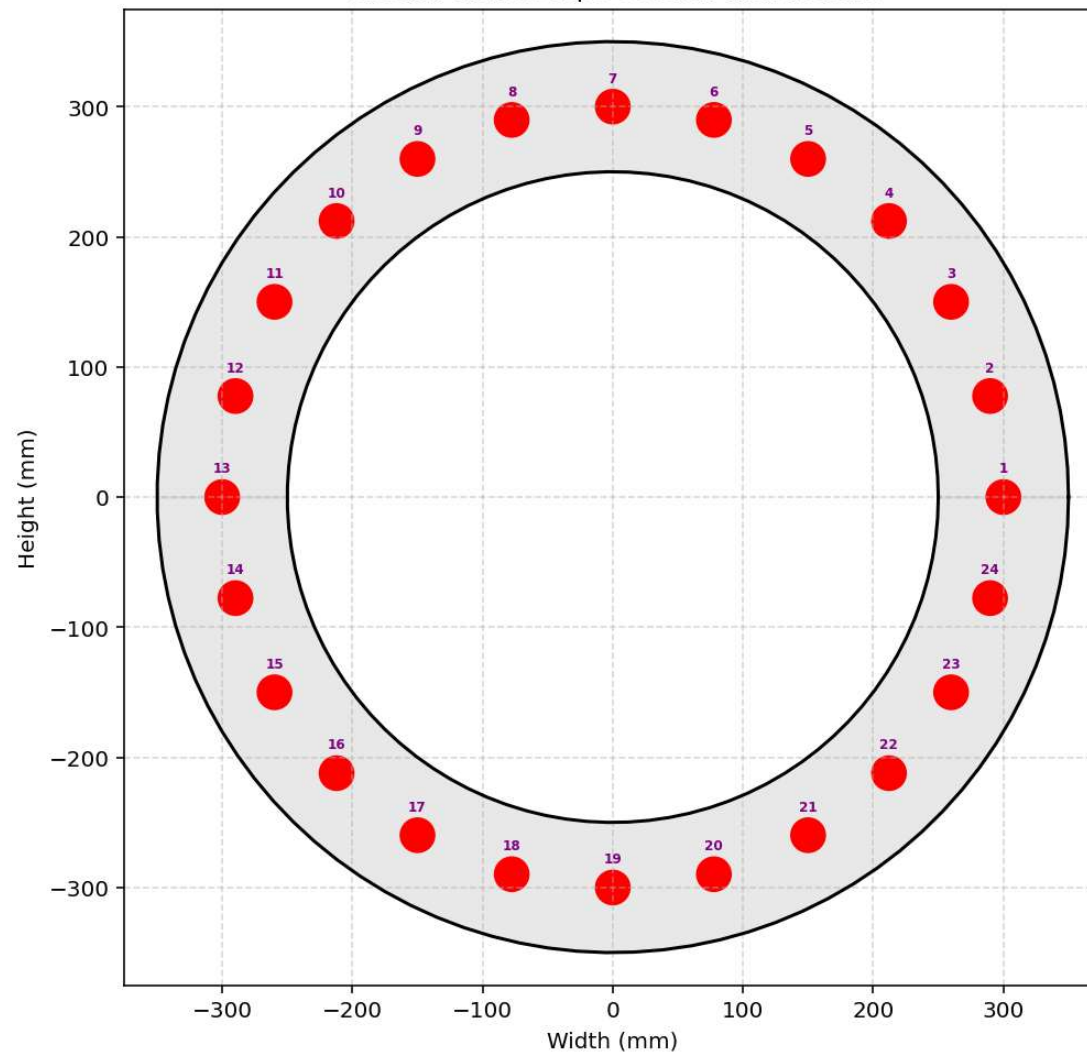
$$W(x,t) = P(t) = W_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

$$m\ddot{u} + c\dot{u} + ku = P(t)$$

$$\textit{Structure Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

$$\rho A \frac{\partial^2 v}{\partial t^2} + EI \frac{\partial^4 v}{\partial x^4} = w(x,t).$$

Circular Hollow Pipe Section with Rebars



C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...CONCRETE_CIRCULARR_HOLLOW_PIPE_SECTION_FUN_TWO.py

W(x,t)_CONCRETE_COLUMN_8_ANA.py × CONCRETE_CIRCULARR...SECTION_FUN_TWO.py ×

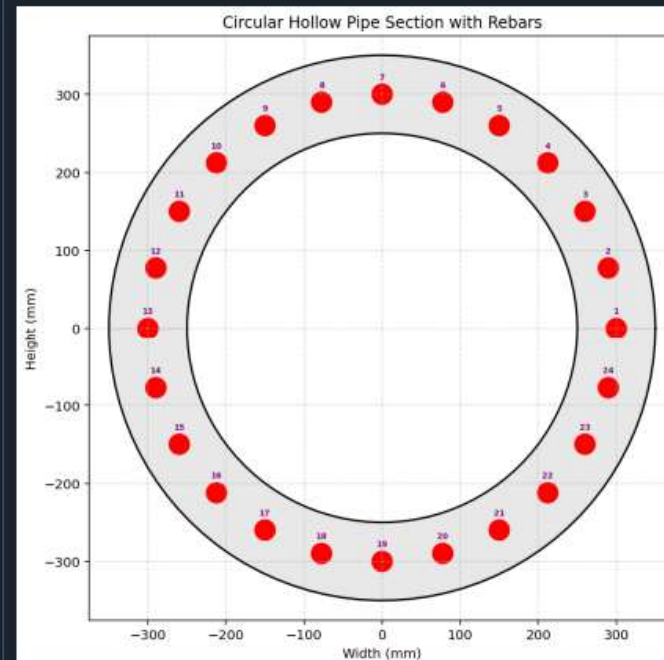
```

1 def CONCRETE_CIRCULAR_HOLLOW_PIPE_SECTION_FUN(
2     secTag, STEEL_TYPE, Do, t,
3     numSubdivCirc, numSubdivRad,
4     fc, Kfc,
5     CONCRETE_DENSITY,
6     nRebars=68,
7     rebar_dia=25,
8     plot=True
9 ):
10     """
11     Create a circular hollow confined-concrete fiber section for OpenSees,
12     with automatic placement of reinforcing bars along the mid-thickness circle.
13
14     Parameters
15     -----
16     secTag      : int - section identifier
17     STEEL_TYPE  : str - 'ELASTIC' or 'INELASTIC'
18     Do          : float - outer diameter (mm)
19     t           : float - wall thickness (mm)
20     numSubdivCirc : int - number of circumferential subdivisions for concrete patch
21     numSubdivRad  : int - number of radial subdivisions for concrete patch
22     fc          : float - unconfined concrete compressive strength (MPa, negative)
23     Kfc         : float - ratio of confined to unconfined concrete strength
24     CONCRETE_DENSITY: float - density of concrete (kg/m³) - converted to N·s²/mm³ internally
25     plot        : bool - if True, draw a 2D sketch of the section
26     nRebars     : int - total number of reinforcing bars (default = 68)
27     rebar_dia   : float - diameter of each rebar (mm), uniform for all bars (default = 25)
28
29     Returns
30     -----
31     Do          : float - outer diameter (mm)
32     ELE_MASS    : float - element mass per unit length (kg/mm)
33

```

C:\Users\Dell\Desktop\OPENSEES_FILES

42 %



IPython Console Plots Variable Explorer Help Debugger History Files

File Edit Search Source Run Debug Consoles Projects Tools View Help

W(x,t)_CONCRETE_COLUMN_8_ANA.py

CONCRETE_CIRCULAR...SECTION_FUN_TWO.py

1 #####

2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A CONCRETE COLUMN : AN OPENSEES

4 # FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HIST

5 # -----

6 # EQUIVALENT VISCOUS DAMPING RATIO: $\xi_{eq} = 100 * E_d / (4 * \pi * E_s)$

7 # -----

8 # ENERGY DISSIPATION CAPACITY INDEX = $100 * E_d(\text{earthquake}) / E_d(\text{cyclic displacement})$

9 # -----

10 # $W(x,t) = W_0 \sin(wt)$

11 # $W(x,t) = W_0 \exp(-0.05wt) \sin(wt)$

12 # -----

13 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND EVAL

14 # OF ENERGY DISSIPATION CAPACITY INDEX

15 # -----

16 # THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)

17 # EMAIL: salar.d.ghashghaei@gmail.com

18 #####

19 """

20 Nonlinear Seismic Performance Assessment of a concrete column:

21 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and Ear

22

23 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a concrete

24 for performance-based earthquake engineering. The 2D model incorporates both material nonlin

25 (elastic-perfectly plastic or hysteretic steel with strain hardening)

26 and geometric nonlinearity to capture P-delta effects

27 and large displacements—essential for collapse assessment.

28

29 Eight analysis protocols are implemented:

30 (1) [PERIOD] : Structural Period

31 (2) [STATIC] : Gravity load analysis establishing dead load state

32 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves

33 and plastic mechanism identification

34

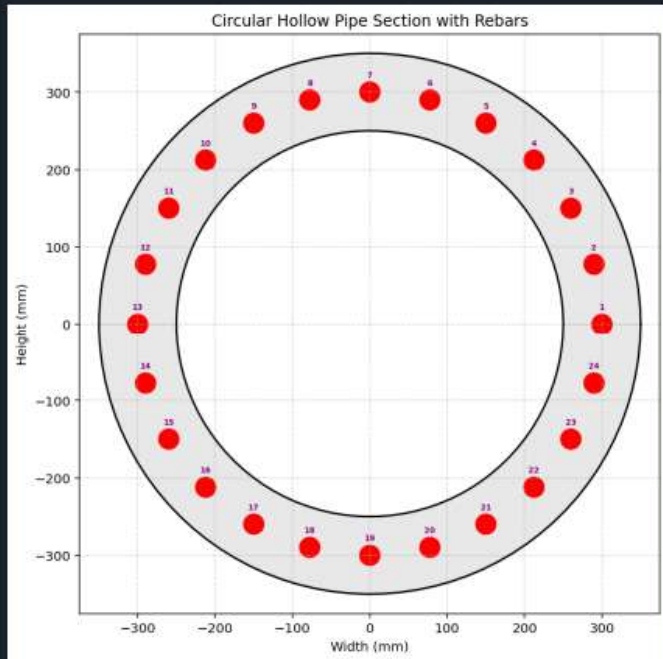
42 %

+

-

...

Circular Hollow Pipe Section with Rebars



IPython Console Plots Variable Explorer Help Debugger History Files

Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 112, Col 32 UTF-8 CRLF RW Mem 51%

STATIC ANALYSIS

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

W(x,t)_CONCRETE_COLUMN_8_ANA.py x CONCRETE_CIRCULARR...SECTION_FUN_TWO.py x

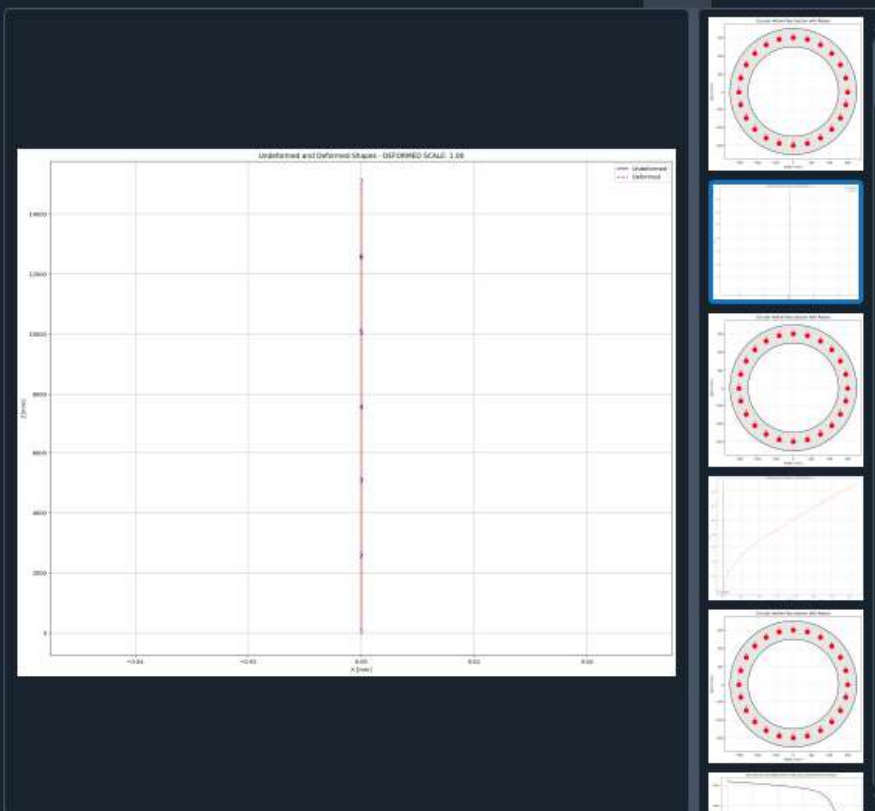
```

386
387 if ANAL_TYPE == 'STATIC':
388     ops.integrator('LoadControl', 1.0)
389     ops.analysis('Static')
390     OK = ops.analyze(1)
391     S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
392     ops.reactions()
393     reactionX.append(ops.nodeReaction(1, 1))
394     reactionY.append(ops.nodeReaction(1, 2))
395     reactionZ.append(ops.nodeReaction(1, 3))
396     disp_mid.append(ops.nodeDisp(center_node, 1))
397     dispX.append(ops.nodeDisp(center_node, 1))
398     dispY.append(ops.nodeDisp(center_node, 2))
399     # Store axial forces, strain and stress
400     for ele_id in ele_axialforce.keys():
401         ele_axialforce[ele_id].append(ops.eleResponse(ele_id, 'force')[0])
402         ele_shearforce[ele_id].append(ops.eleResponse(ele_id, 'force')[1])
403         ele_momentforce[ele_id].append(ops.eleResponse(ele_id, 'force')[2])
404     # Store displacements
405     for node_id in node_displacementsX.keys():
406         node_displacementsX[node_id].append(ops.nodeDisp(node_id, 1))
407         node_displacementsY[node_id].append(ops.nodeDisp(node_id, 2))
408     else:
409         print('\n\nSTATIC ANALYSIS DONE.\n\n')
410
411 DATA = (reactionX, reactionY, reactionZ, disp_mid,
412         ele_axialforce, ele_shearforce, ele_momentforce,
413         node_displacementsX, node_displacementsY,
414         dispX, dispY)
415
416 return DATA
417
418 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
419     INCR1DNE = 1 # 1: Horizontal Displacement - 2: Vertical Displacement

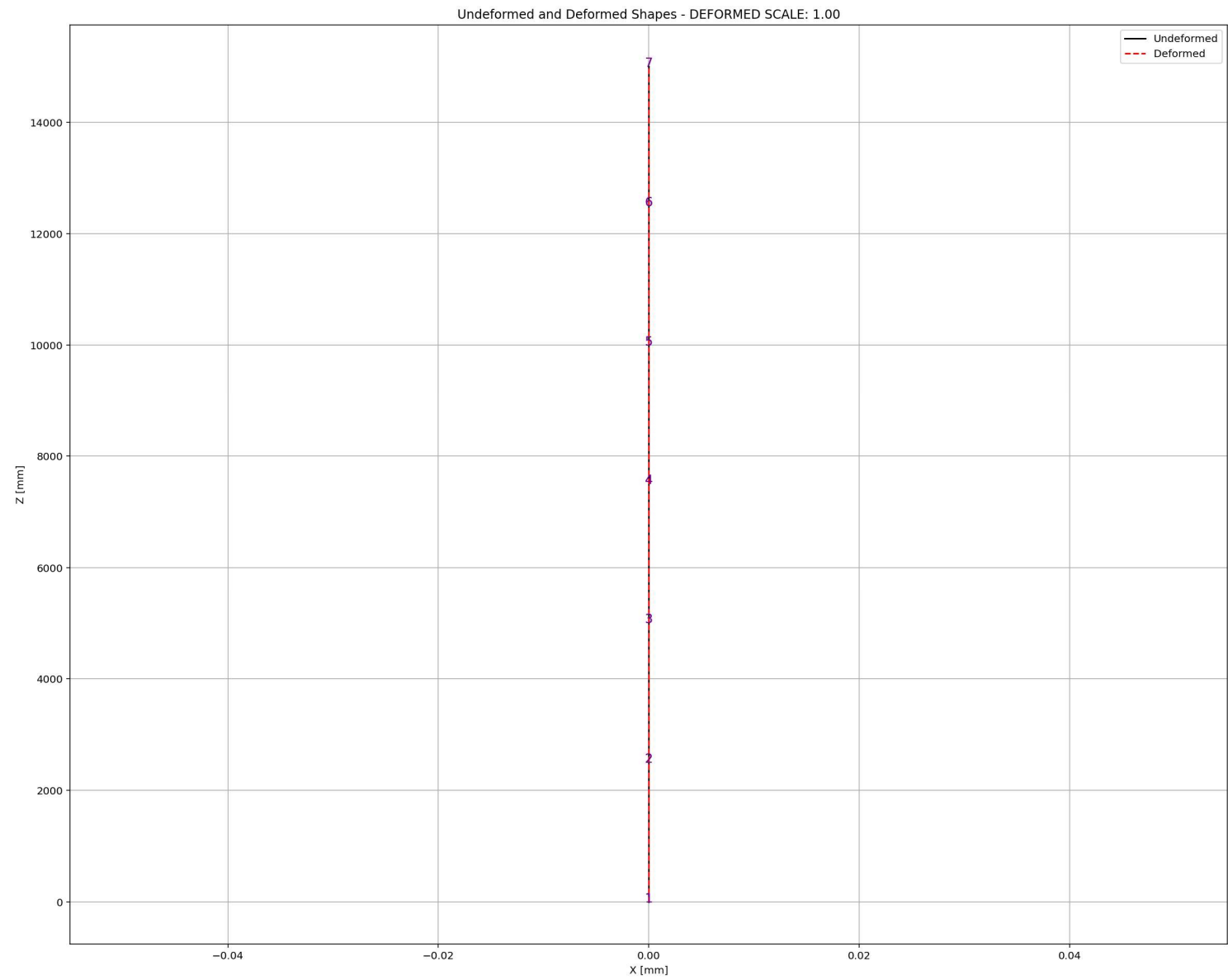
```

...NERGY DISSIPATION CAPACITY INDEX\14_CONCRETE_COLUMN_8_ANA

18 %



IPython Console Plots Variable Explorer Help Debugger History Files



PUSHOVER ANALYSIS

C:\Users\Del\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

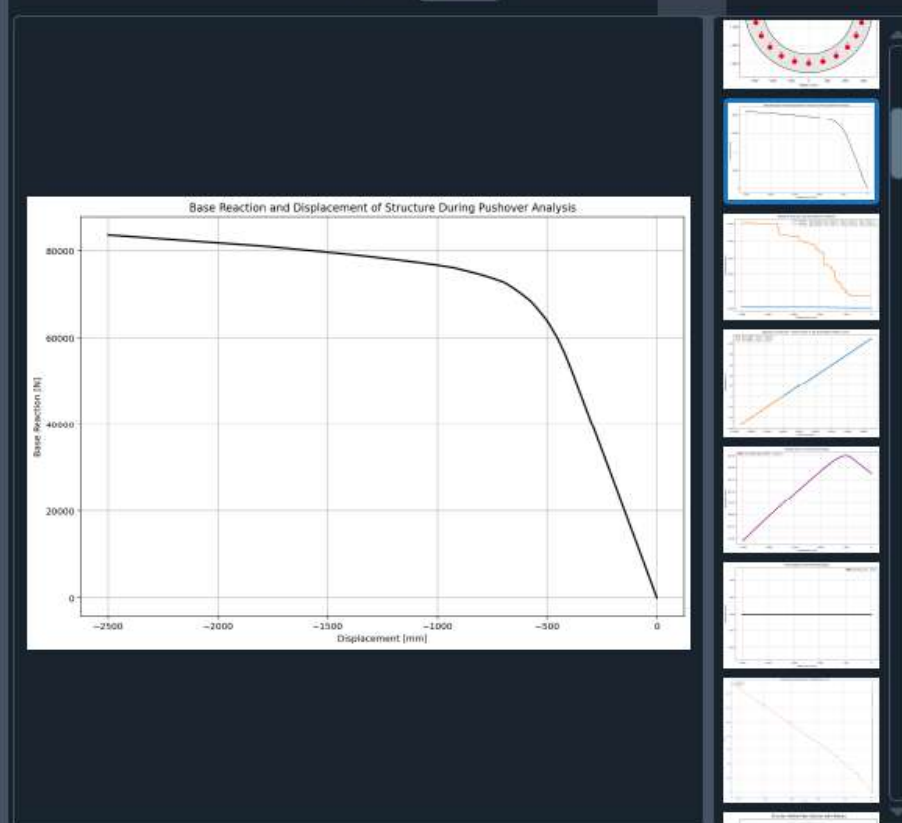
W(x,t)_CONCRETE_COLUMN_8_ANA.py X CONCRETE_CIRCULAR...SECTION_FUN_TWO.py X

```

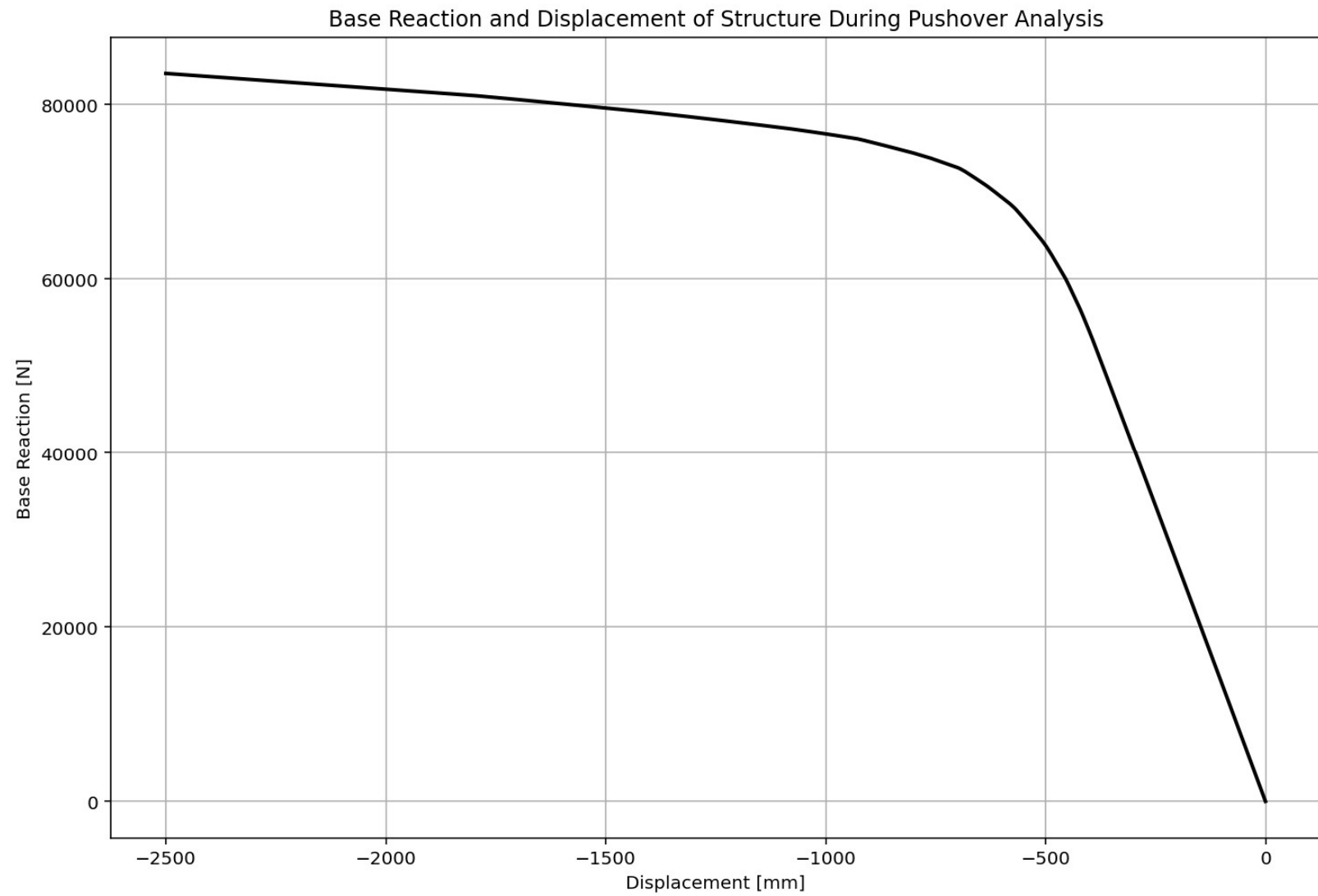
418 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
419     IDctrlDOF = 1 # 1: Horizontal Displcement - 2: Vertical Displcement
420     DINCR = -0.1 # [mm] Incremental Displacement
421     DMAX = -2500.0 # [mm] Max. Displacement
422     ops.integrator('DisplacementControl', center_node, IDctrlDOF, DINCR)
423     ops.analysis('Static')
424     Nsteps = int(np.abs(DMAX / DINCR))
425     STEP = 0.0
426     for step in range(Nsteps):
427         OK = ops.analyze(1)
428         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
429         ops.reactions()
430         reactionX.append(ops.nodeReaction(1, 1)) # SHEAR BASE REAC
431         reactionY.append(ops.nodeReaction(1, 2)) # AXIAL BASE REAC
432         reactionZ.append(ops.nodeReaction(1, 3)) # MOMENT BASE REA
433         disp_mid.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT NO
434         dispX.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT NO
435         dispY.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT NO
436         # Store elements force, strain and stress
437         for ele_id in ele_axialforce.keys():
438             ele_axialforce[ele_id].append(ops.eleResponse(ele_id, 'force')[0]) # [
439             ele_shearforce[ele_id].append(ops.eleResponse(ele_id, 'force')[1]) # [
440             ele_momentforce[ele_id].append(ops.eleResponse(ele_id, 'force')[2]) # [
441         # Store displacements
442         for node_id in node_displacementsX.keys():
443             node_displacementsX[node_id].append(ops.nodeDisp(node_id, 1))
444             node_displacementsY[node_id].append(ops.nodeDisp(node_id, 2))
445         # IN EACH STEP, STRUCTURE PERIOD GOING TO BE CALCULATED
446         #PERIODmin, PERIODmax = S06.RAYLEIGH_DAMPING(3, 0.5*DR, DR, 0, 1)
447         PERIODmin, PERIODmax = S05.EIGENVALUE_ANALYSIS(3, PLOT=True)
448         PERIOD_MIN.append(PERIODmin)
449         PERIOD_MAX.append(PERIODmax)
450         if ELE_TYPE == 'nonLinearBeamColumn':
451             # SECTION STRESS-STRAIN FOR BOTTOM STRUT

```

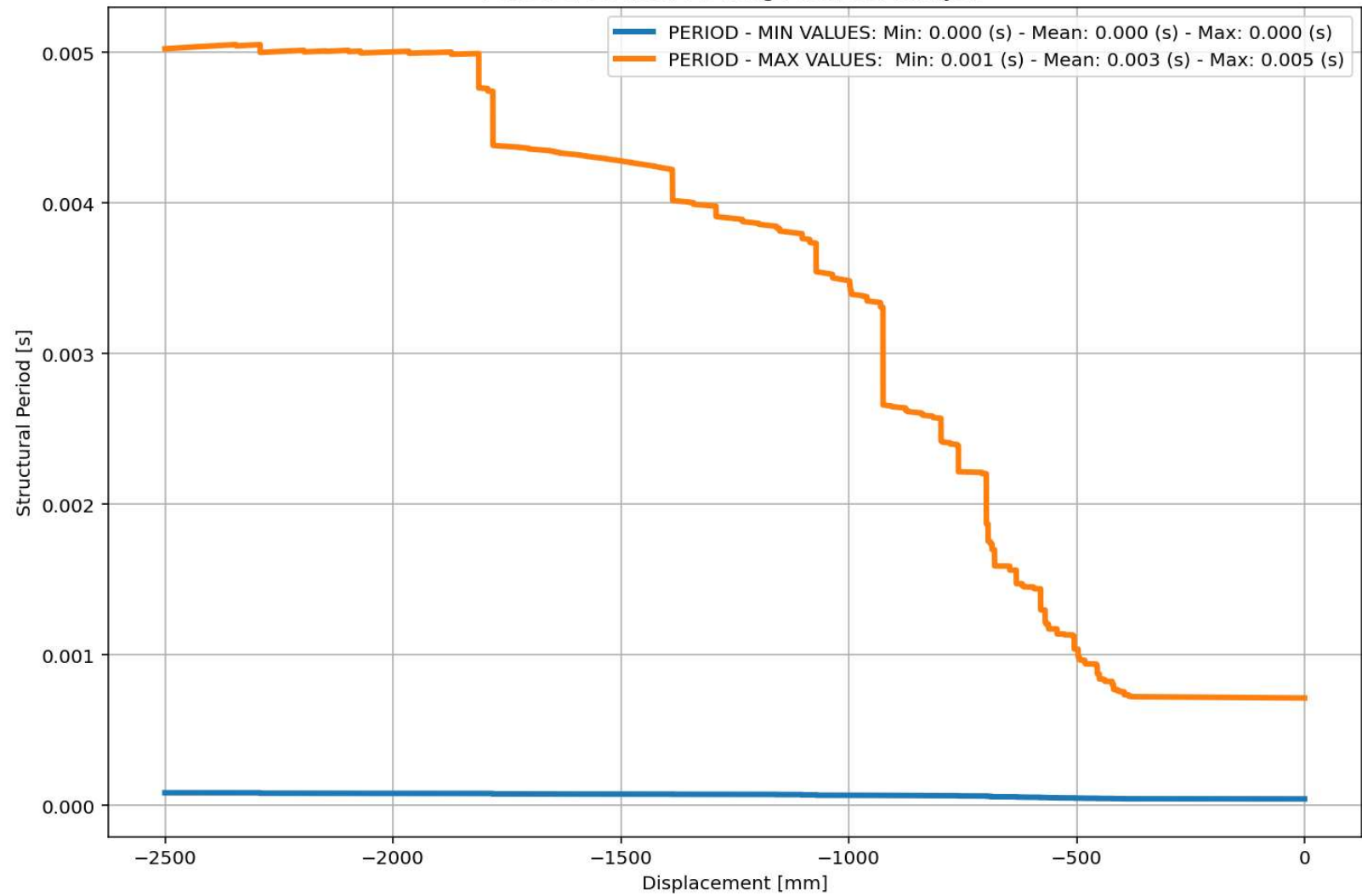
29 %



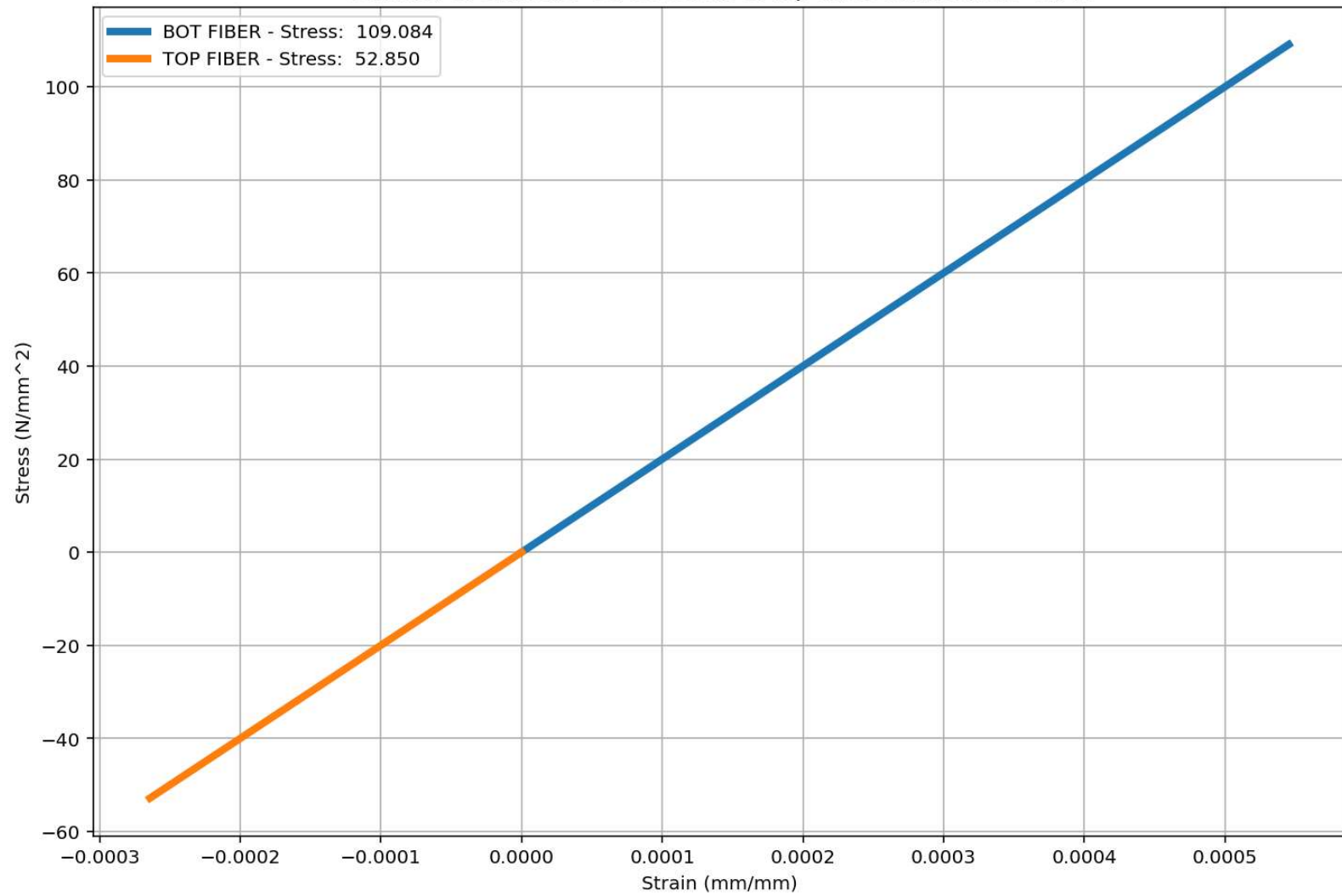
IPython Console Plots Variable Explorer Help Debugger History Files

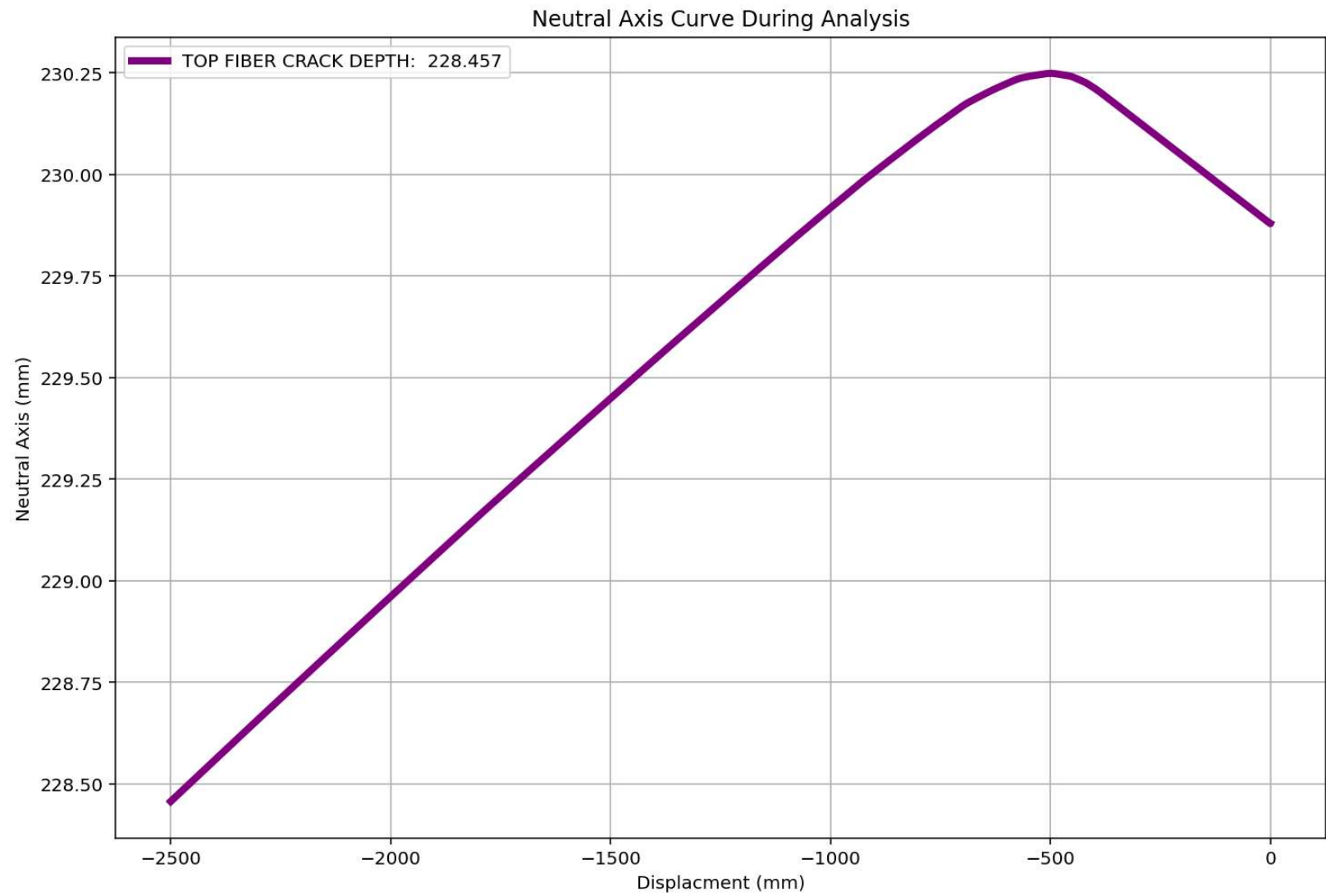


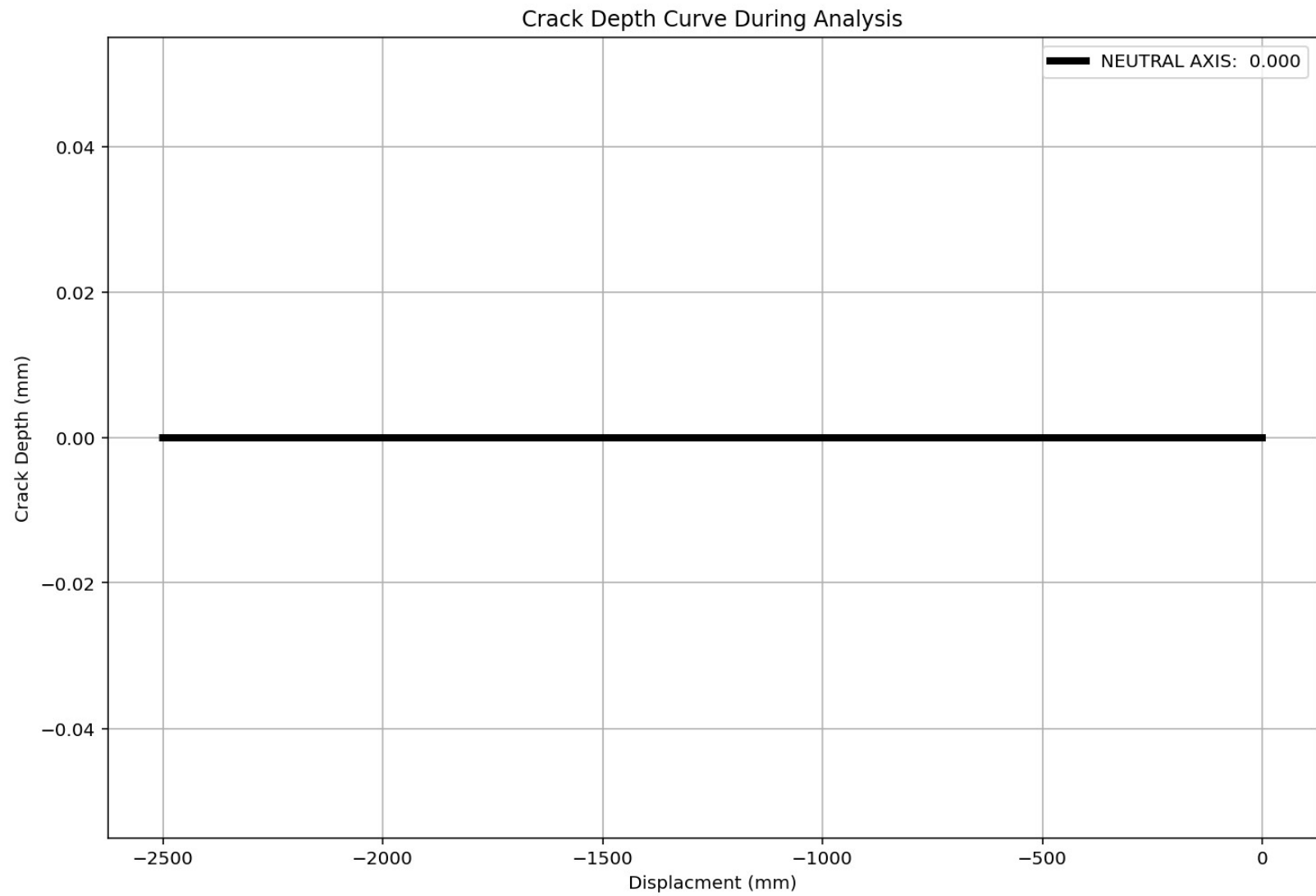
Period of Structure During Pushover Analysis

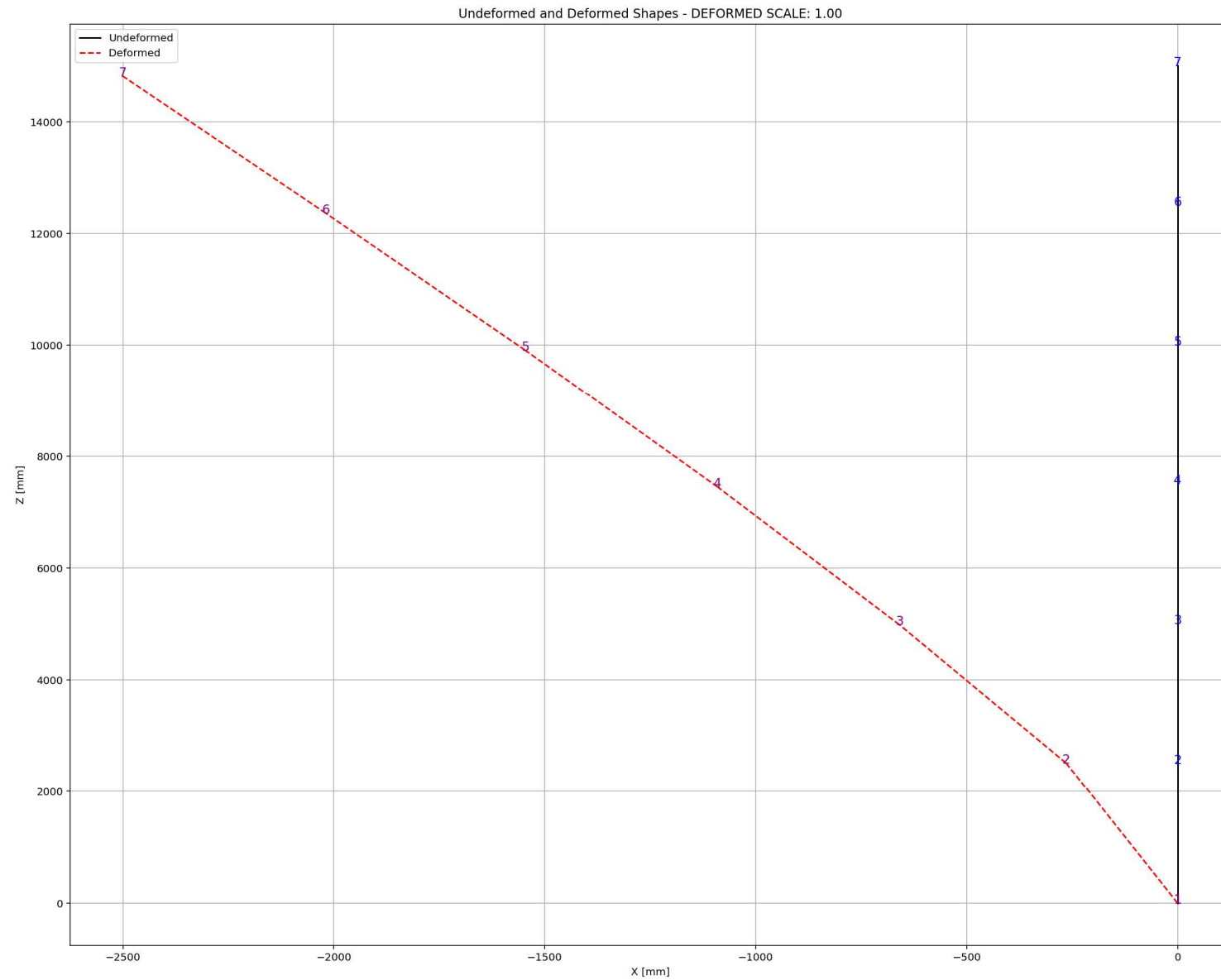


Behavior of element - Stress-strain of Top and Bottom Fibers Curve









CYCLIC DISPLACEMENT ANALYSIS

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

W(x,t)_CONCRETE_COLUMN_8_ANA.py X CONCRETE_CIRCULARR...SECTION_FUN_TWO.py X

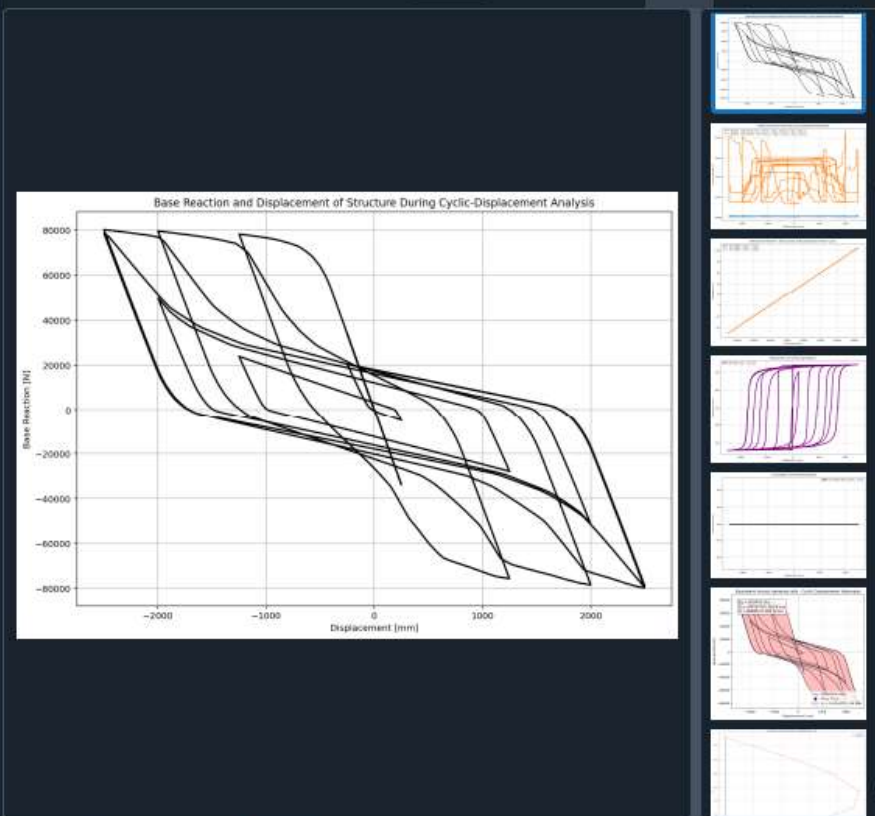
```

491
492 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
493     IDctrlDOF = 1 # 1: Horizontal Dispalcement - 2: Vertical Dispalcement
494     DMAX = -2500.0 # [mm] Max. Displacement
495     n_points = 10000
496     # 4. CYCLIC DISPALCEMENT PROTOCOL
497     # Key strain points (same logic as your protocol)
498     key_disp = np.array([
499         0.0,
500         0.1*DMAX, -0.1*DMAX,
501         0.5*DMAX, -0.5*DMAX,
502         0.8*DMAX, -0.8*DMAX,
503         DMAX, -DMAX,
504         0.1*DMAX, -0.1*DMAX,
505         0.5*DMAX, -0.5*DMAX,
506         0.8*DMAX, -0.8*DMAX,
507         DMAX, -DMAX,
508         0.0
509     ])
510
511     # Generate 1000-point displacement protocol
512     disp_protocol = np.interp(
513         np.linspace(0, len(key_disp) - 1, n_points),
514         np.arange(len(key_disp)),
515         key_disp
516     )
517
518     ops.analysis('Static')
519     STEP = 0.0
520     for target_disp in disp_protocol:
521         current_disp = ops.nodeDisp(center_node, 1) # DISPALCEMENT APPLIED IN NODE IN X D
522         dU = target_disp - current_disp
523         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
524         ops.analyze(1)

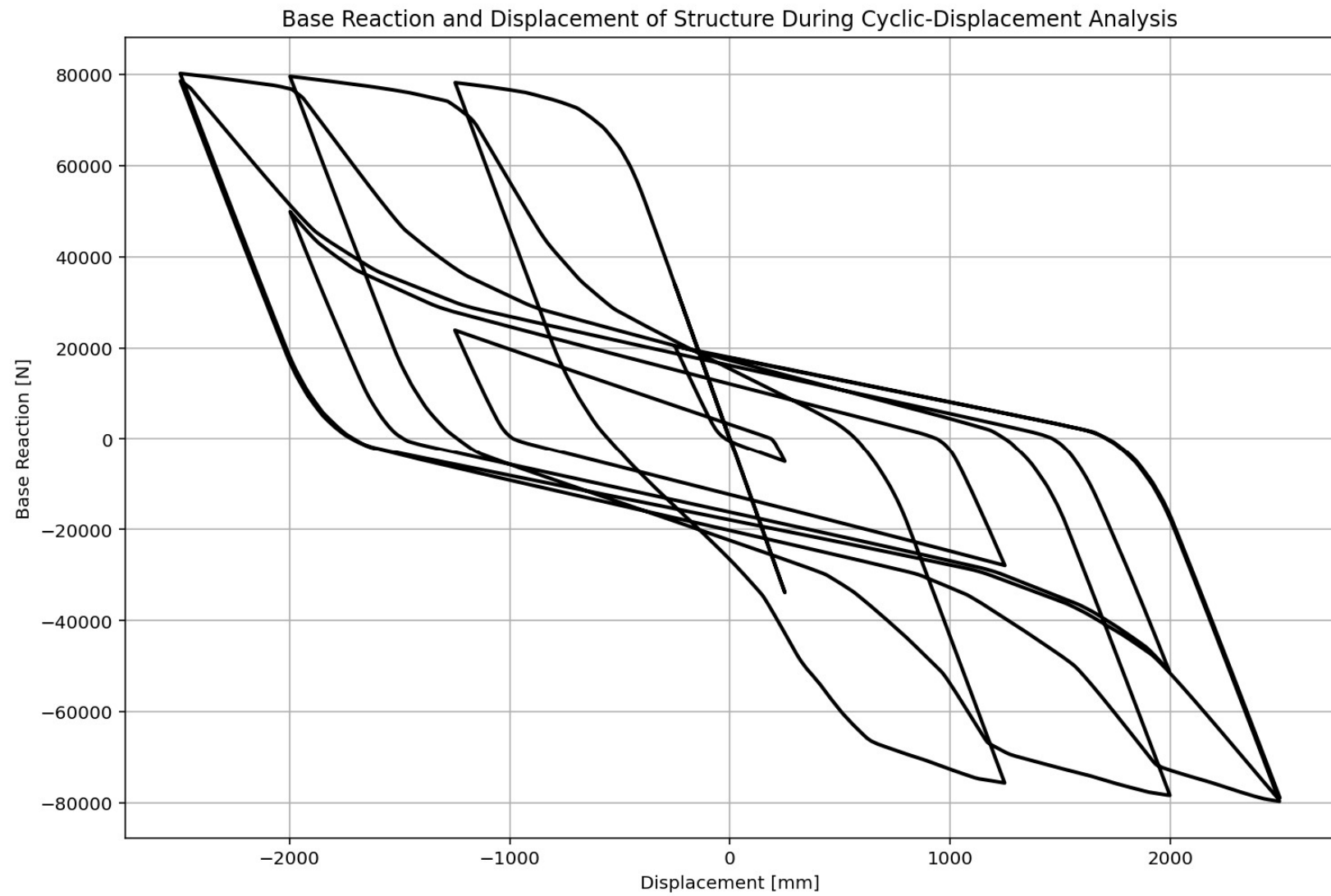
```

...NERGY DISSIPATION_CAPACITY_INDEX\14_CONCRETE_COLUMN_8_ANA

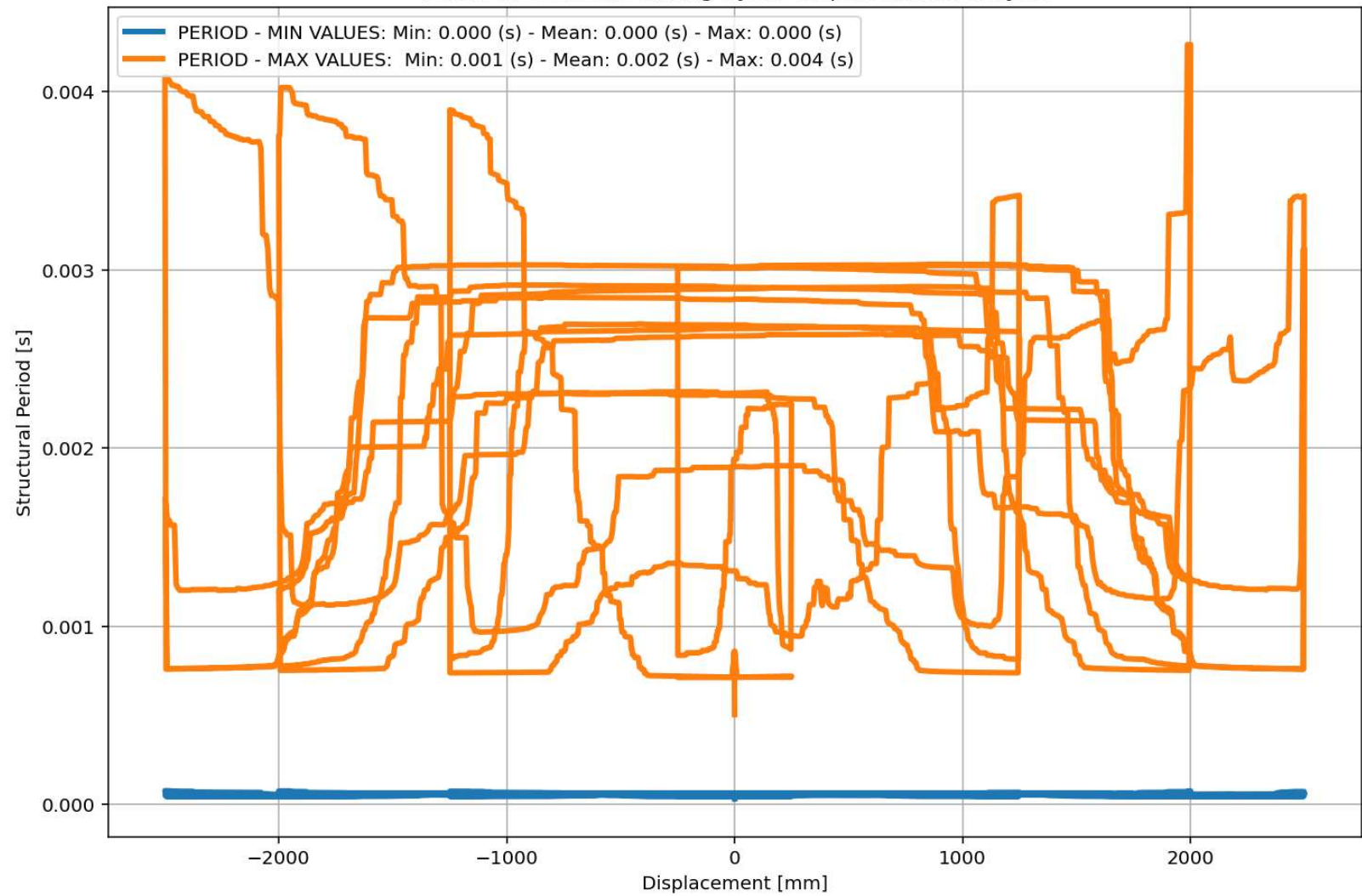
29 %



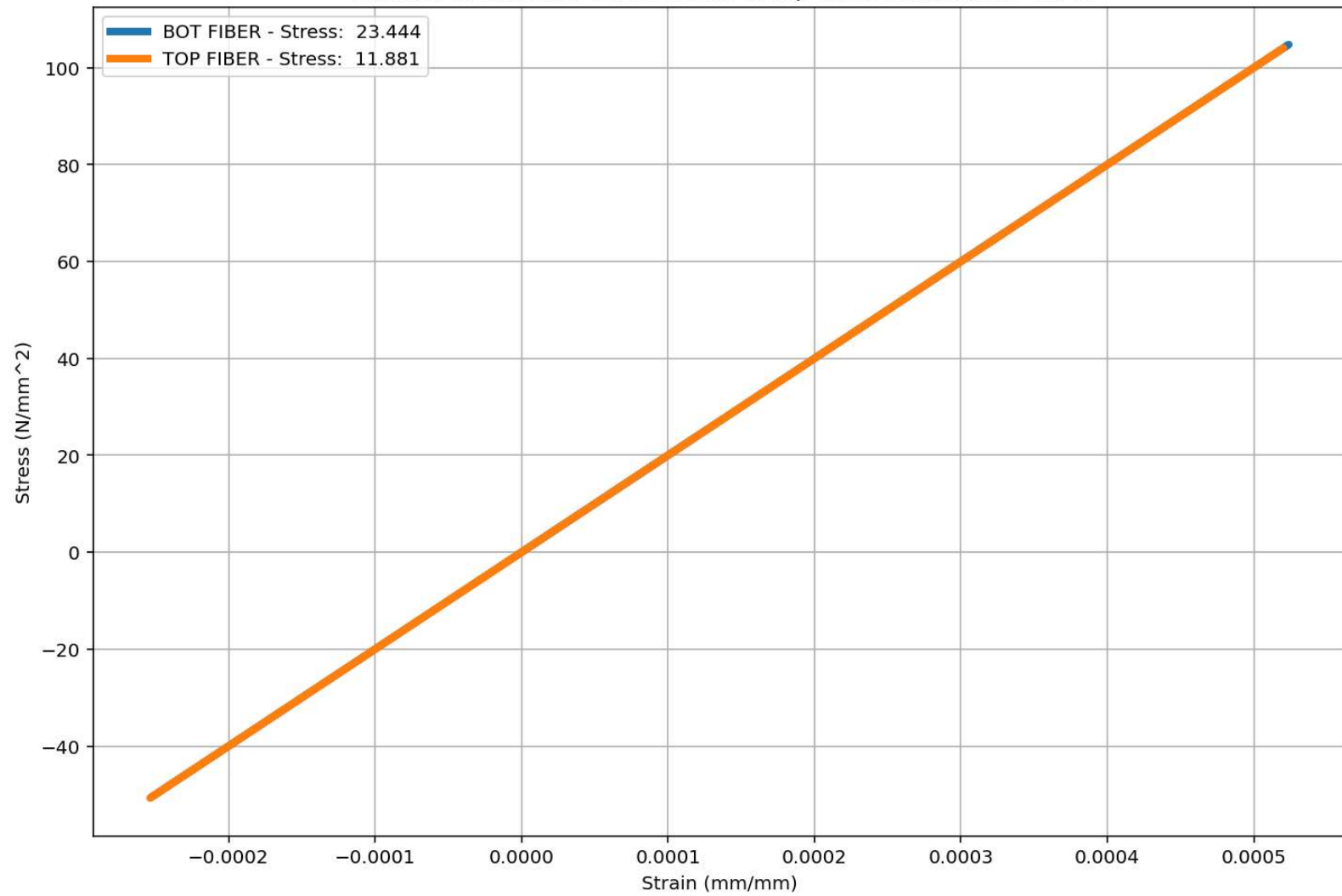
IPython Console Plots Variable Explorer Help Debugger History Files

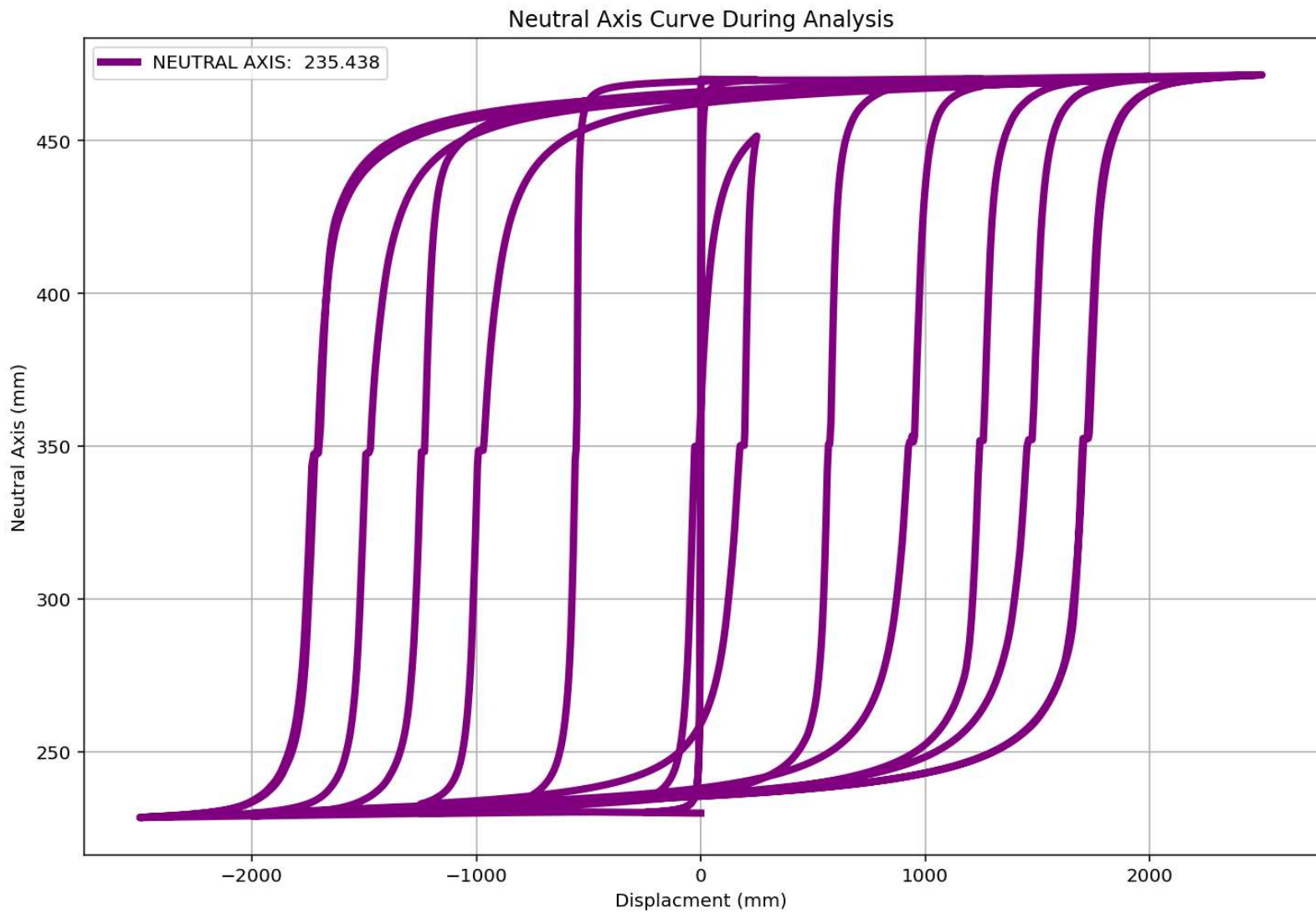


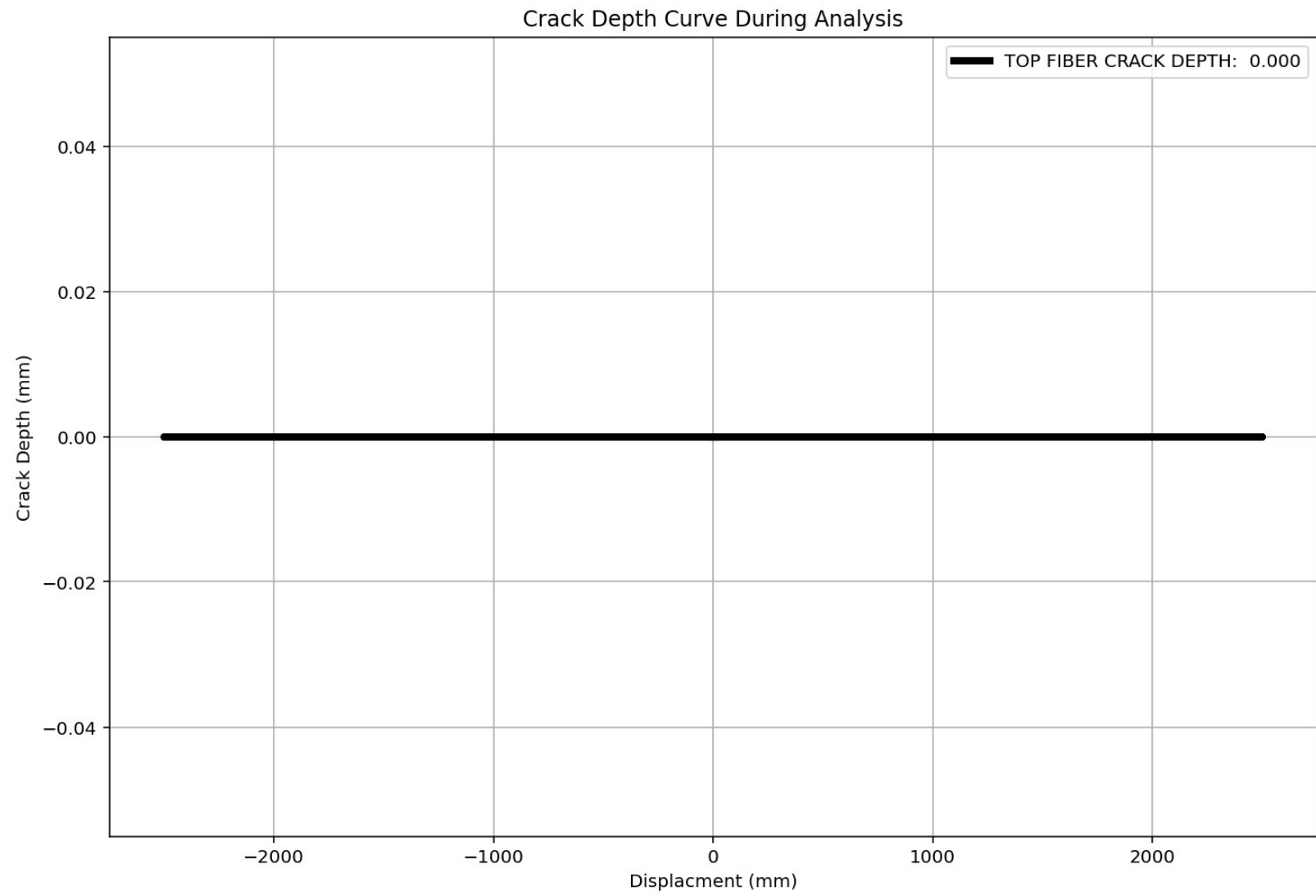
Period of Structure During Cyclic Displacement Analysis



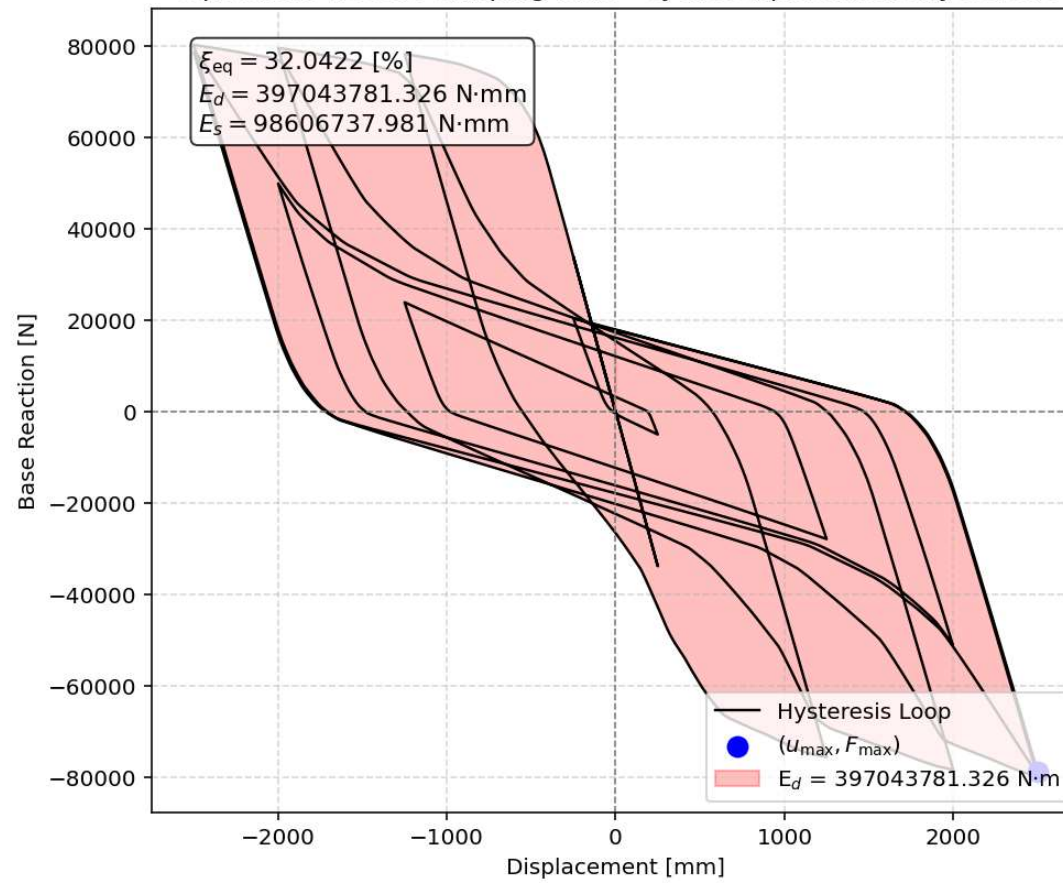
Behavior of element - Stress-strain of Top and Bottom Fibers Curve

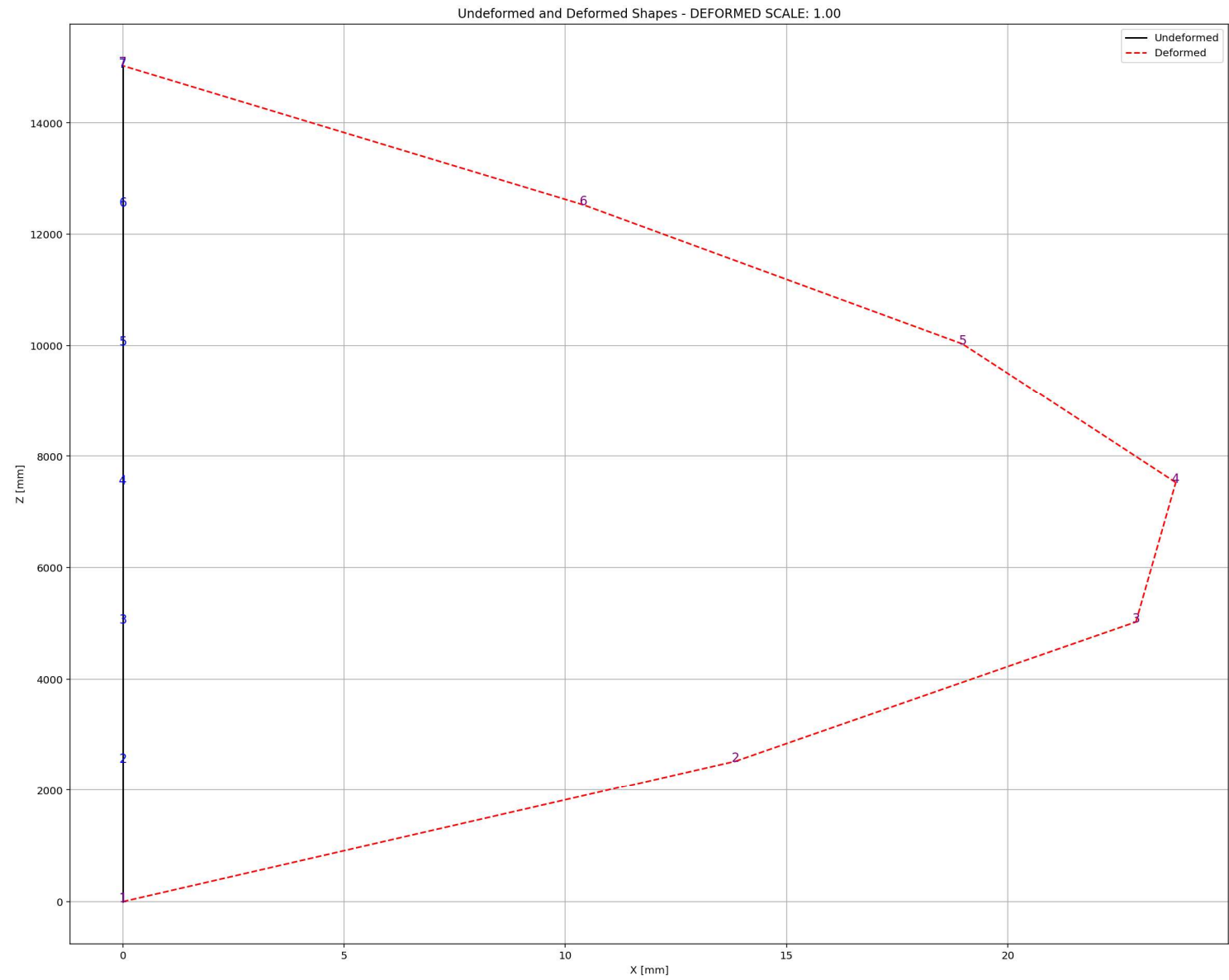






Equivalent viscous damping ratio - Cyclic Displacement Hysteresis





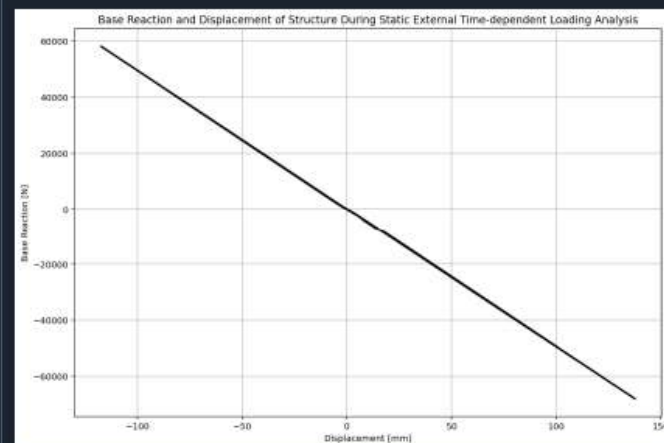
STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

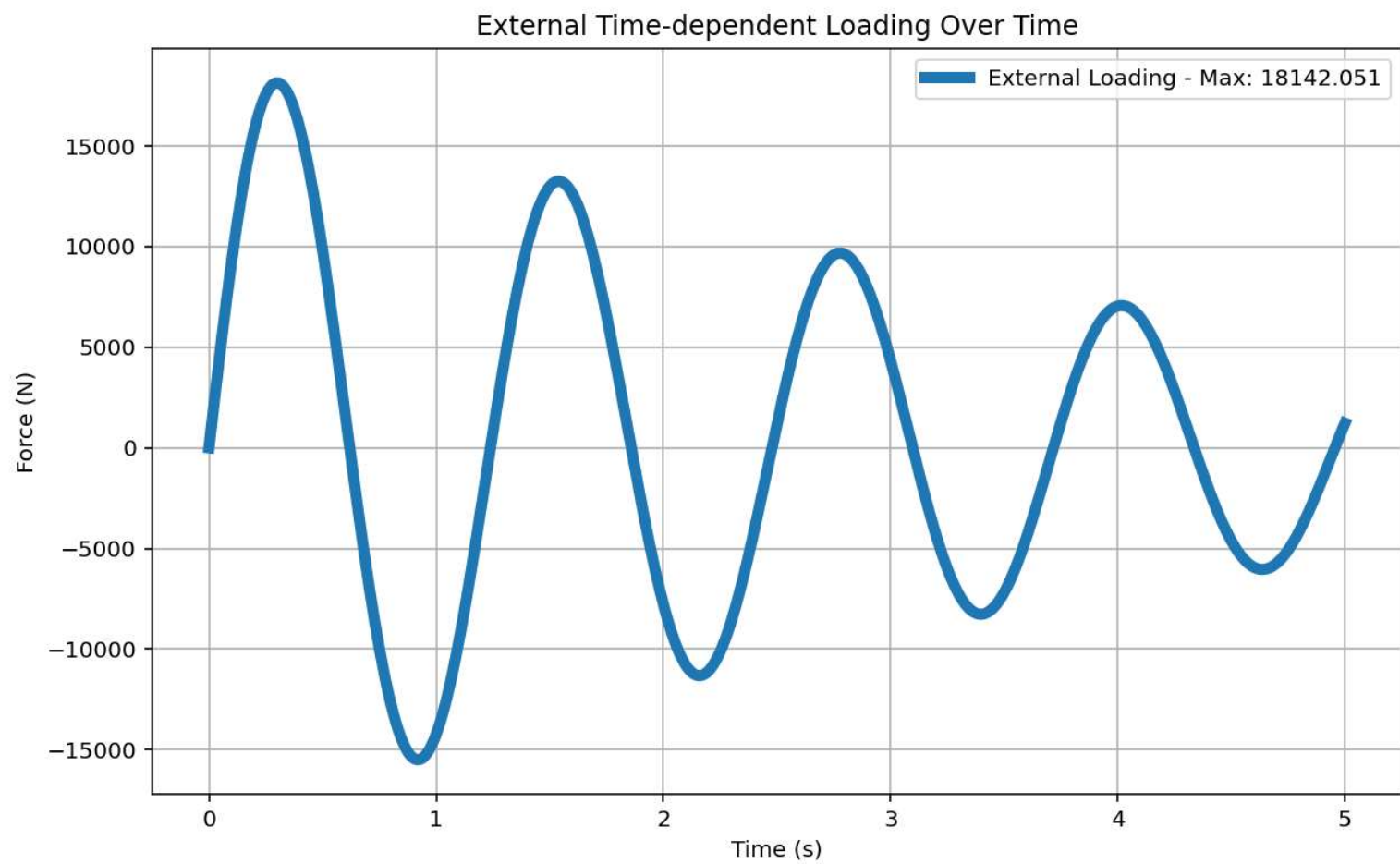
W(x,t)_CONCRETE_COLUMN_8_ANA.py X CONCRETE_CIRCULAR...SECTION_FUN_TWO.py X

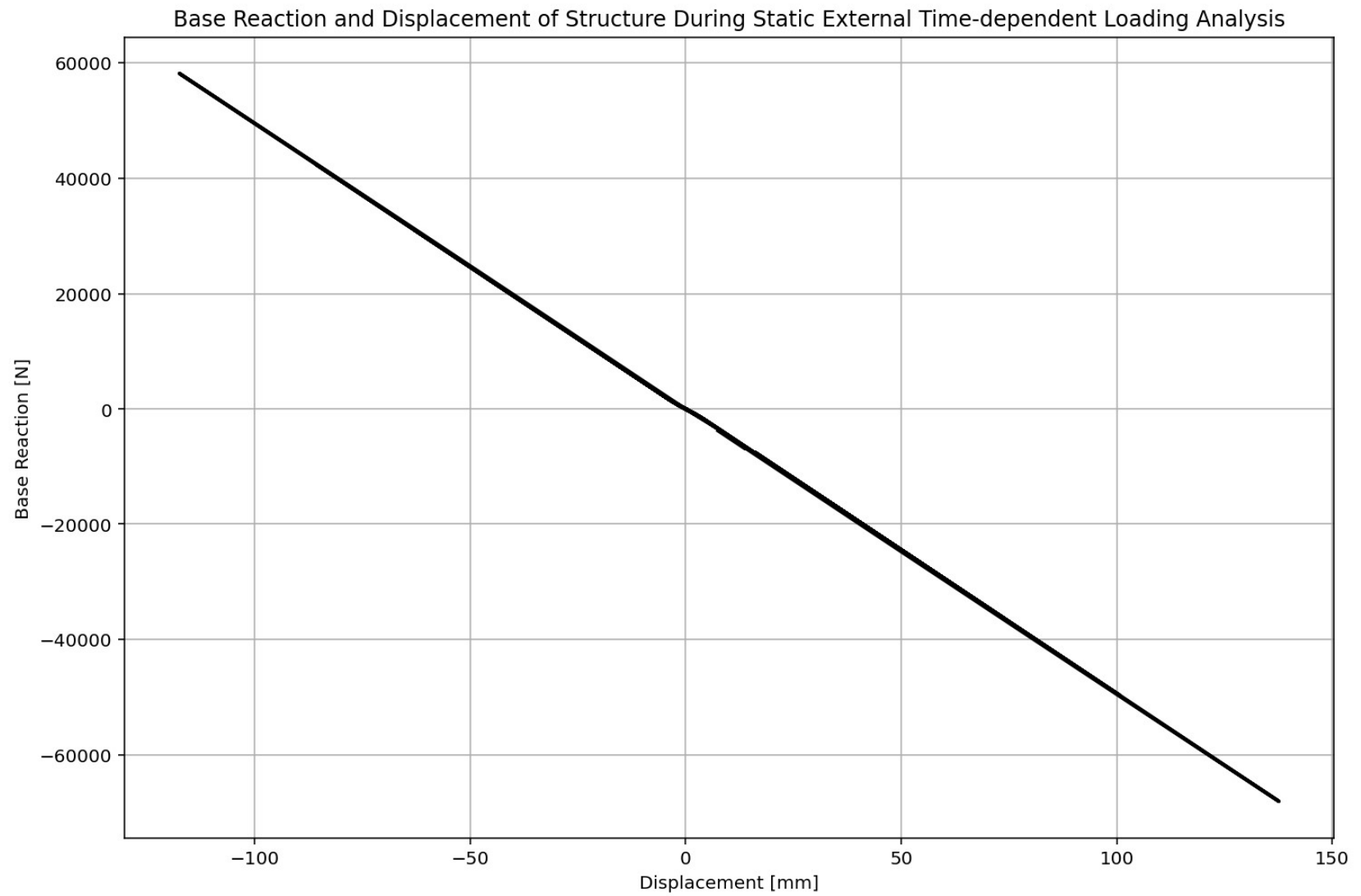
```
588
589 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYSIS
590     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
591     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
592     duration = 20.0 # [s] Analysis duration
593     dt = 0.01 # [s] Time step
594     DT = dt # [s] Time step
595     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
596     force_amplitude = 19600.0 # [N] Amplitude Force
597     omega_DT = 5.0715 # [rad/s] Natural angular frequency
598     time_steps = int(duration/dt)
599
600 # Check function
601 def CHECK_FUN(DT_time ,duration):
602     if DT_time > duration:
603         print('\n\nAnalysis Duration Must be greater than External Load Duration\n\n')
604         exit()
605     return -1
606
607 CHECK_FUN(DT_time ,duration)
608 def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): #  $P(t) = P_0 \sin(\omega t)$ 
609     import numpy as np
610     import matplotlib.pyplot as plt
611     # External Load Durations
612     num_steps = int(DT_time / DT)
613     load_time = np.linspace(0, DT_time, num_steps)
614     target_frequency = 1.0 * omega_DT # Target excitation frequency
615     DT_load = force_amplitude * np.sin(target_frequency * load_time)
616     # Plot External Time-dependent Loading
617     plt.figure(figsize=(10, 6))
618     plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load):.3f}')
619     plt.title('External Time-dependent Loading Over Time')
620     plt.xlabel('Time (s)')
621     plt.ylabel('Force (N)')
```

28 %

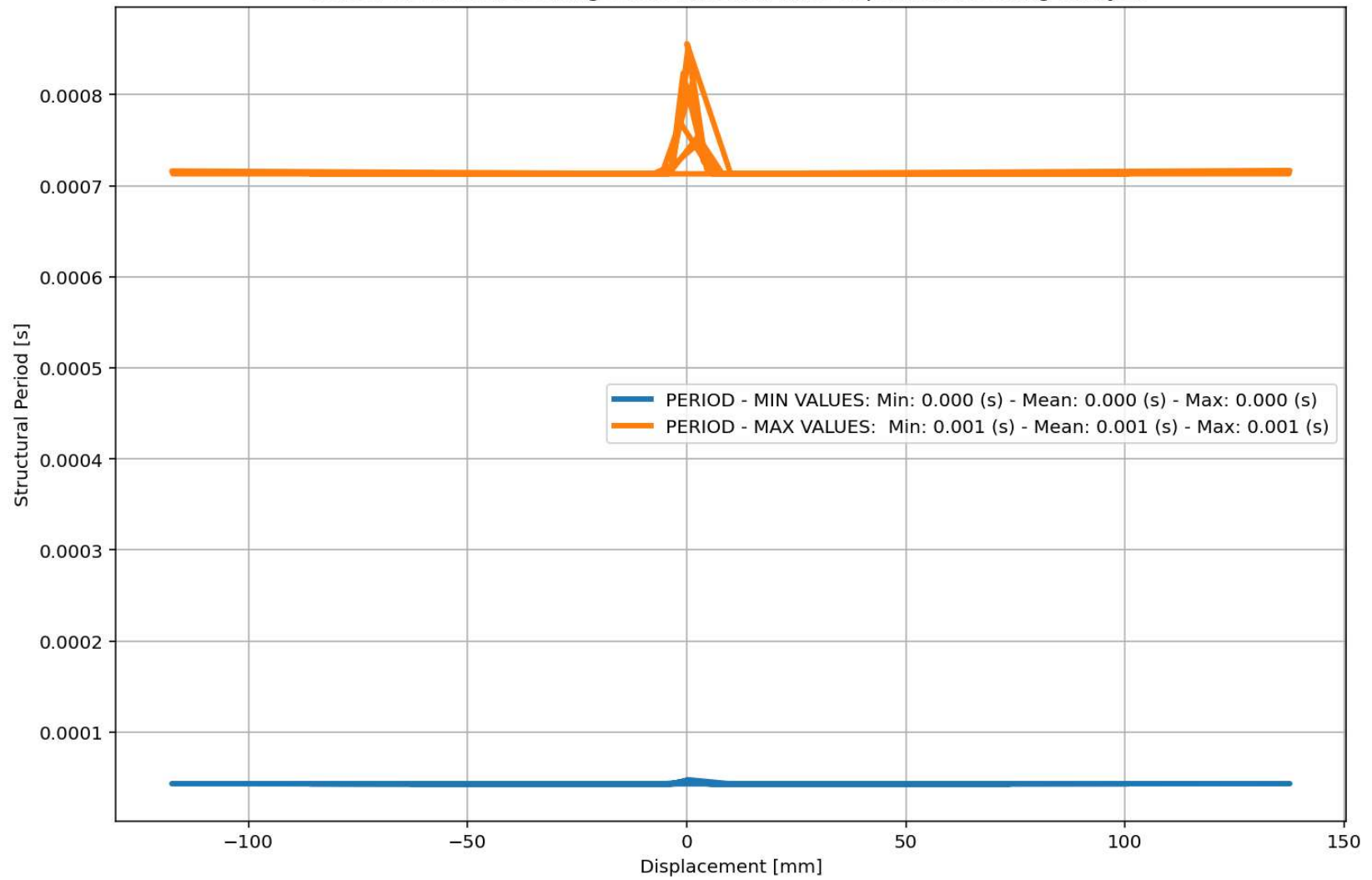


IPython Console Plots Variable Explorer Help Debugger History Files

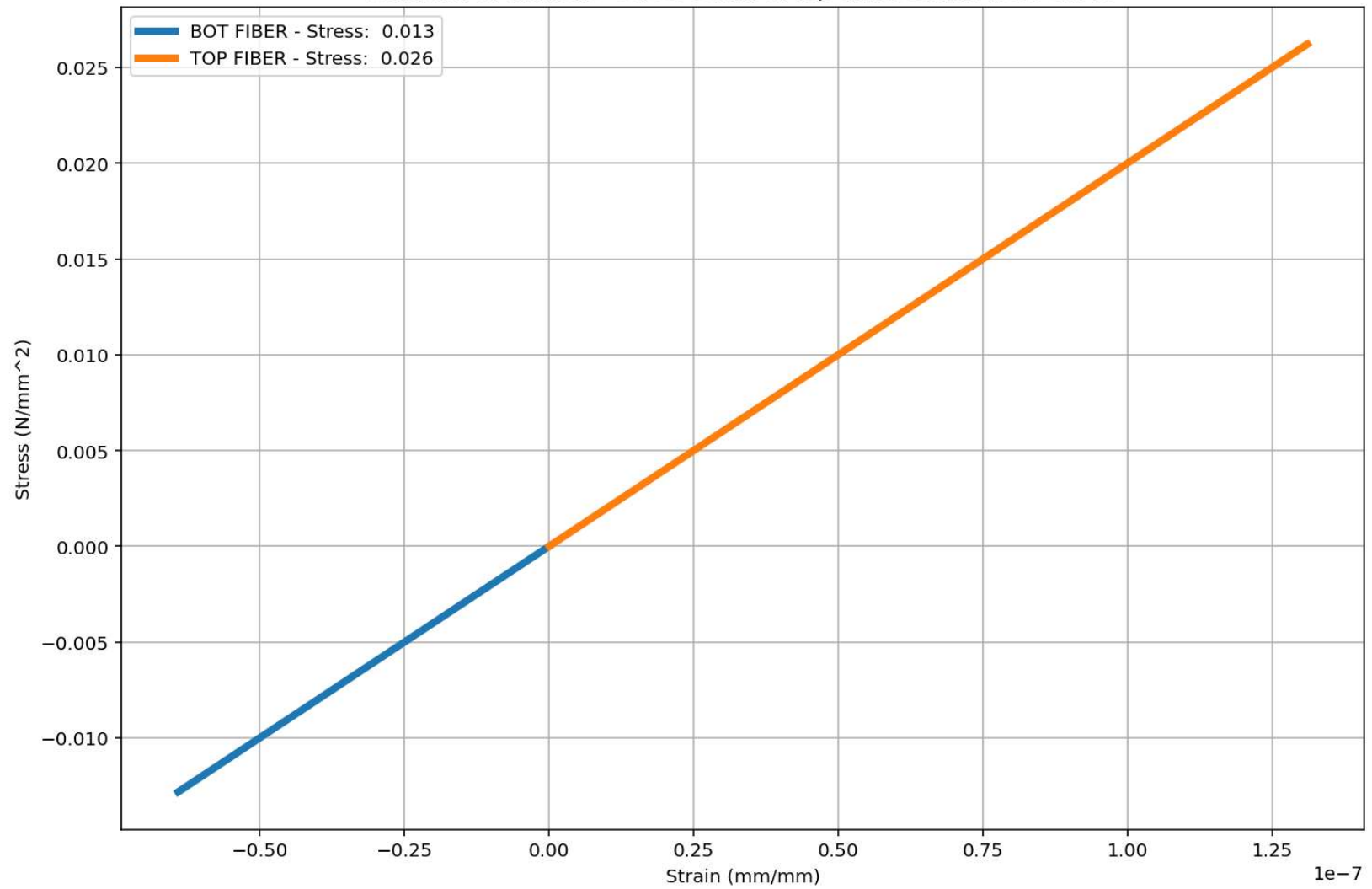


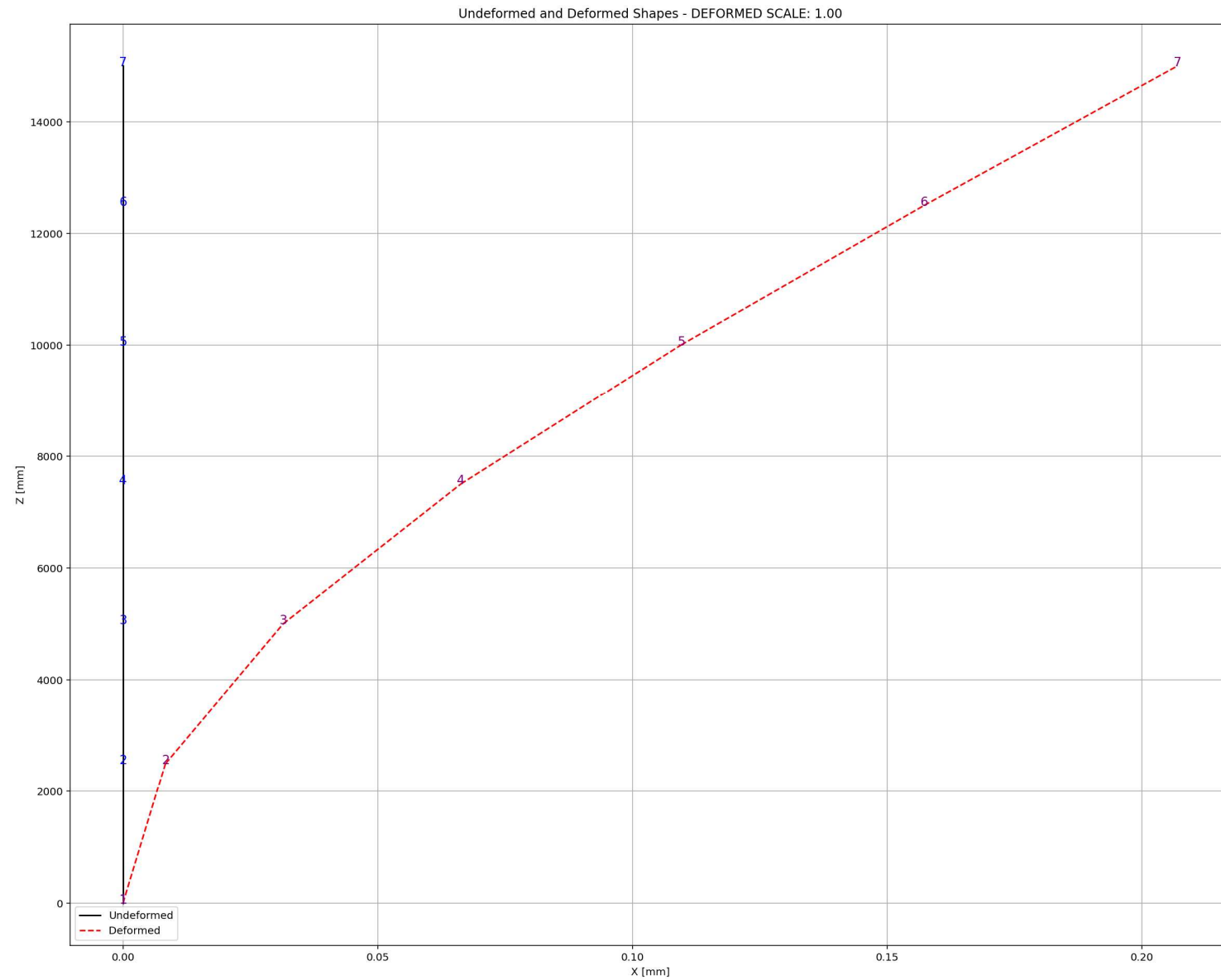


Period of Structure During Static External Time-dependent Loading Analysis



Behavior of element - Stress-strain of Top and Bottom Fibers Curve

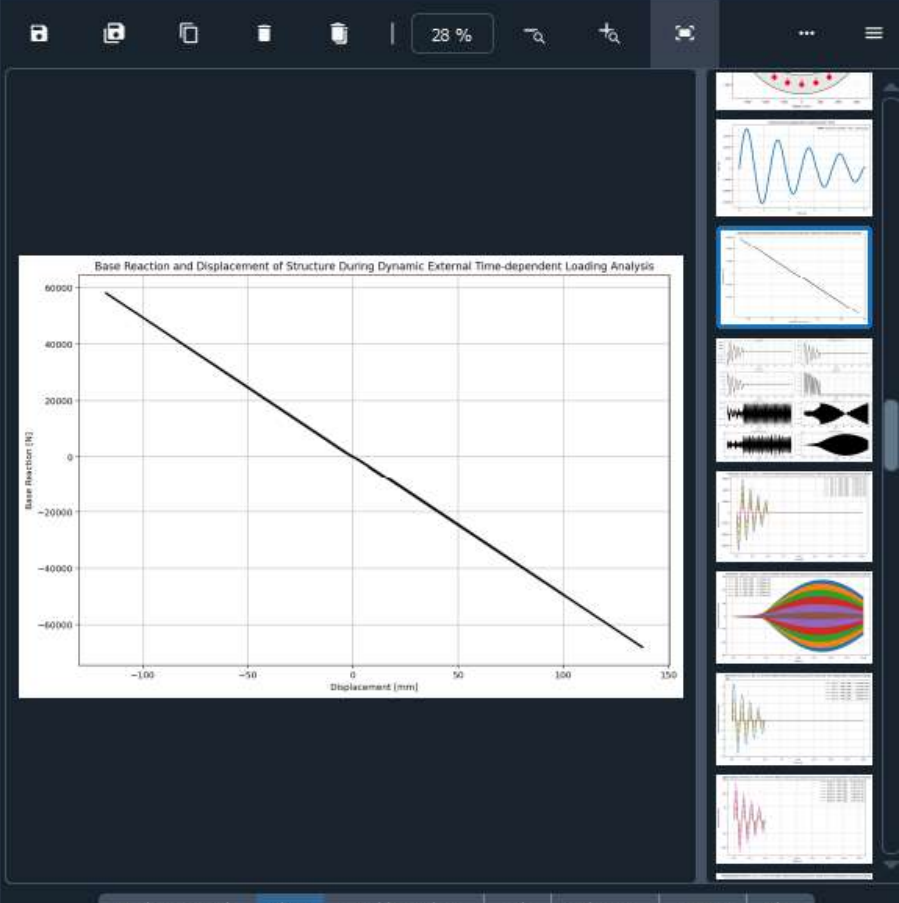




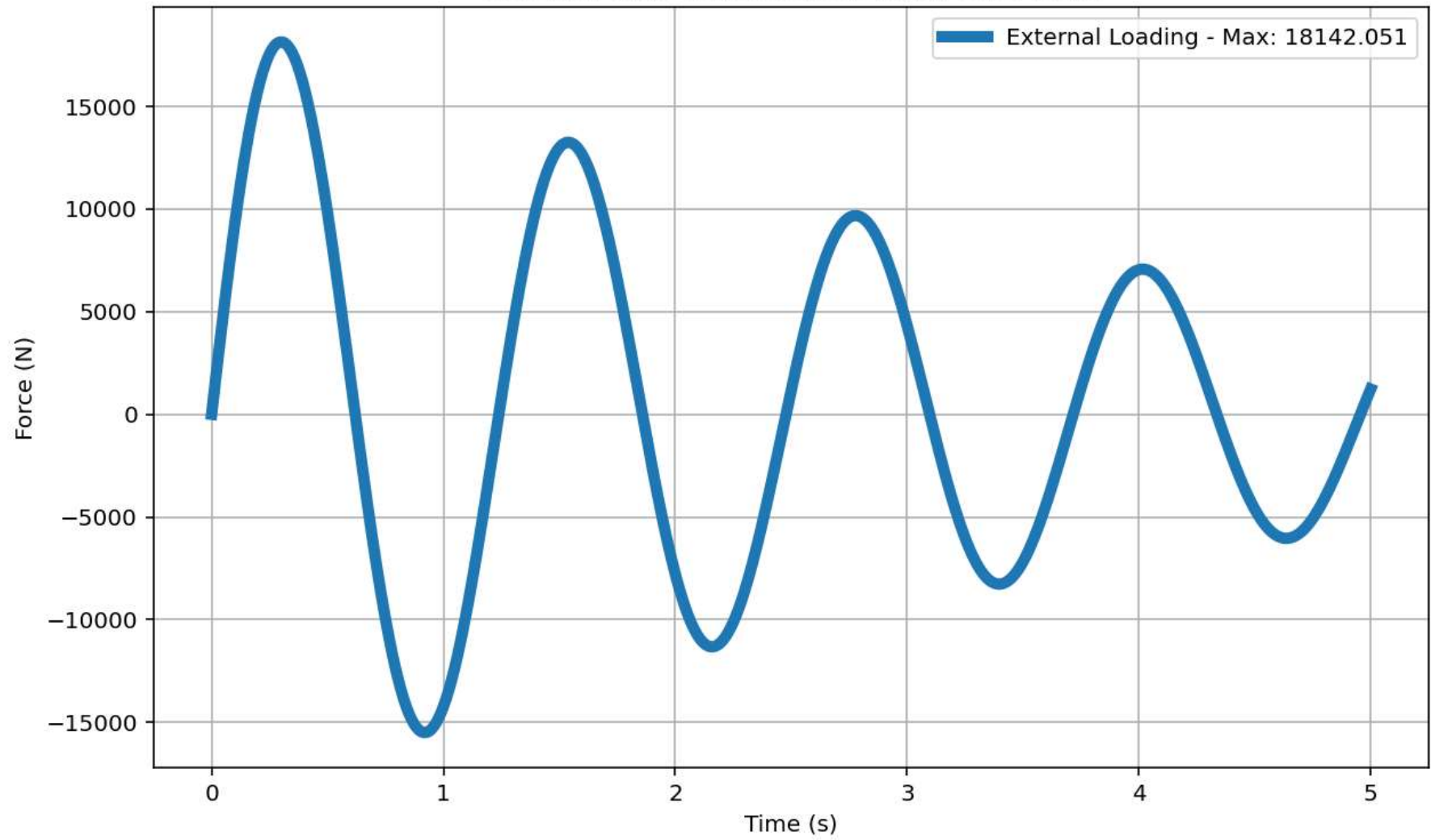
DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

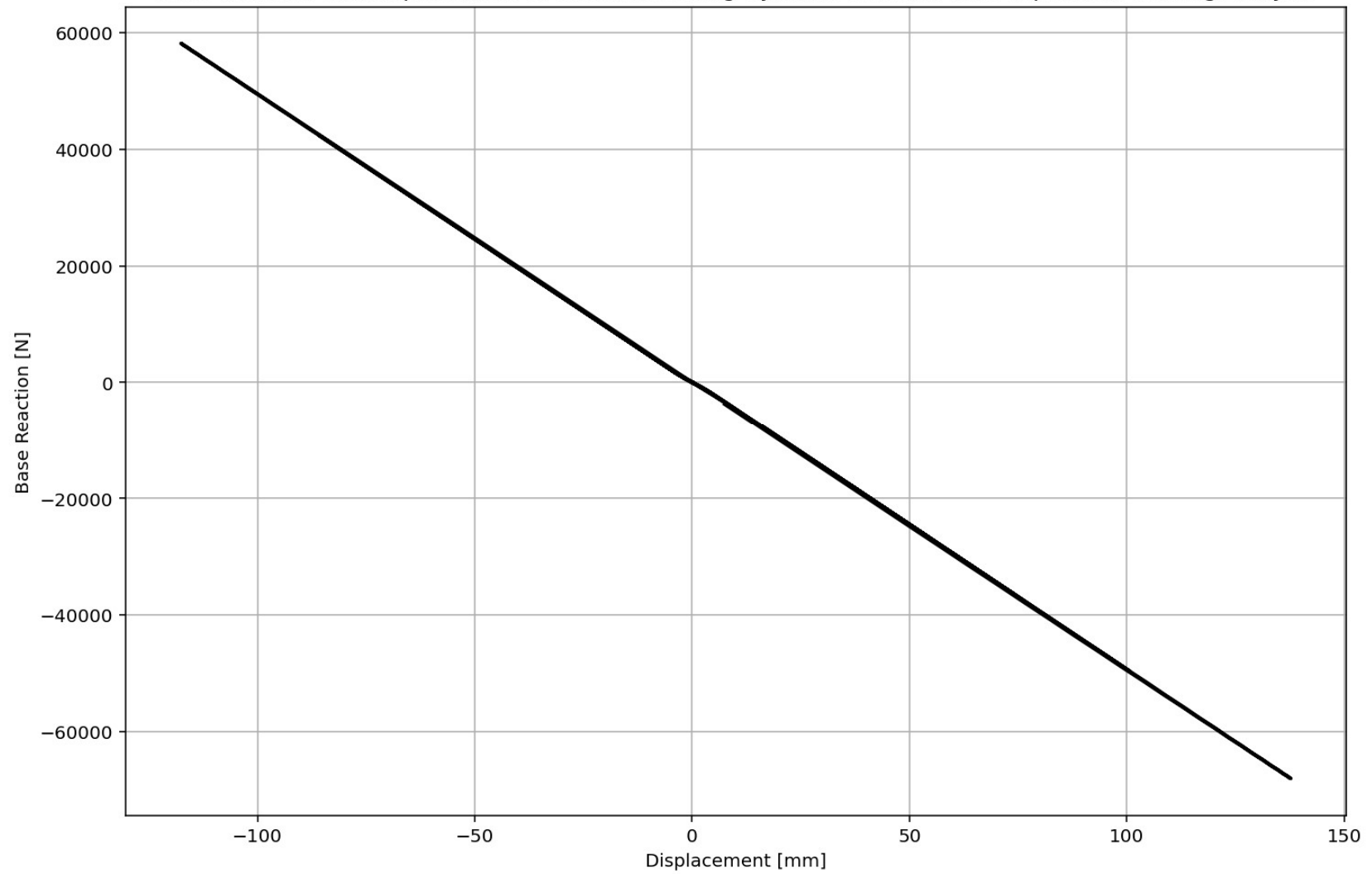
```
W(x,t)_CONCRETE_COLUMN_8_ANA.py X CONCRETE_CIRCULAR...SECTION_FUN_TWO.py X
736 if ANAL_TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANALYSIS
737     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
738     duration = 20.0 # [s] Analysis duration
739     dt = 0.01 # [s] Time step
740     DT = dt # [s] Time step
741     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
742     force_amplitude = 19600.0 # [N] Amplitude Force
743     omega_DT = 5.0715 # [rad/s] Natural angular frequency
744     DR = 0.03 # Damping Ratio
745
746     # Check function
747     def CHECK_FUN(DT_time, duration):
748         if DT_time > duration:
749             print('\n\nAnalysis Duration Must be greater than External Load Duration\n\n')
750             exit()
751         return -1
752
753     CHECK_FUN(DT_time, duration)
754     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(
755         import numpy as np
756         import matplotlib.pyplot as plt
757         # External Load Durations
758         num_steps = int(DT_time / DT)
759         load_time = np.linspace(0, DT_time, num_steps)
760         target_frequency = 1.0 * omega_DT # Target excitation frequency
761         DT_load = force_amplitude * np.sin(target_frequency * load_time)
762         # Plot External Time-dependent Loading
763         plt.figure(figsize=(10, 6))
764         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load):.3f}')
765         plt.title('External Time-dependent Loading Over Time')
766         plt.xlabel('Time (s)')
767         plt.ylabel('Force (N)')
768         plt.grid(True)
769         plt.legend()
```

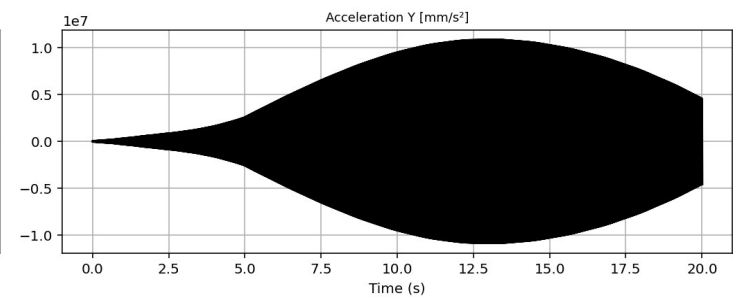
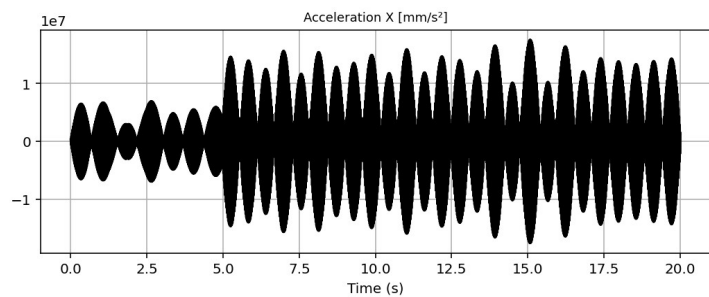
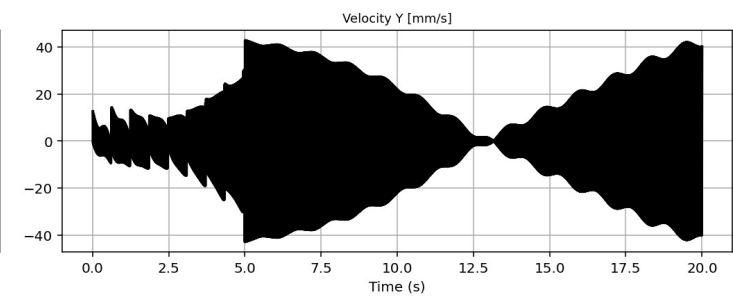
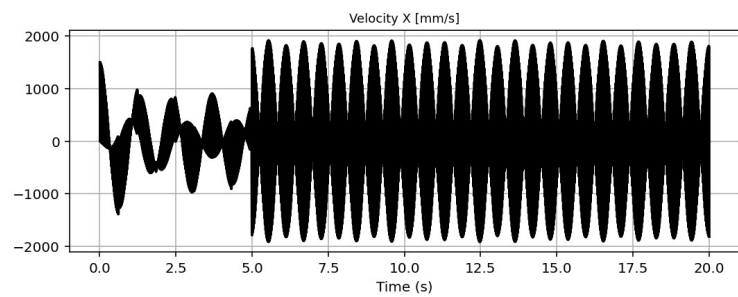
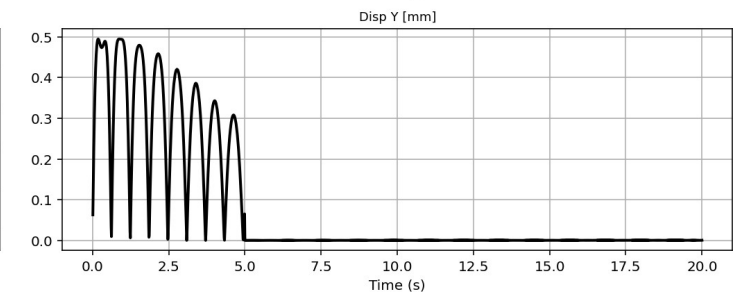
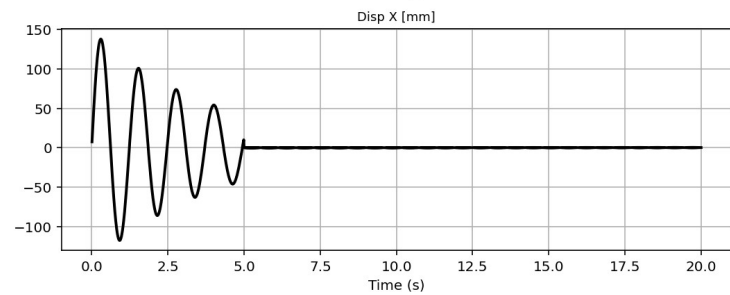
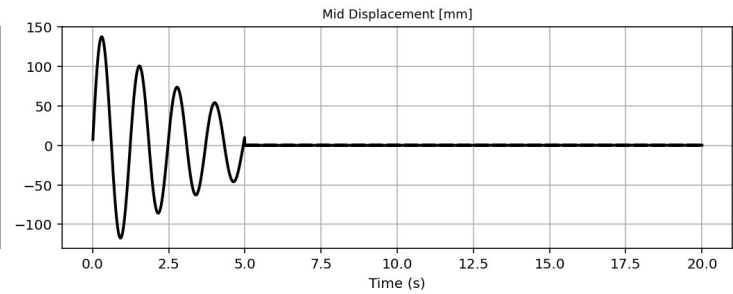
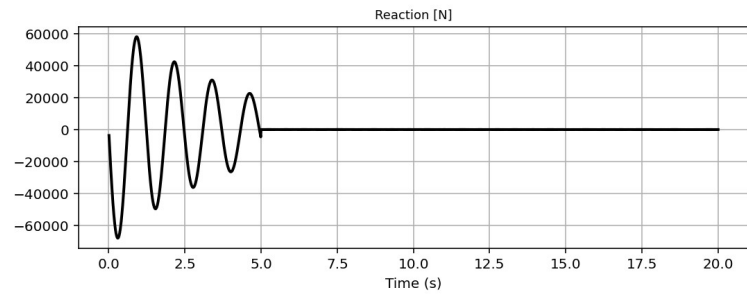


External Time-dependent Loading Over Time

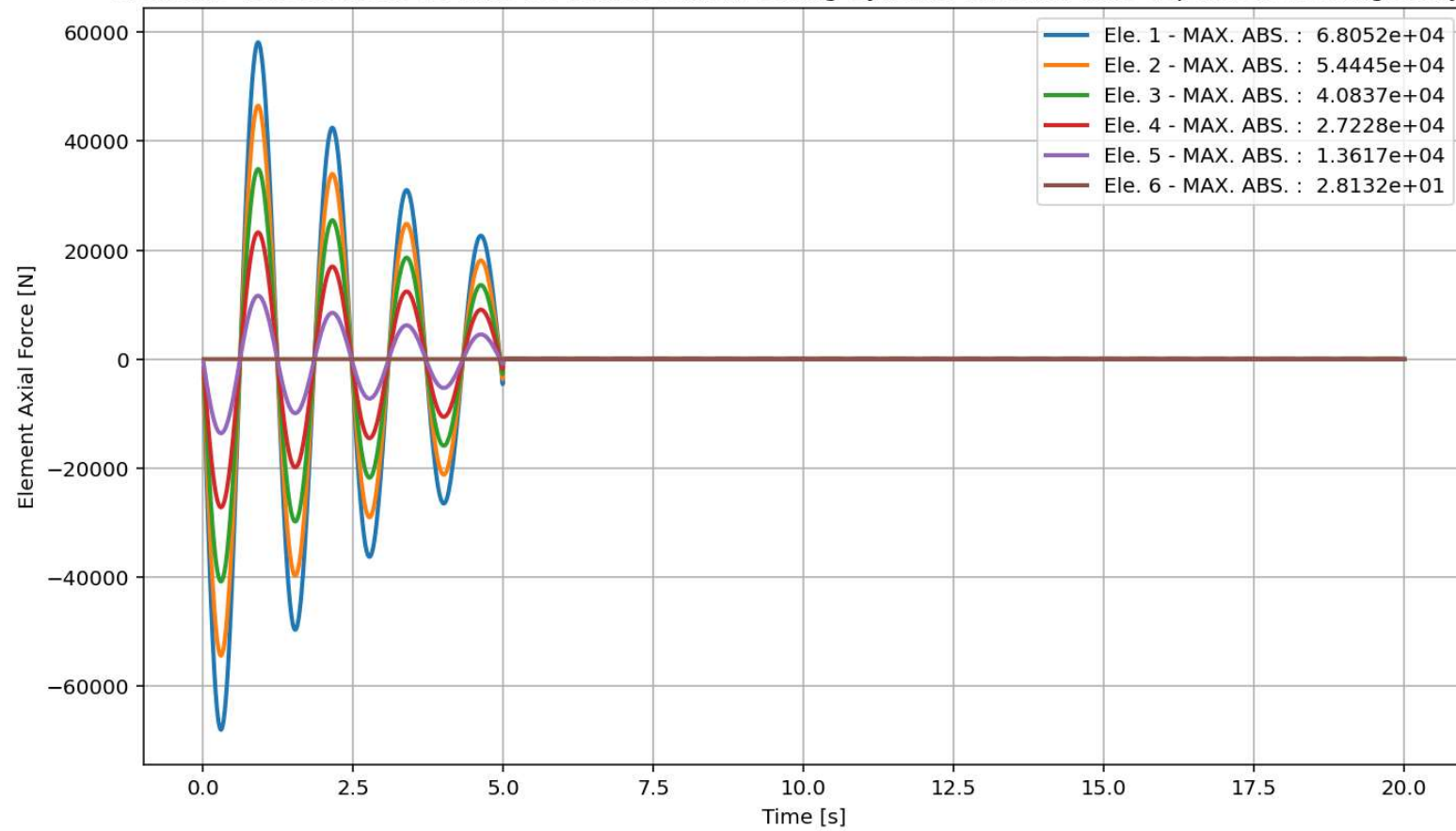


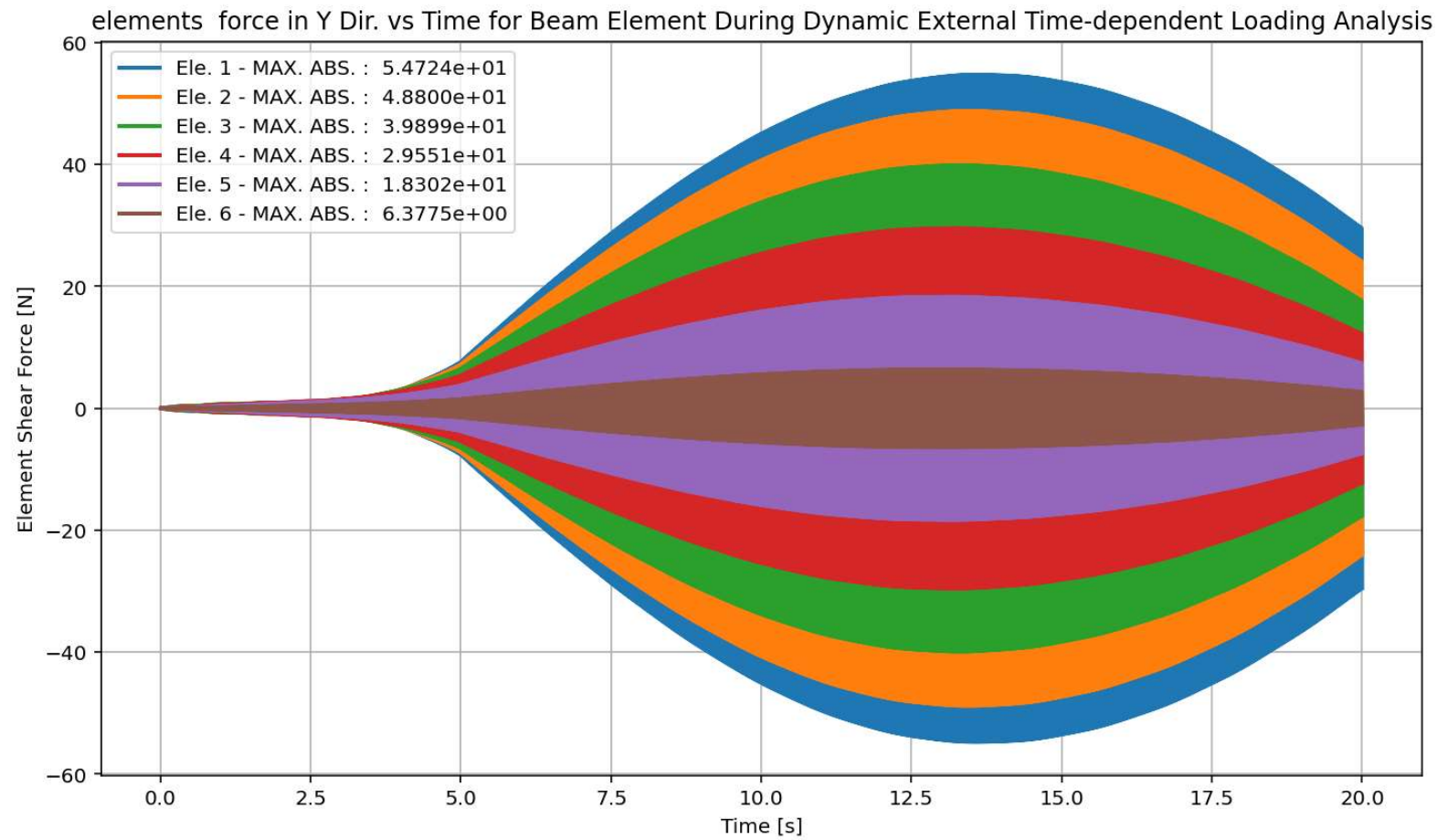
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis



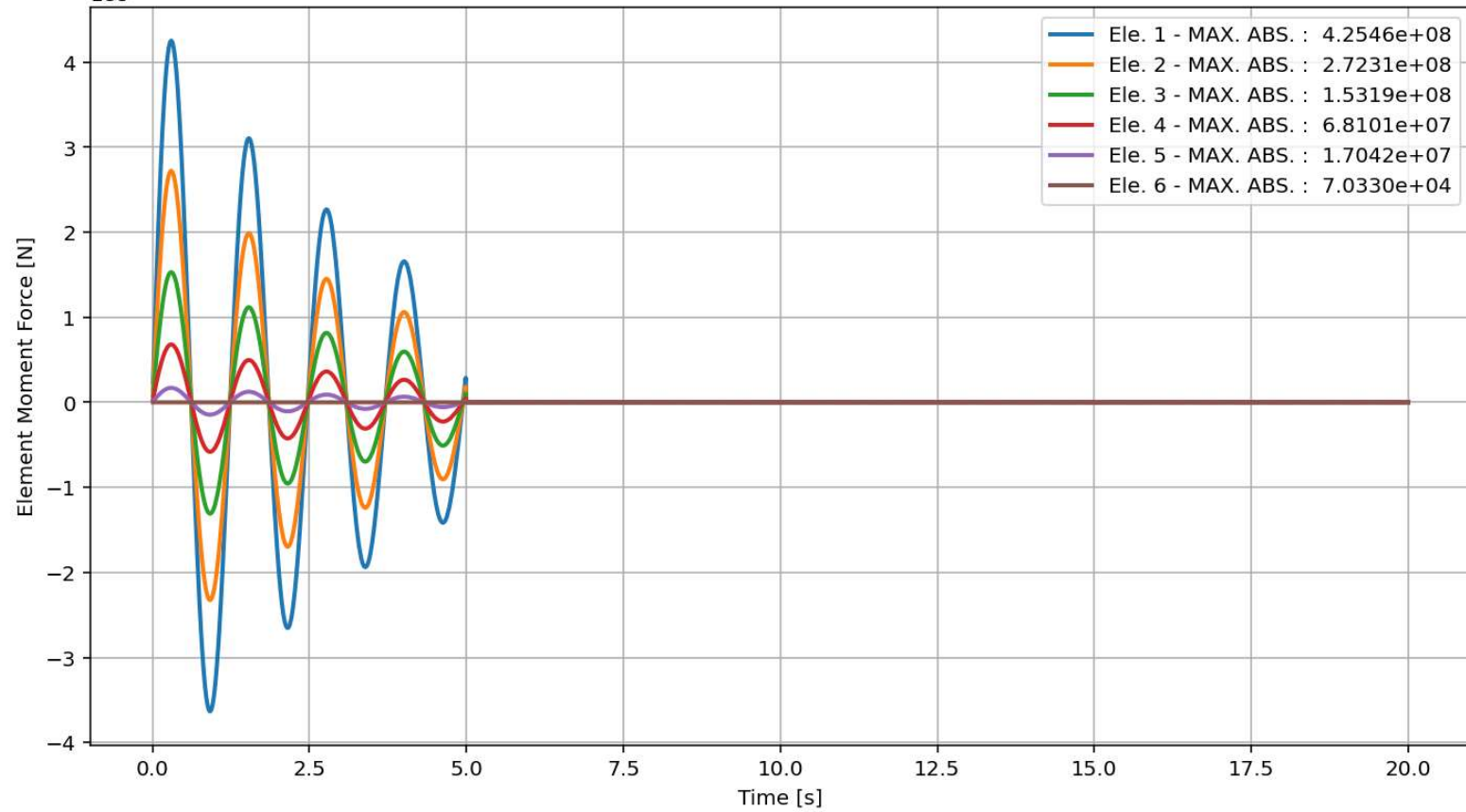


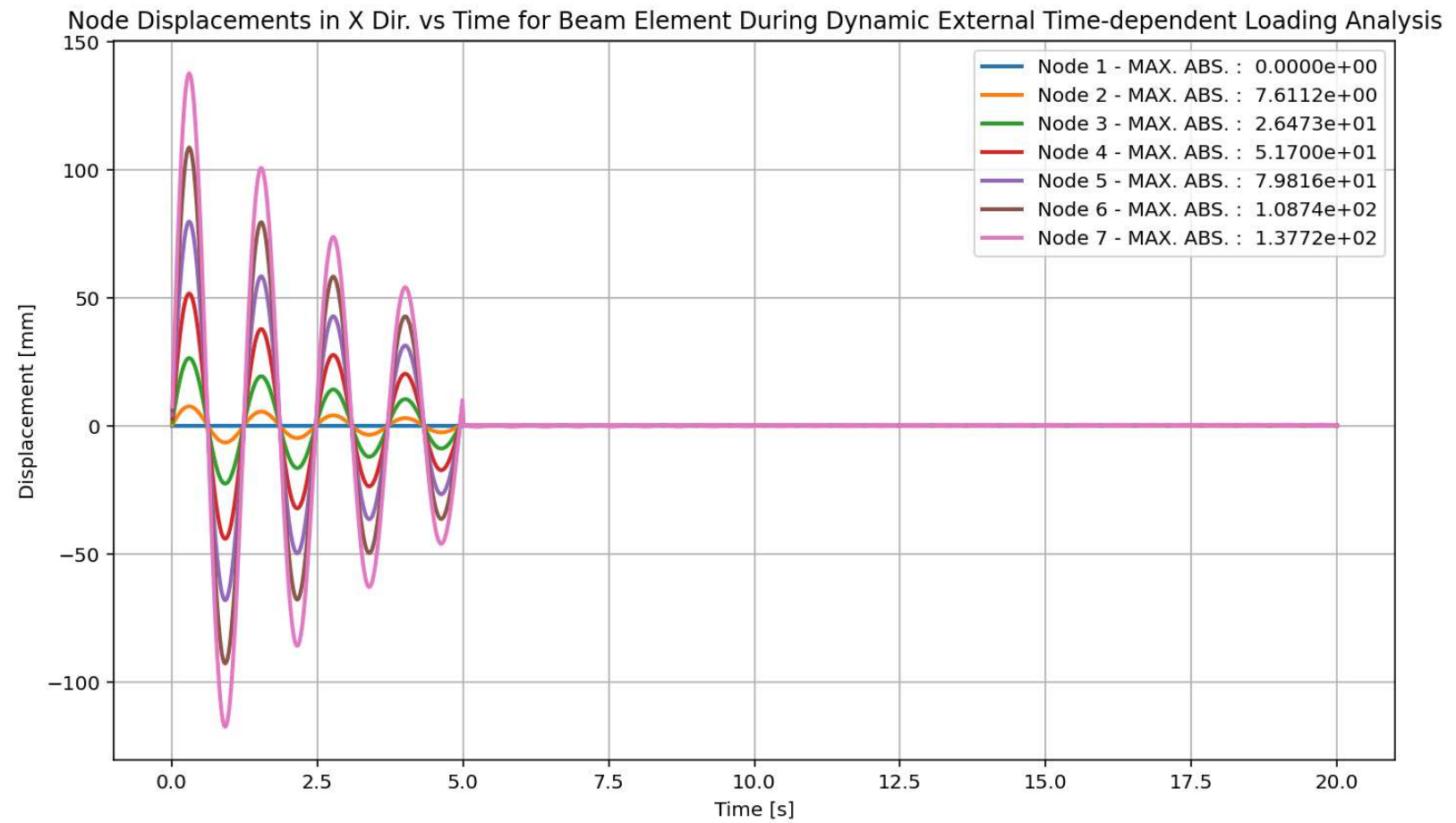
elements force in X Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



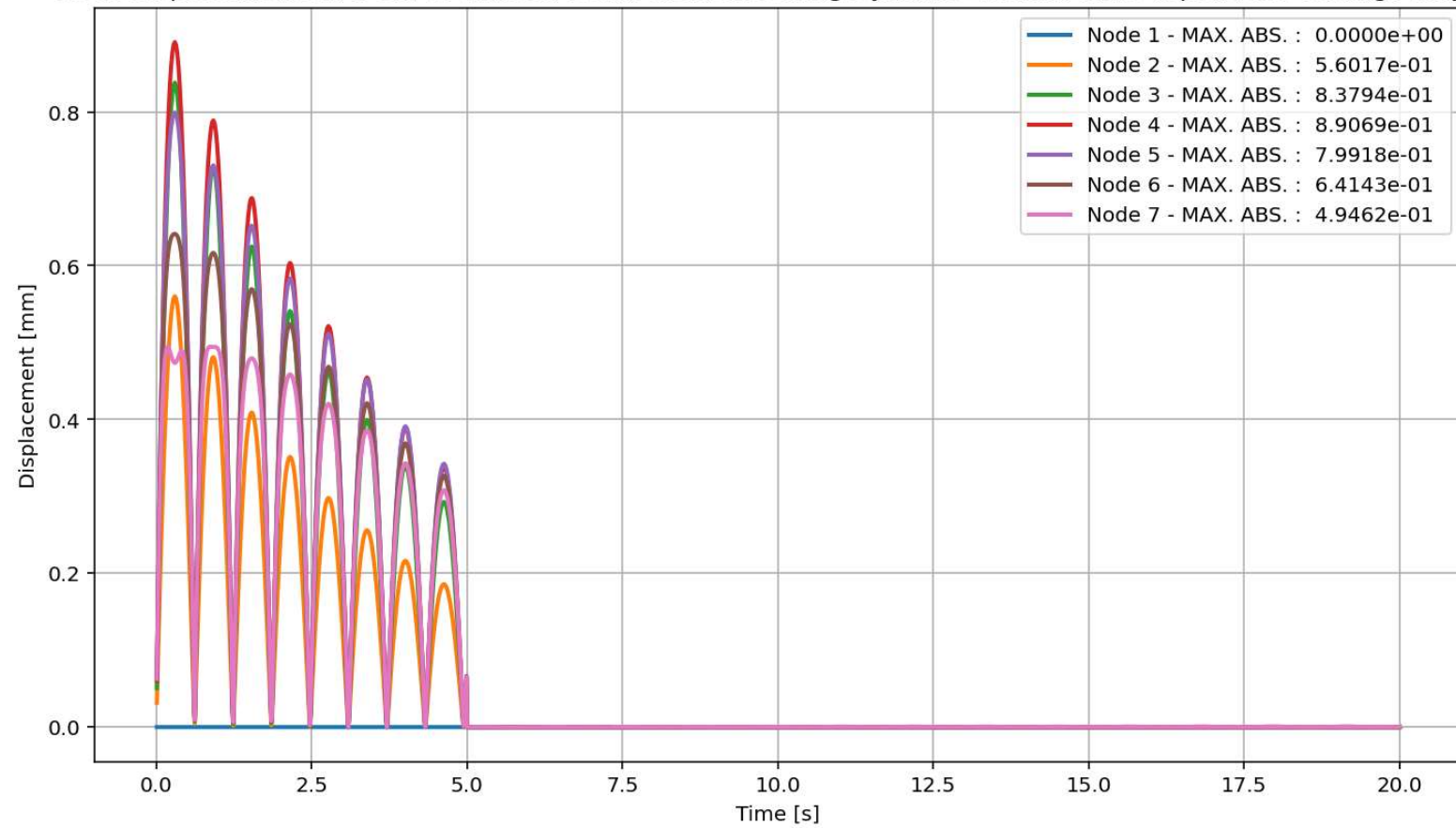


elements force in Z Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis
1e8

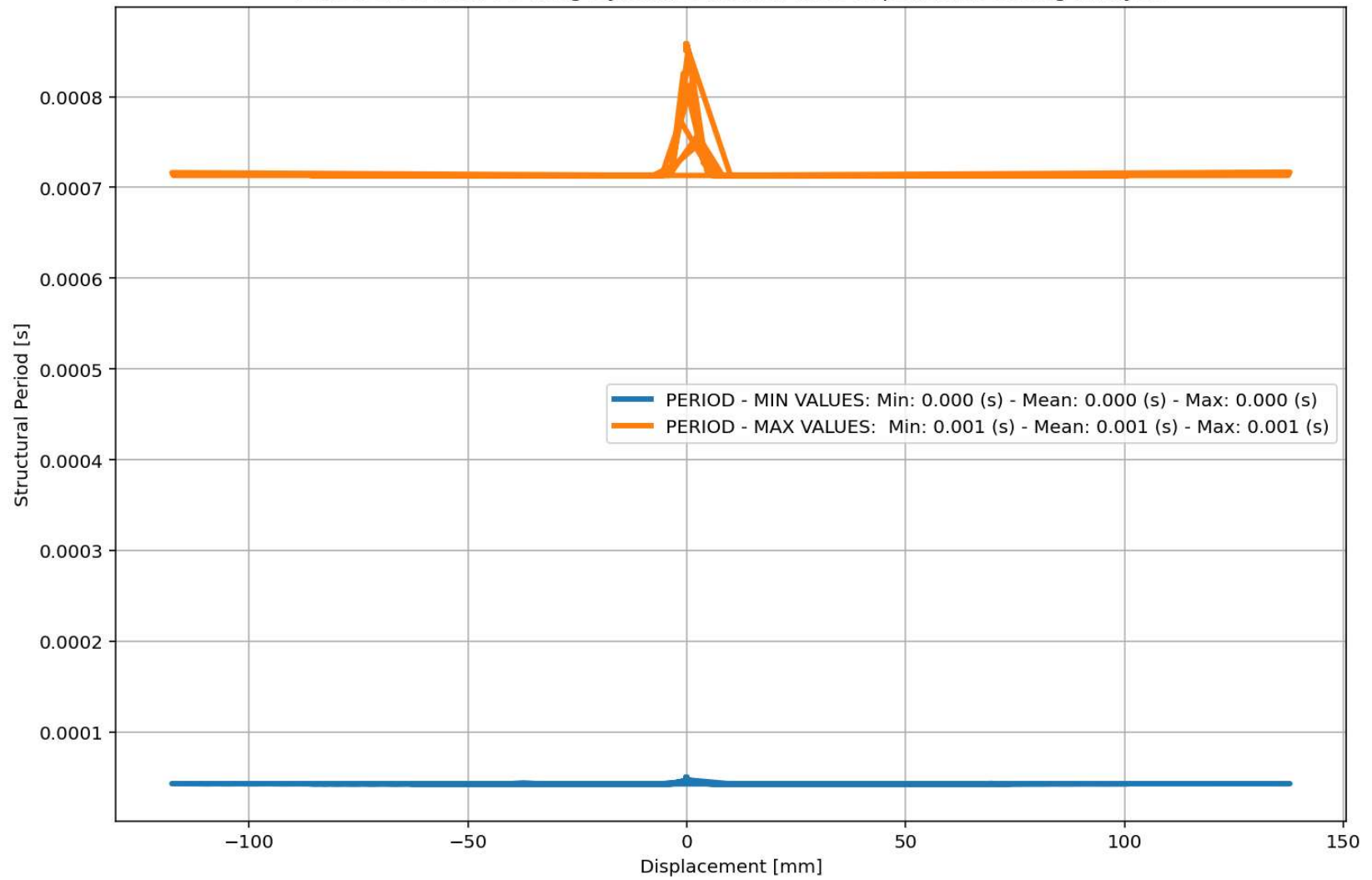


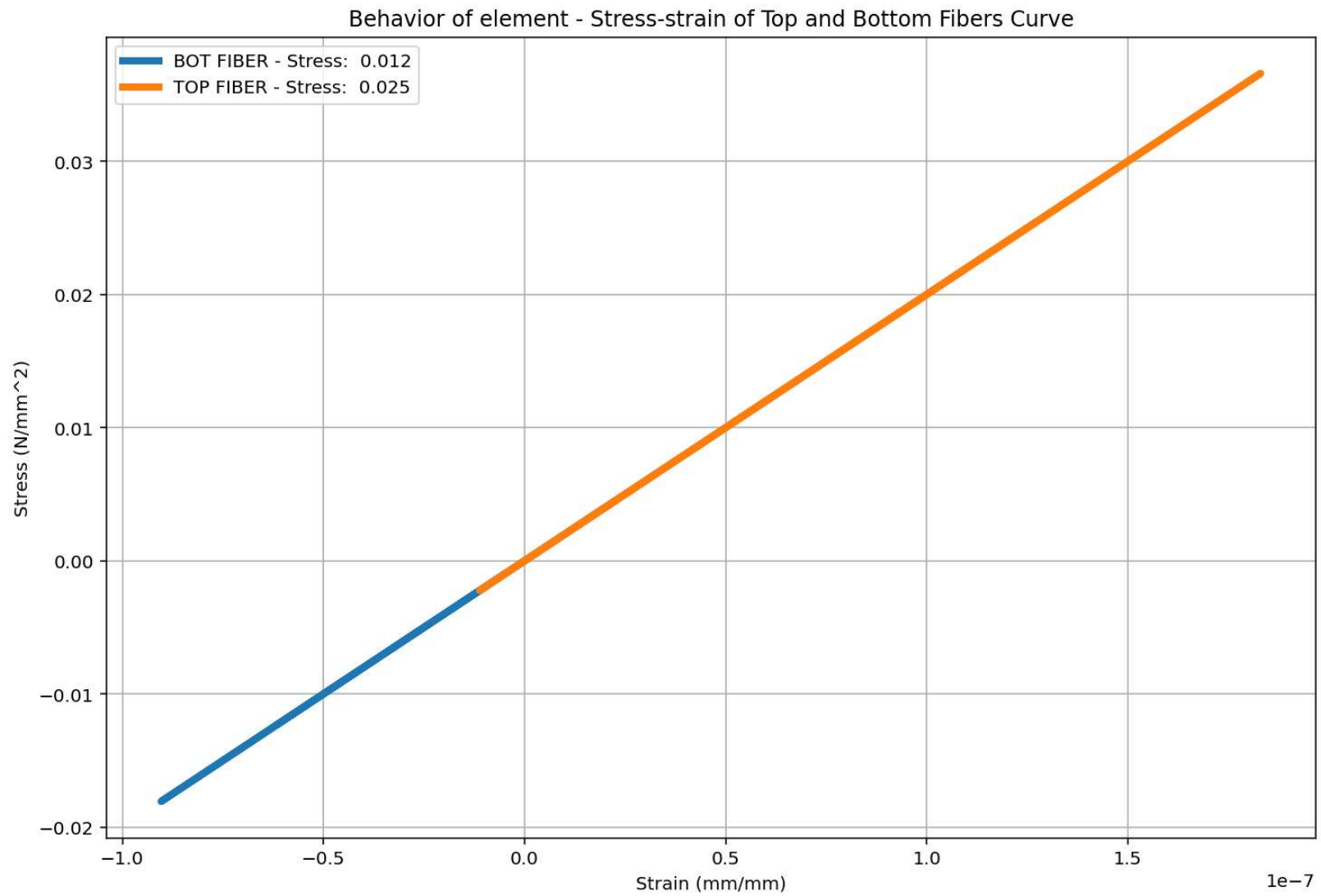


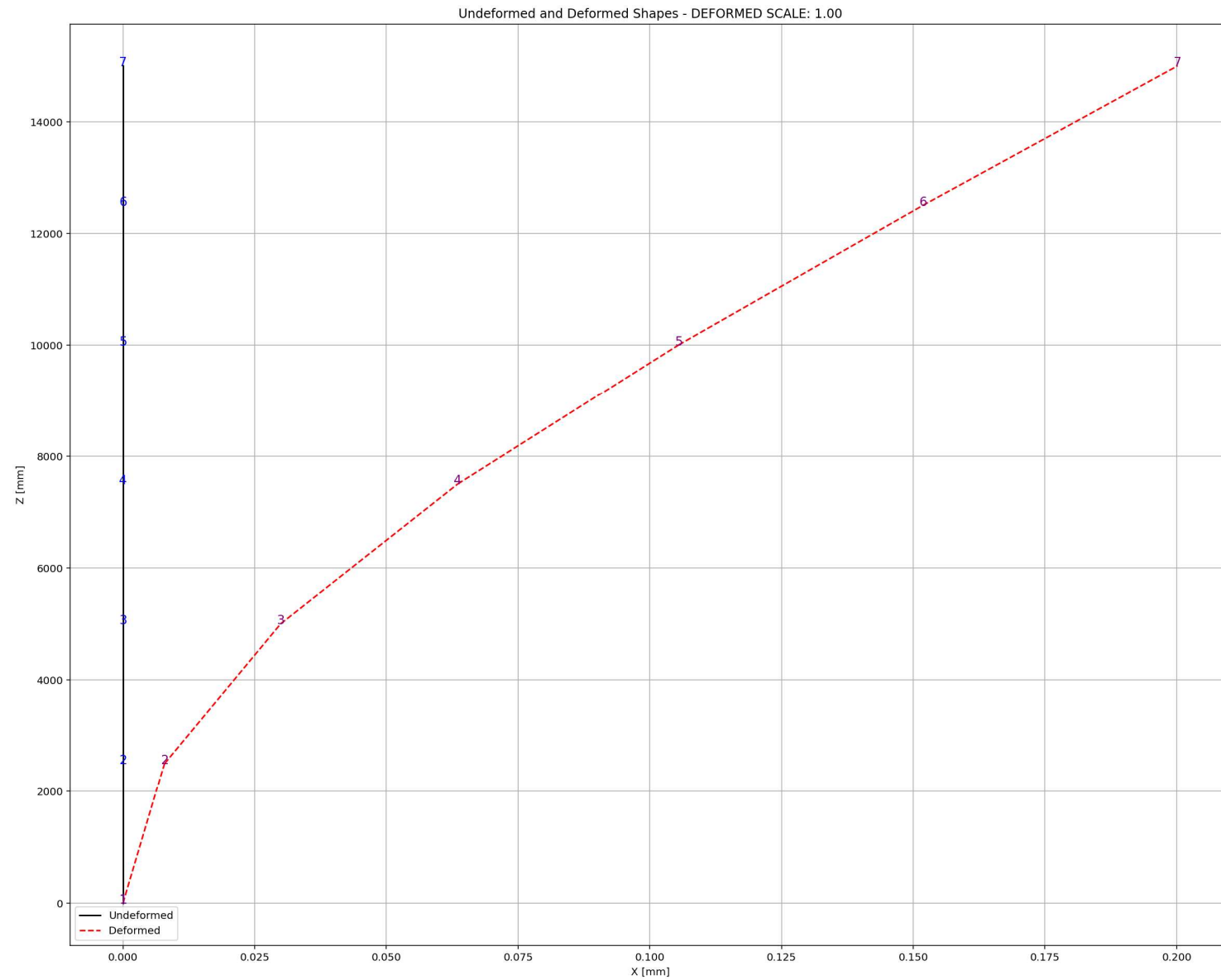
Node Displacements in Y Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



Period of Structure During Dynamic External Time-dependent Loading Analysis







FREE-VIBRATION ANALYSIS

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RETE_COLUMN_8_ANA\W(x,t)_CONCRETE_COLUMN_8_ANA.py

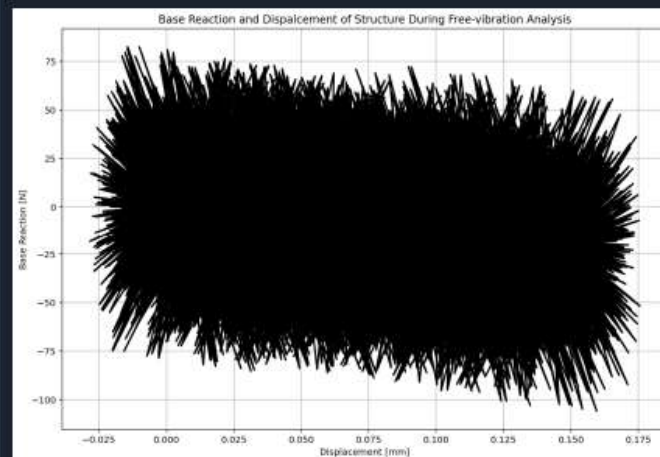
W(x,t)_CONCRETE_COLUMN_8_ANA.py X CONCRETE_CIRCULAR...SECTION_FUN_TWO.py X

```

902
903     if ANAL TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
904         %% DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
905         u0 = -0.35                # [mm] Initial displacement
906         v0 = 0.15                 # [mm/s] Initial velocity
907         a0 = 0.065               # [mm/s^2] Initial acceleration
908         IU = True                # Free Vibration with Initial Displacement
909         IV = True                # Free Vibration with Initial Velocity
910         IA = True                # Free Vibration with Initial Acceleration
911         duration = 20.0          # [s] Analysis duration
912         dt = 0.001              # [s] Time step
913         DR = 0.03                # Damping Ratio
914
915         if IU == True:
916             # Define initial displacment
917             ops.setNodeDisp(center_node, 1, u0, '-commit') # INFO LINK: https://openseespydoc
918         if IV == True:
919             # Define initial velocity
920             ops.setNodeVel(center_node, 1, v0, '-commit') # INFO LINK: https://openseespydoc
921         if IA == True:
922             # Define initial acceleration
923             ops.setNodeAccel(center_node, 1, a0, '-commit') # INFO LINK: https://openseespydoc
924
925         ops.constraints('Plain')
926         ops.numberer('Plain')
927         ops.system('BandGeneral')
928         ops.test('NormDispIncr', MAX_TOLERANCE, MAX_ITERATIONS) # INFO LINK: https://opensees
929         #ops.integrator('CentralDifference') # JUST FOR LINEAR ANALYSIS - INFO LINK: https://opensees
930         alpha=0.5; beta=0.25;
931         ops.integrator('Newmark', alpha, beta) # INFO LINK: https://openseespydoc.readthedocs
932         #alpha=2/3; gamma=1.5-alpha; gamma=(2-alpha)**2/4;
933         #ops.integrator('HHT', alpha, gamma, beta) # INFO LINK: https://openseespydoc.readthedocs
934         ops.algorithm('Newton') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/ops\_analysis/Transient
935

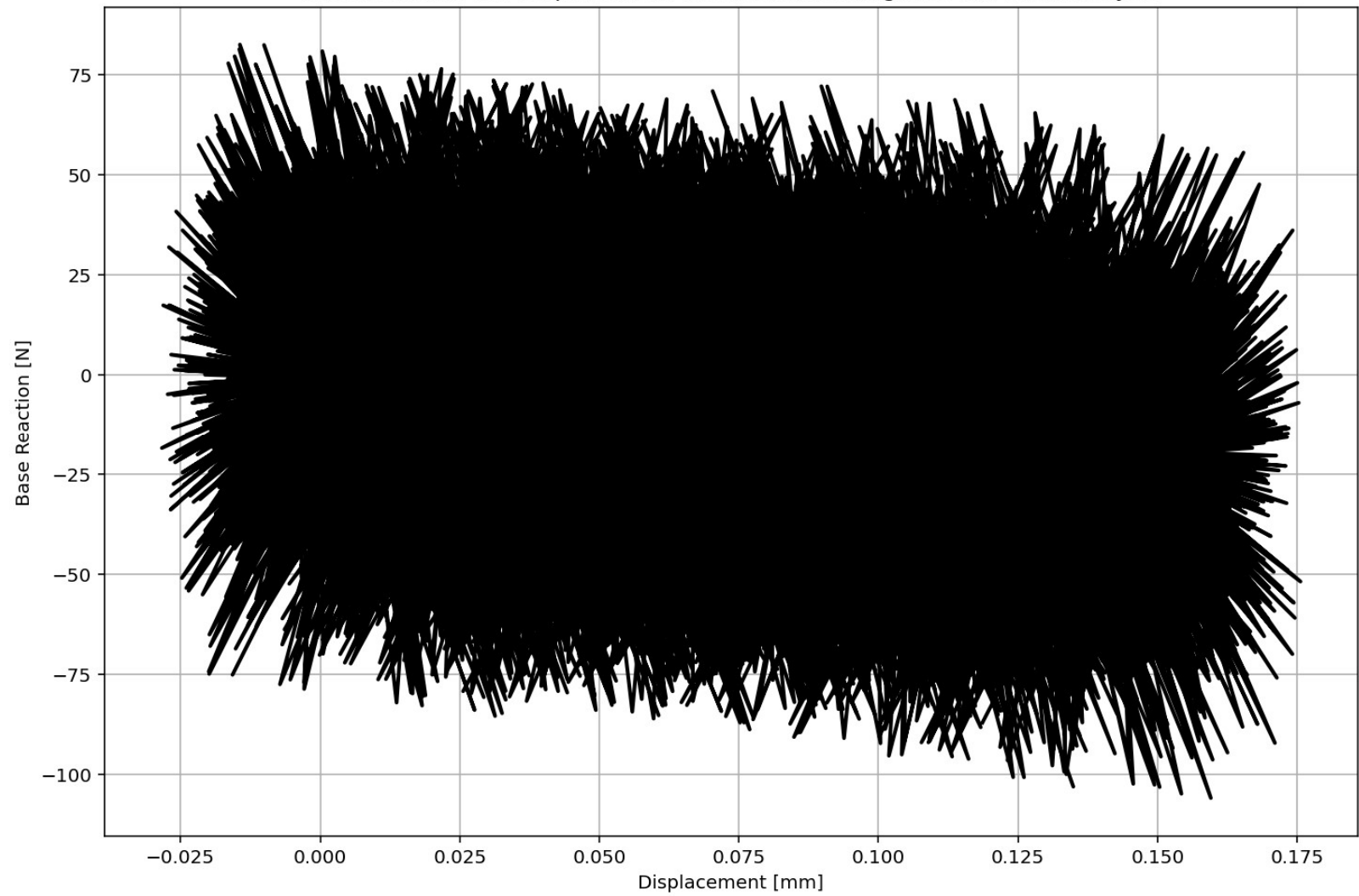
```

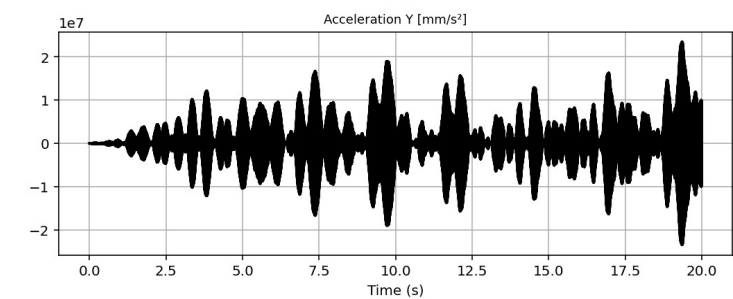
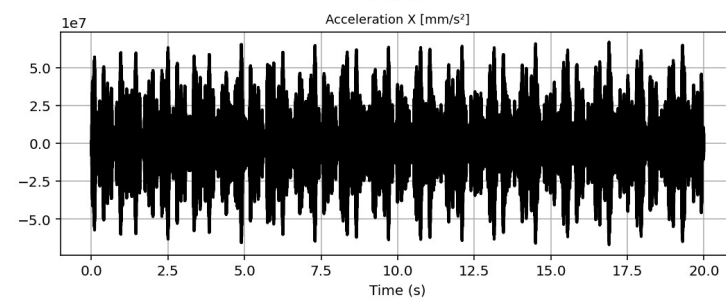
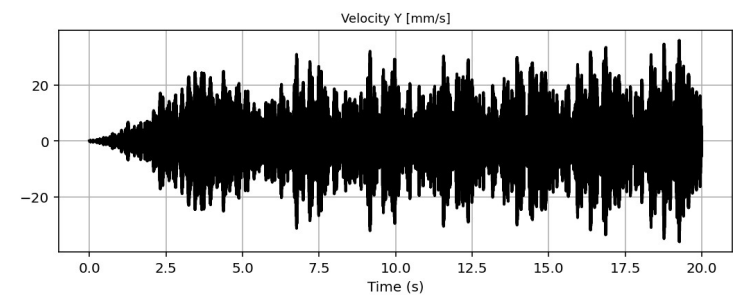
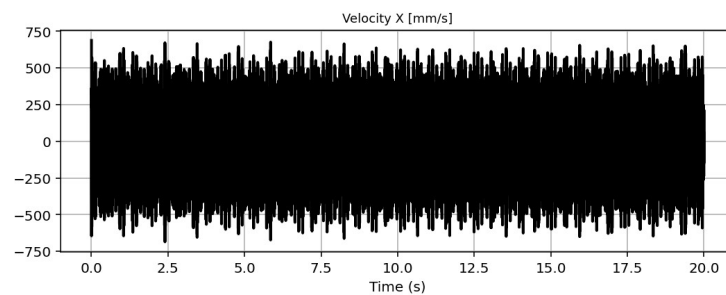
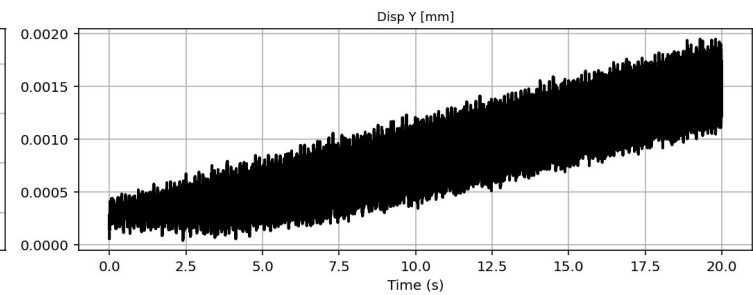
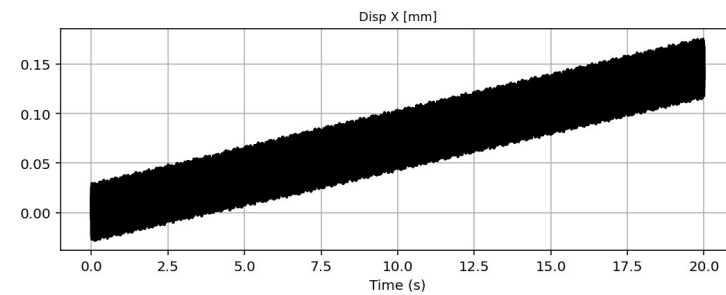
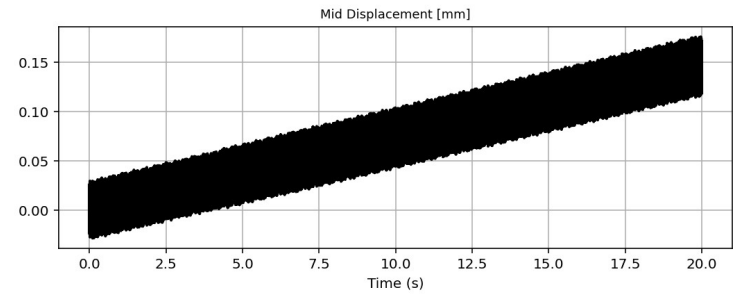
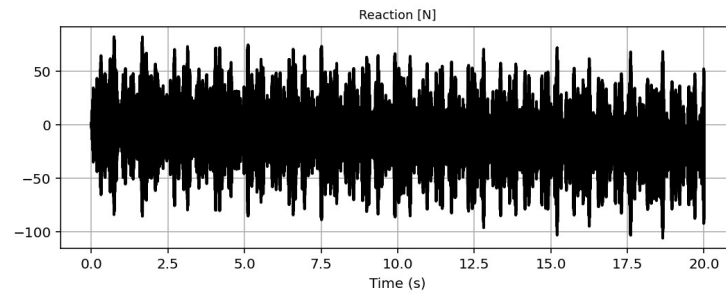
29 %



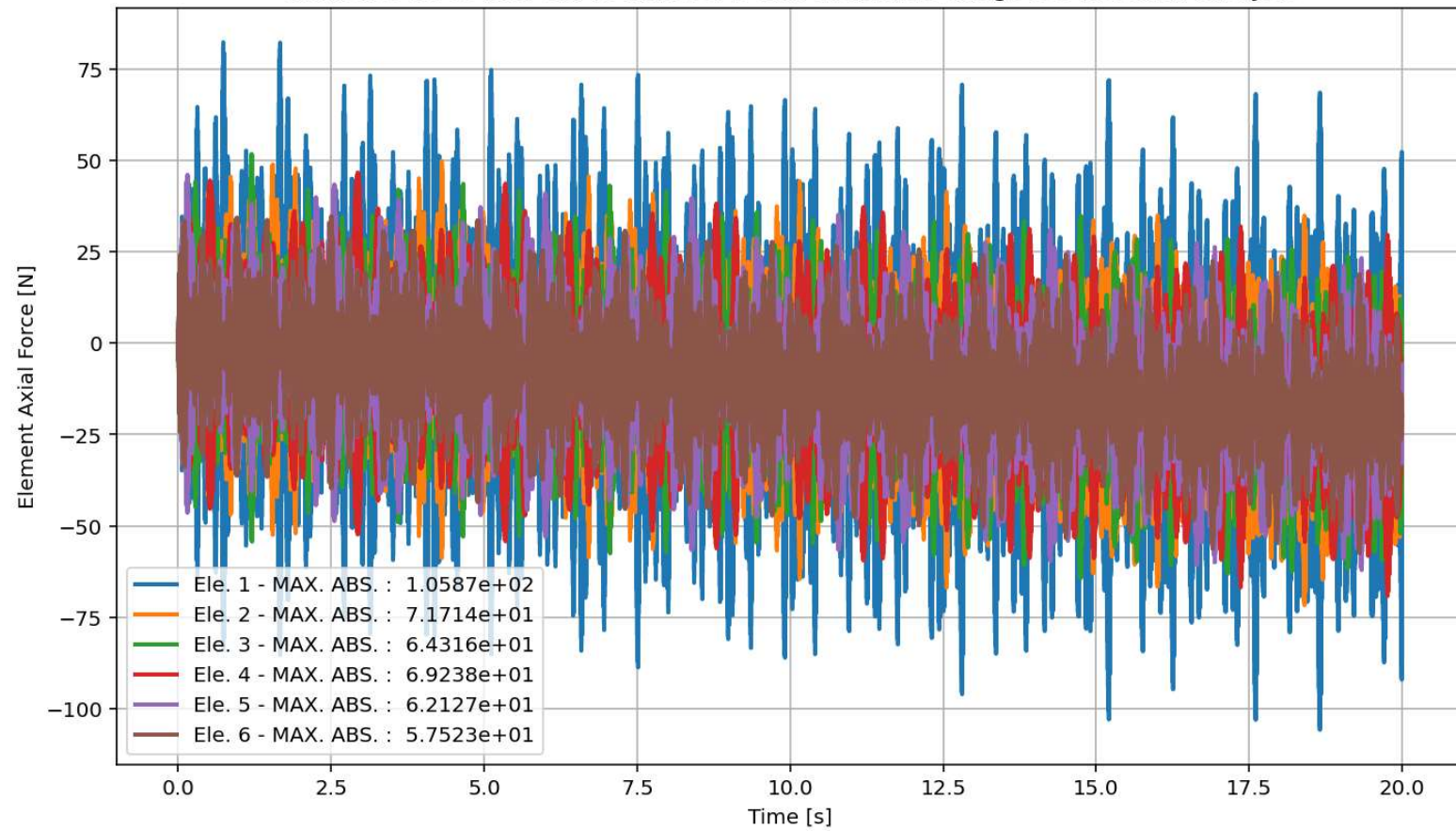
IPython Console Plots Variable Explorer Help Debugger History Files

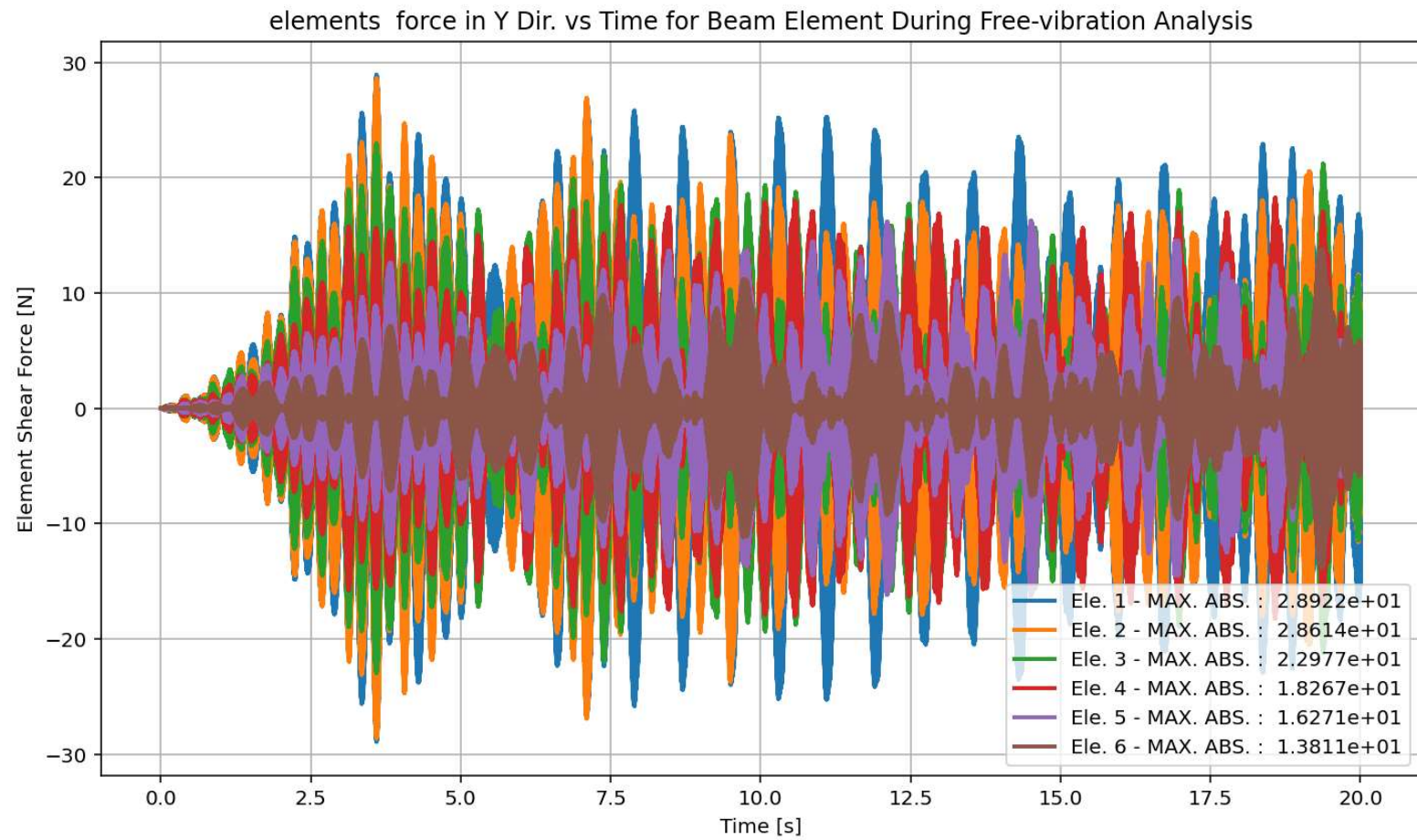
Base Reaction and Dispalcement of Structure During Free-vibration Analysis

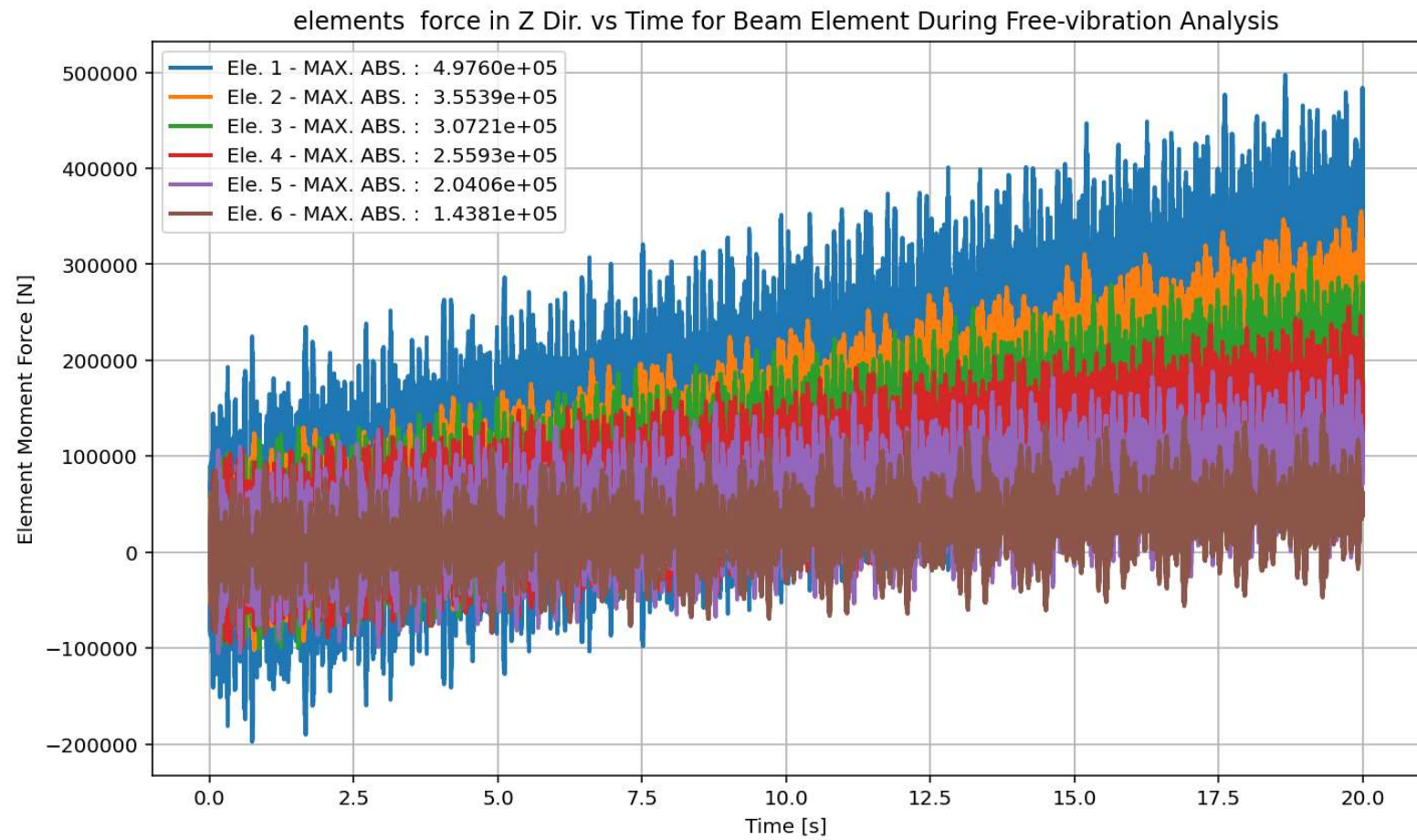


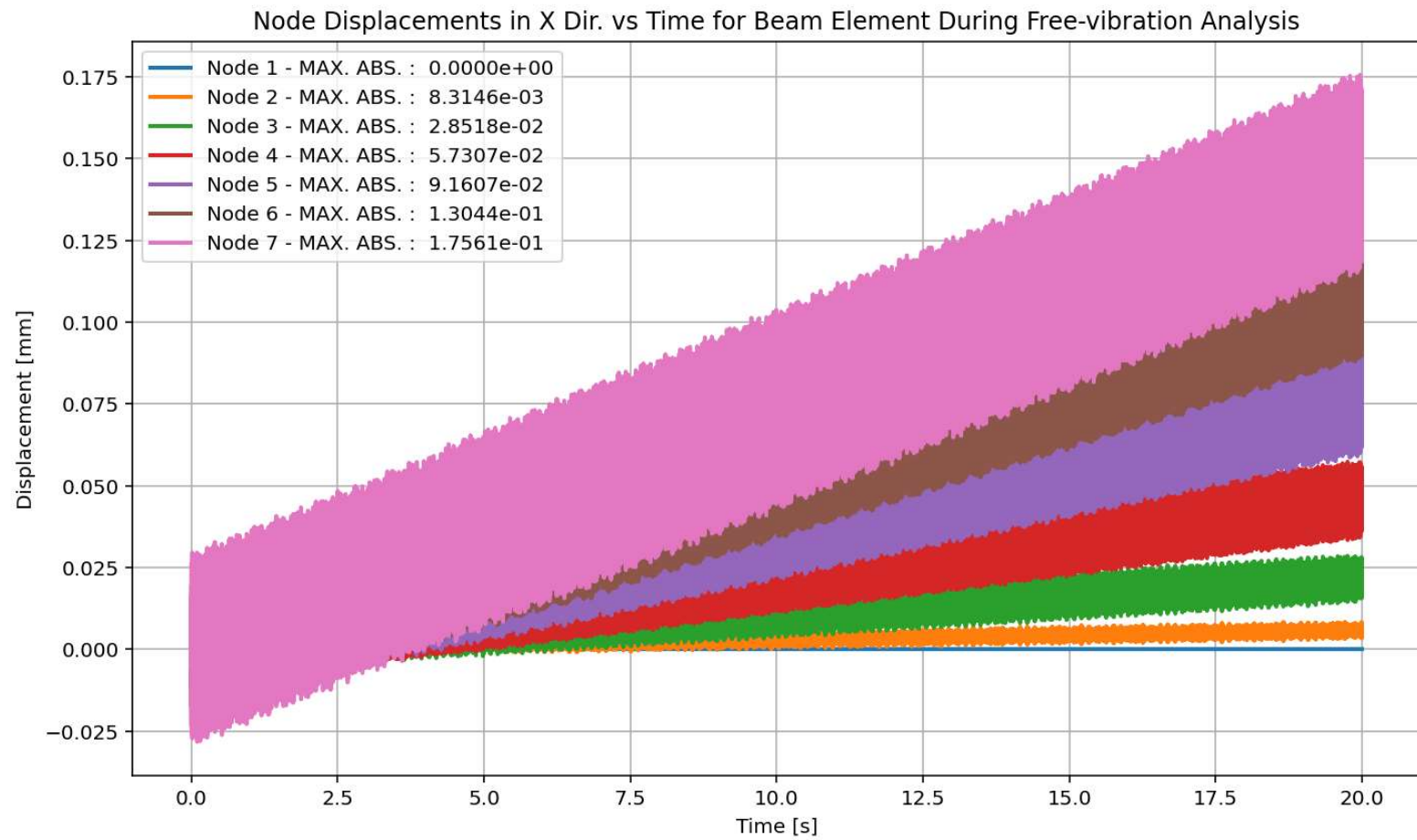


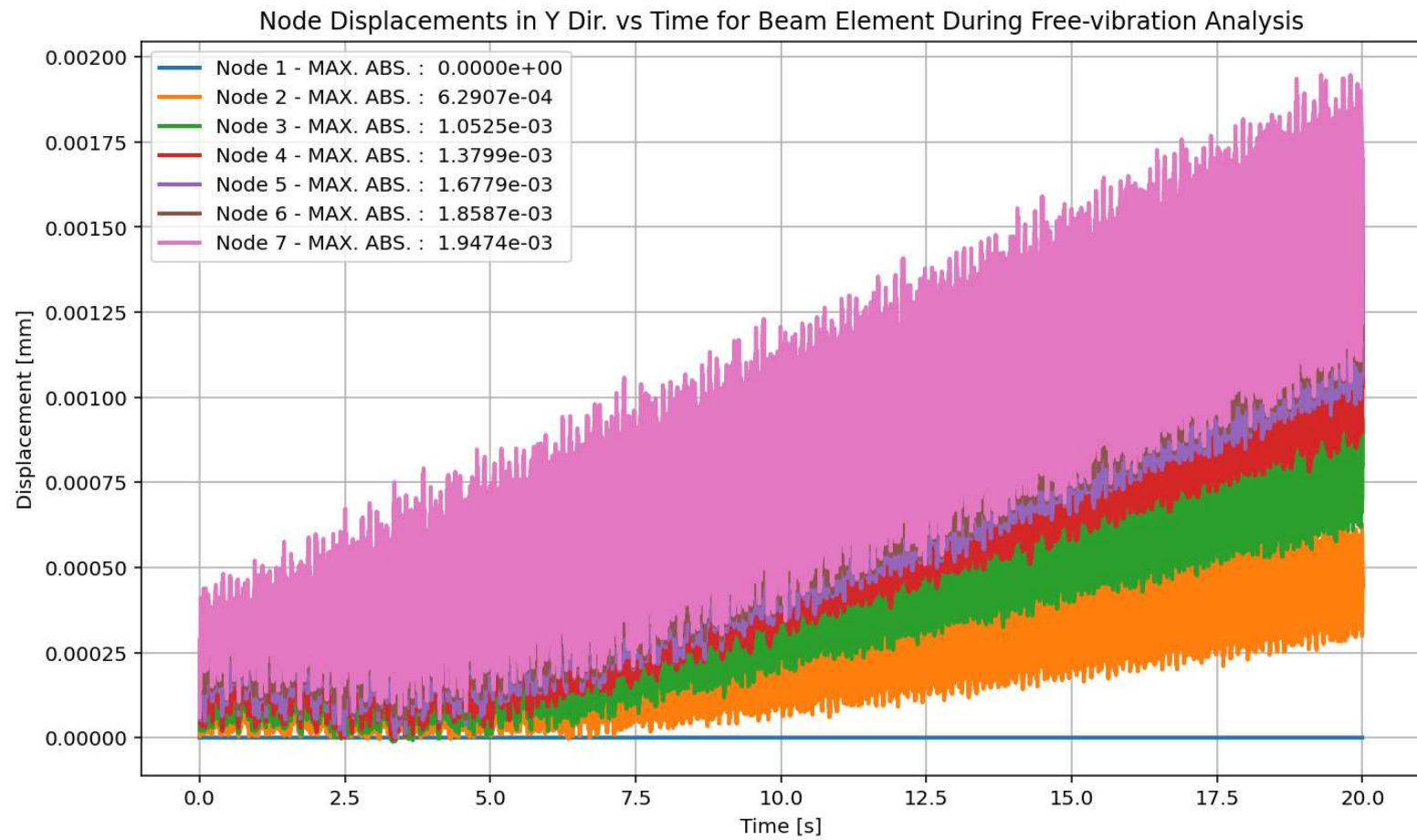
elements force in X Dir. vs Time for Beam Element During Free-vibration Analysis

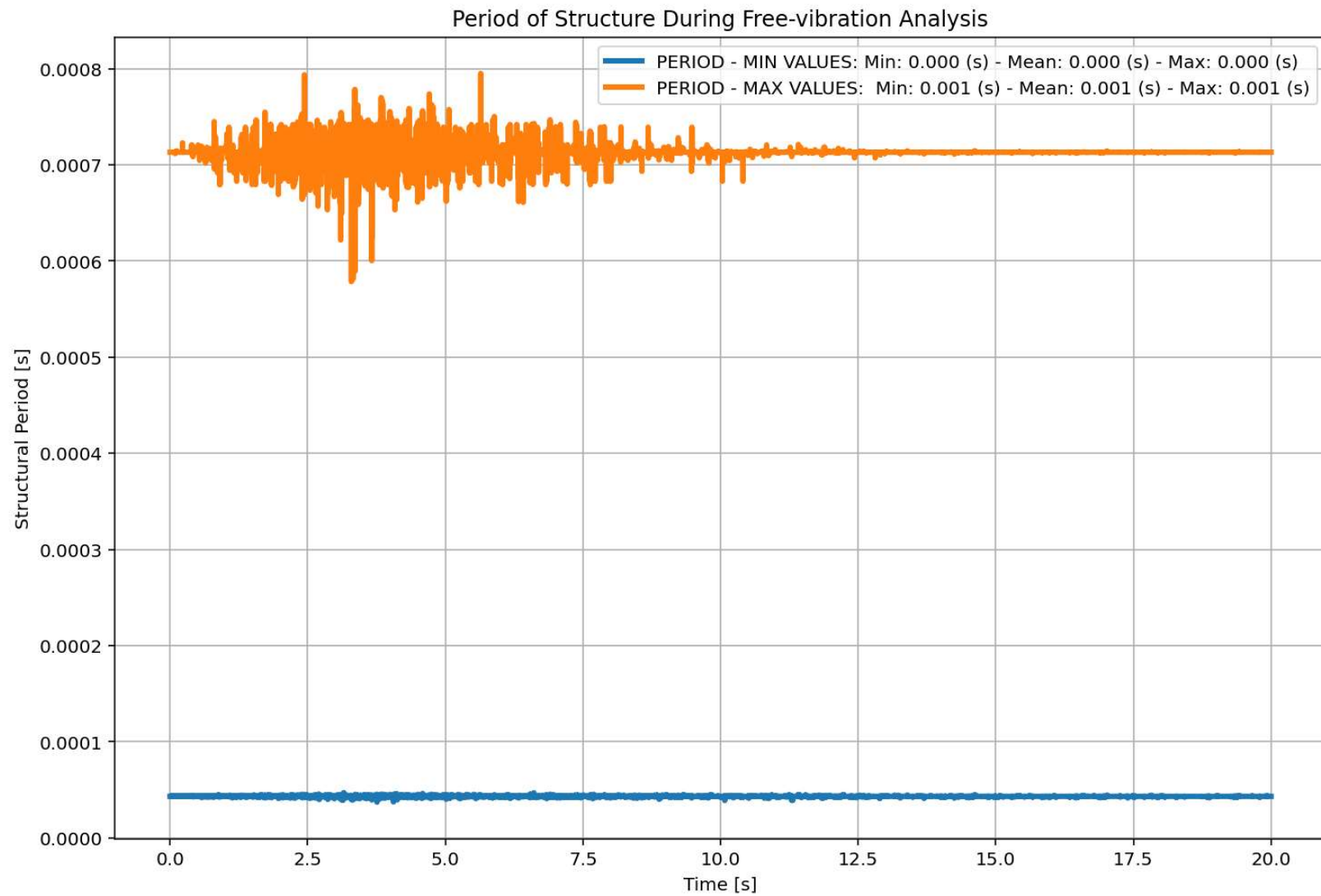




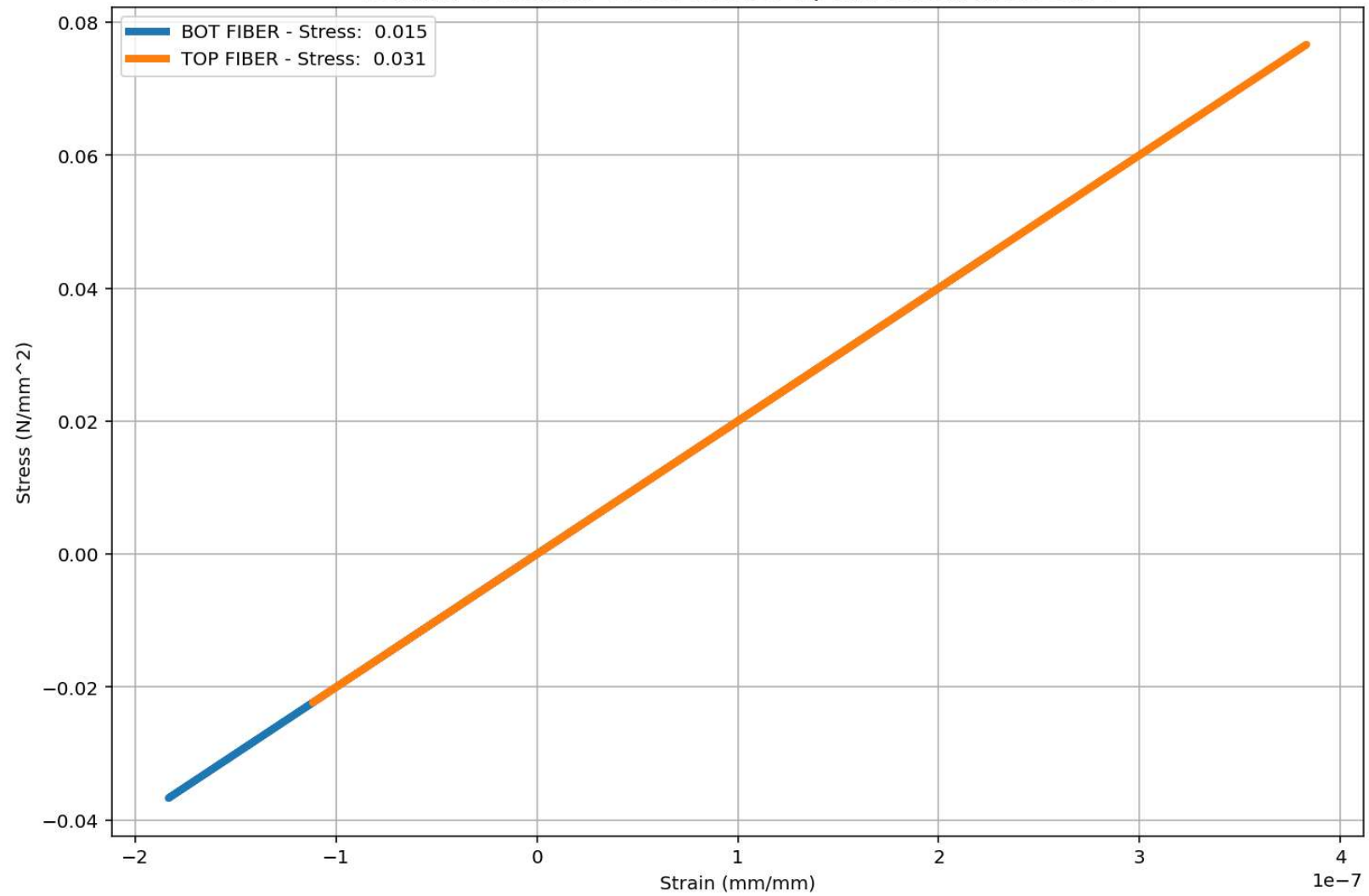


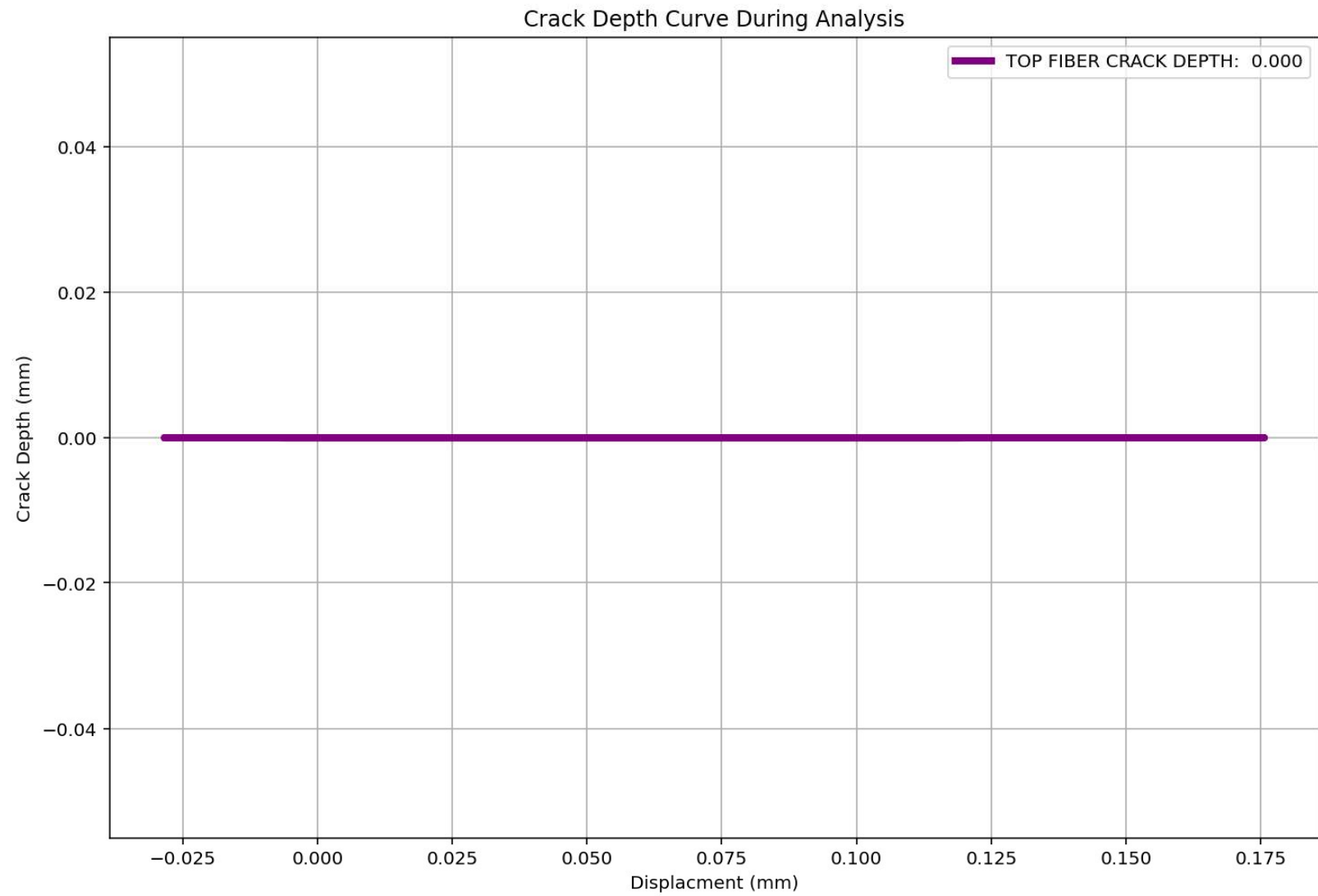


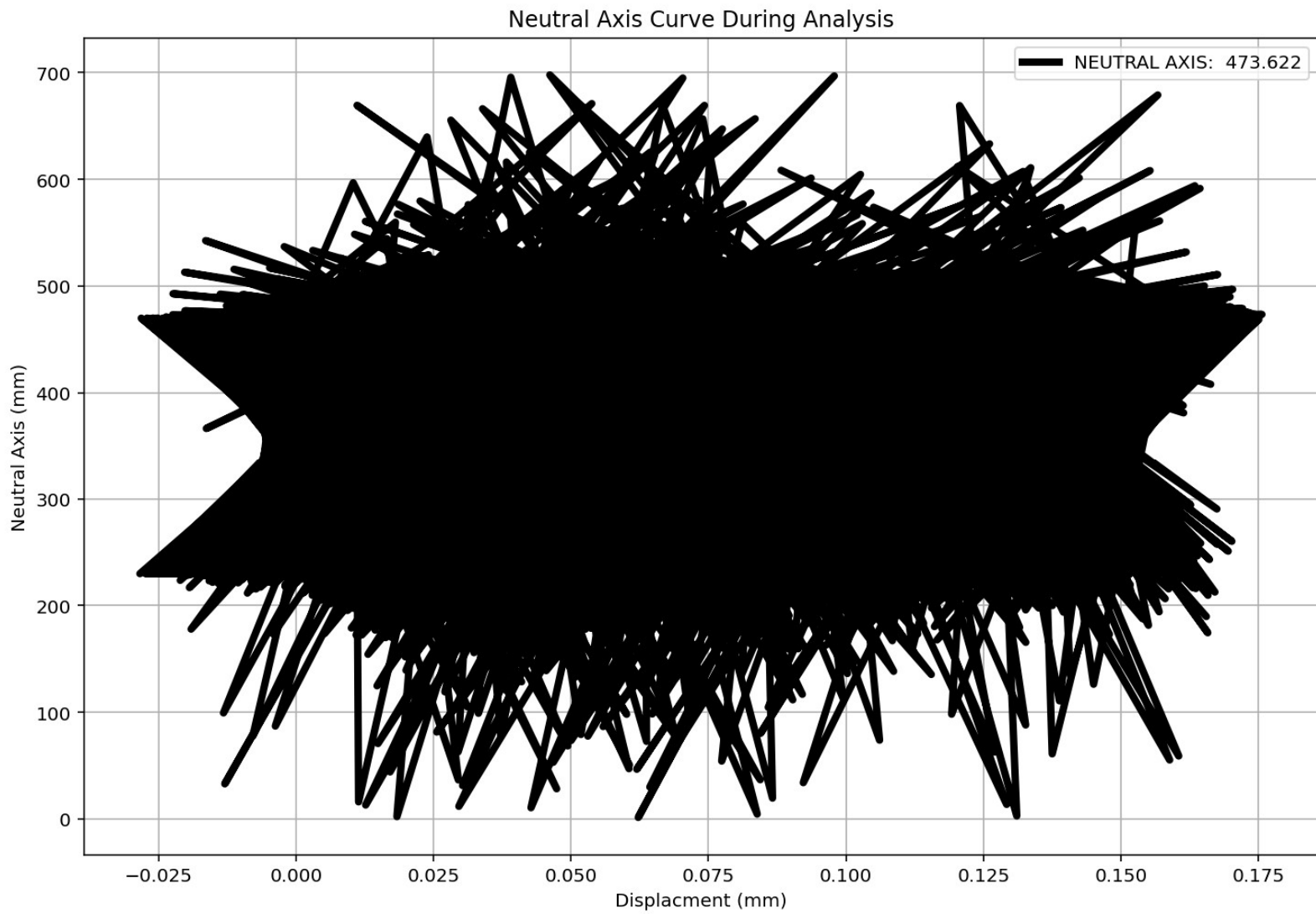




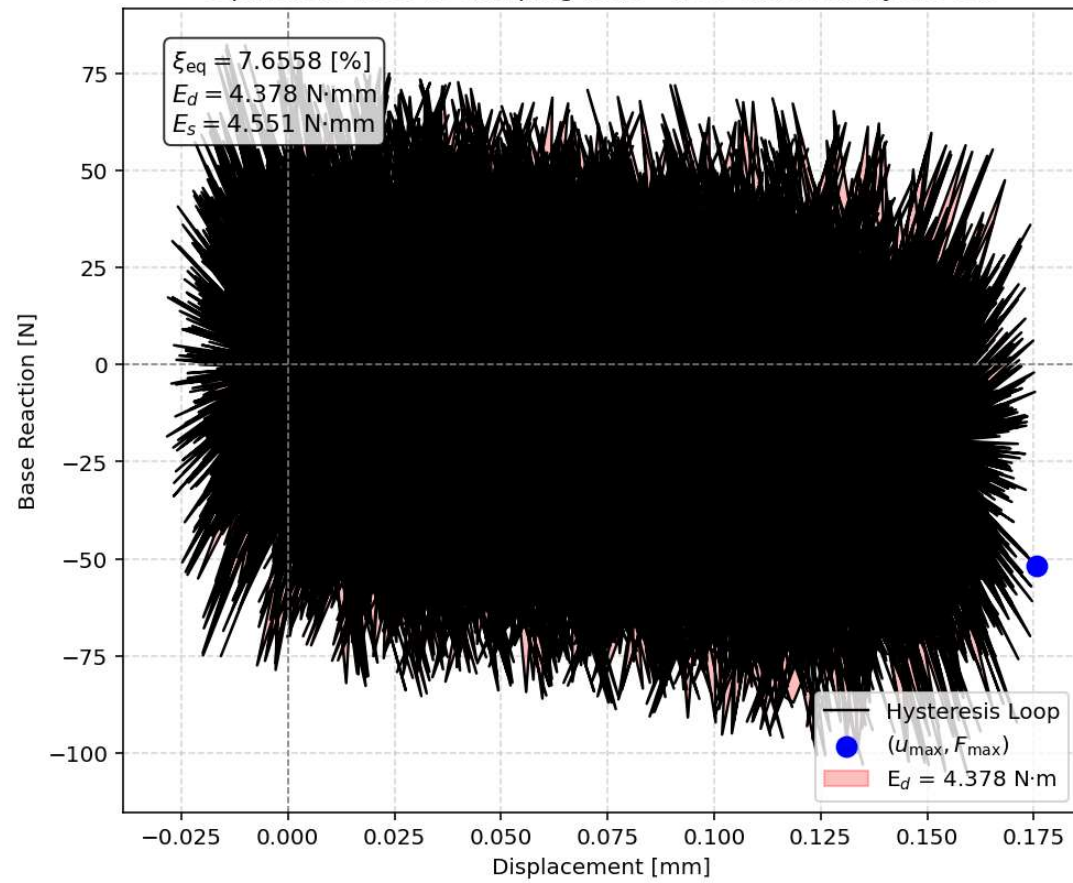
Behavior of element - Stress-strain of Top and Bottom Fibers Curve

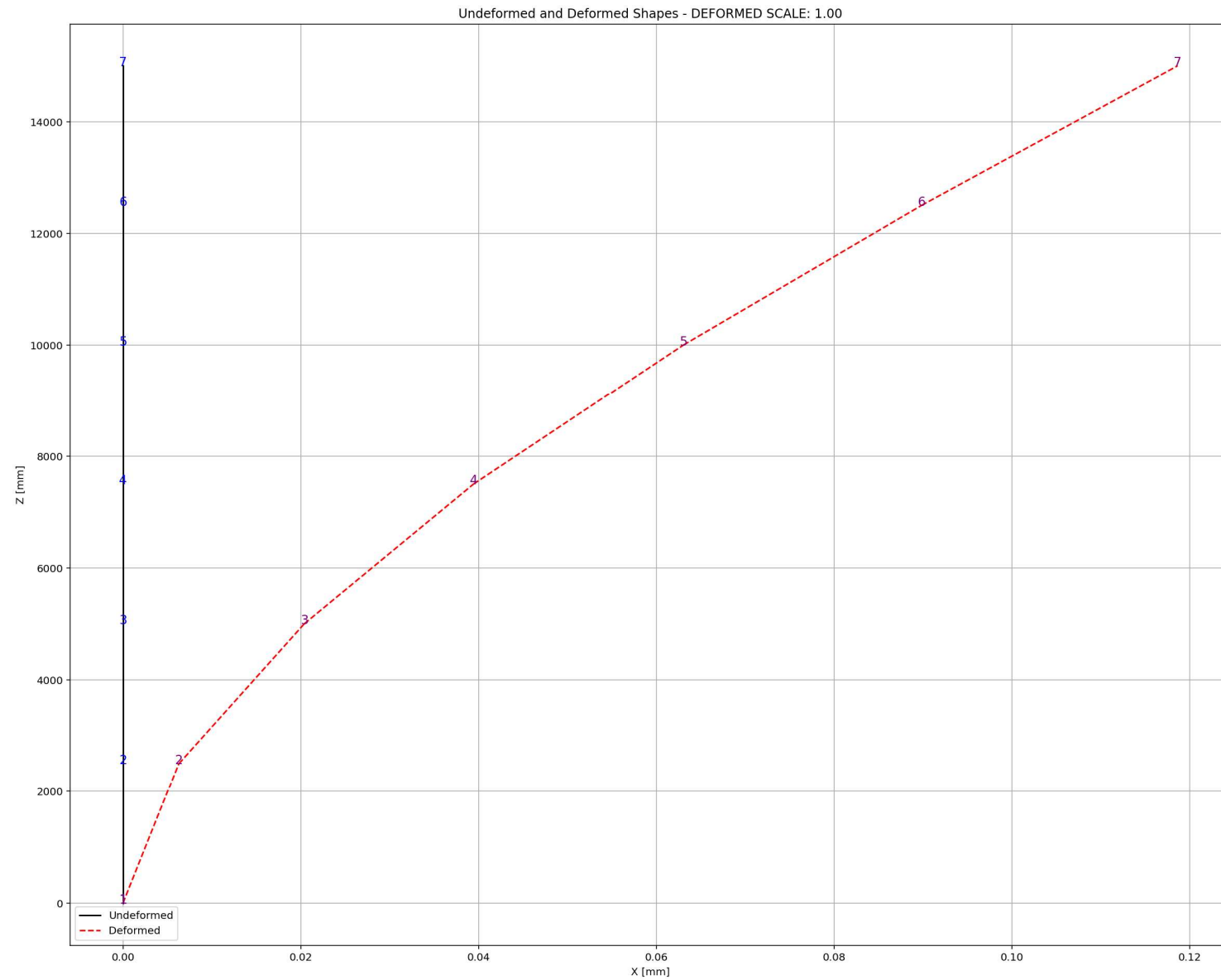






Equivalent viscous damping ratio - Free-vibration Hysteresis

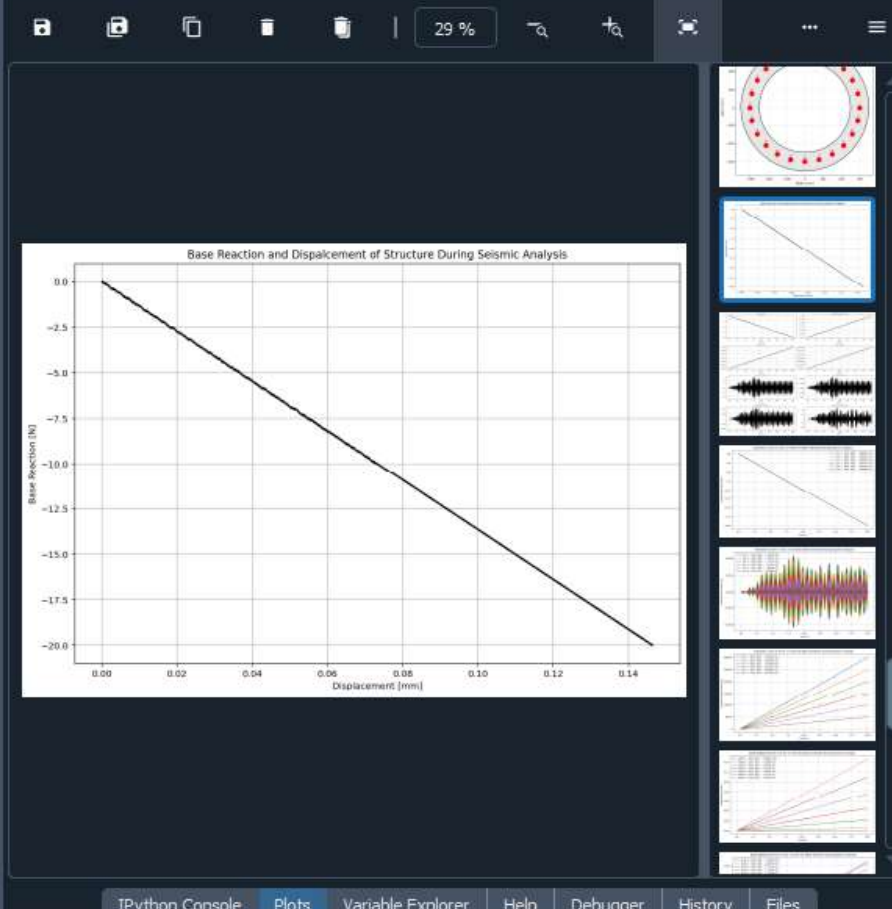


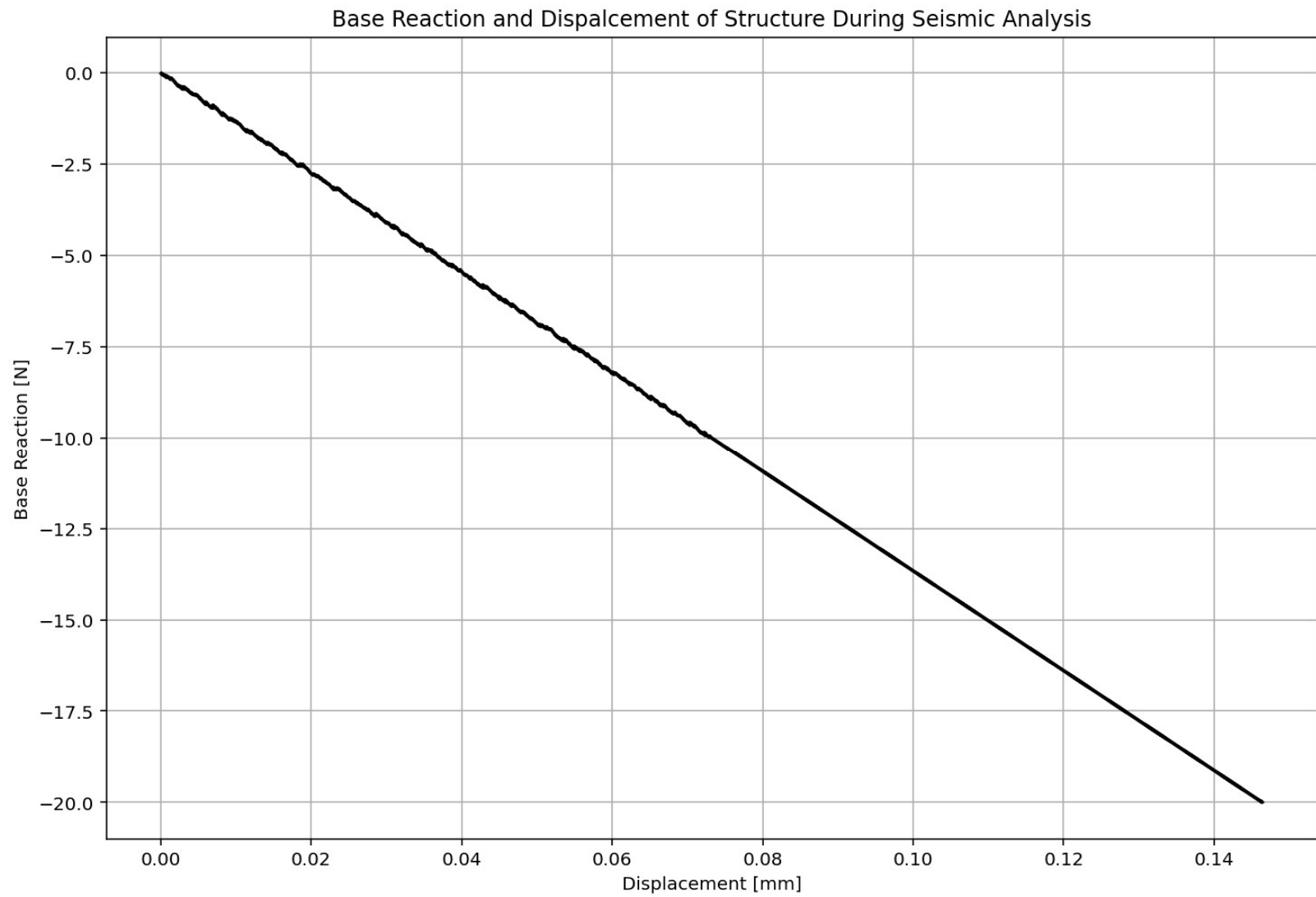


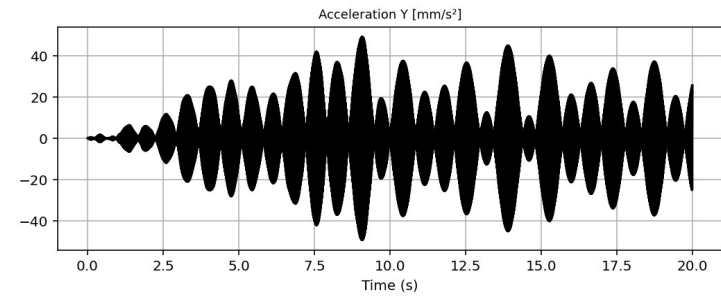
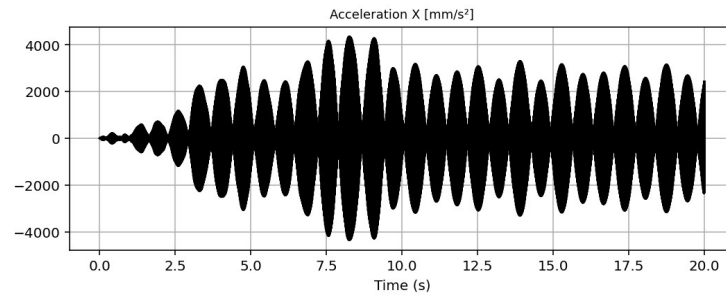
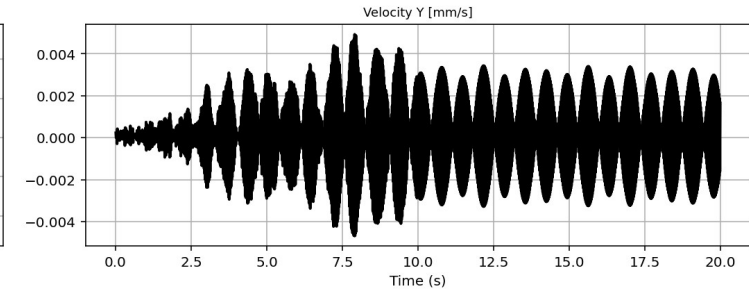
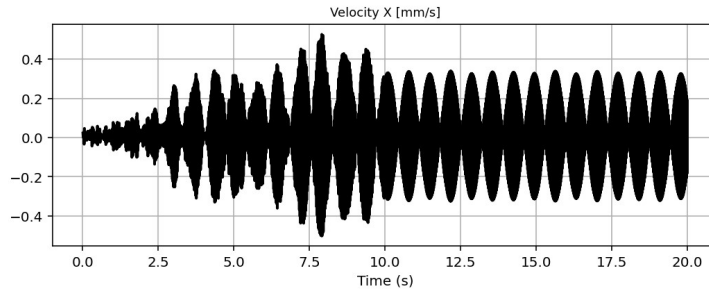
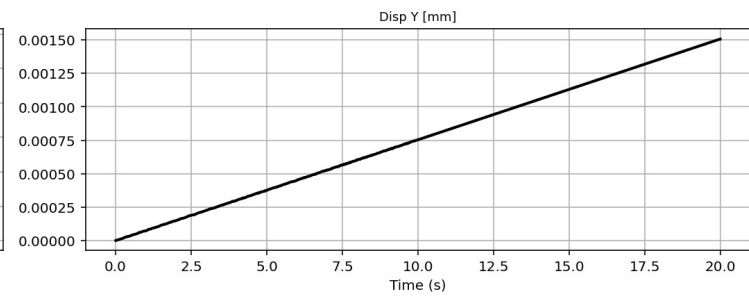
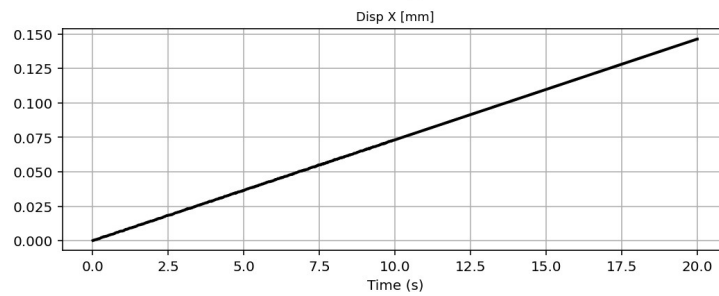
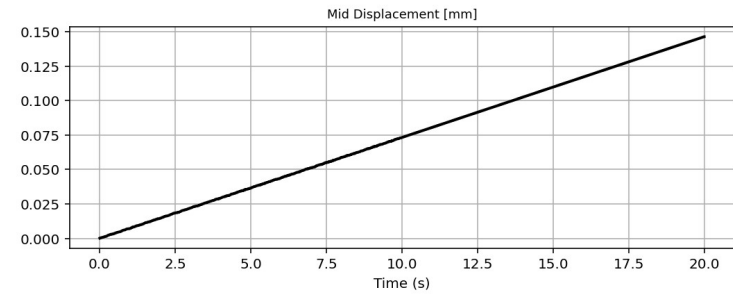
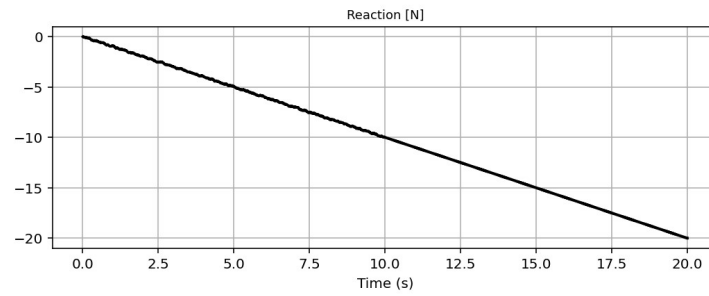
SEISMIC ANALYSIS

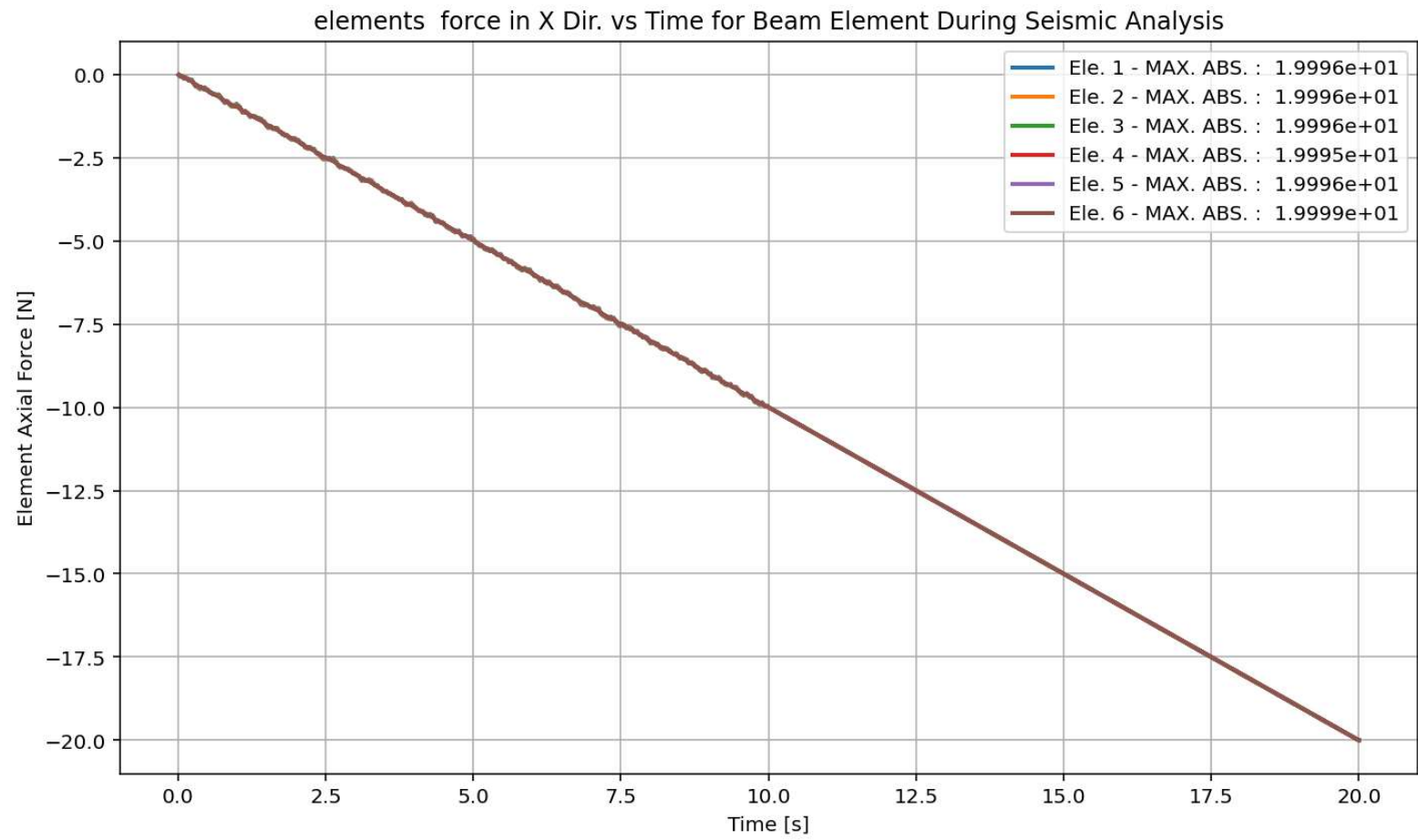
```

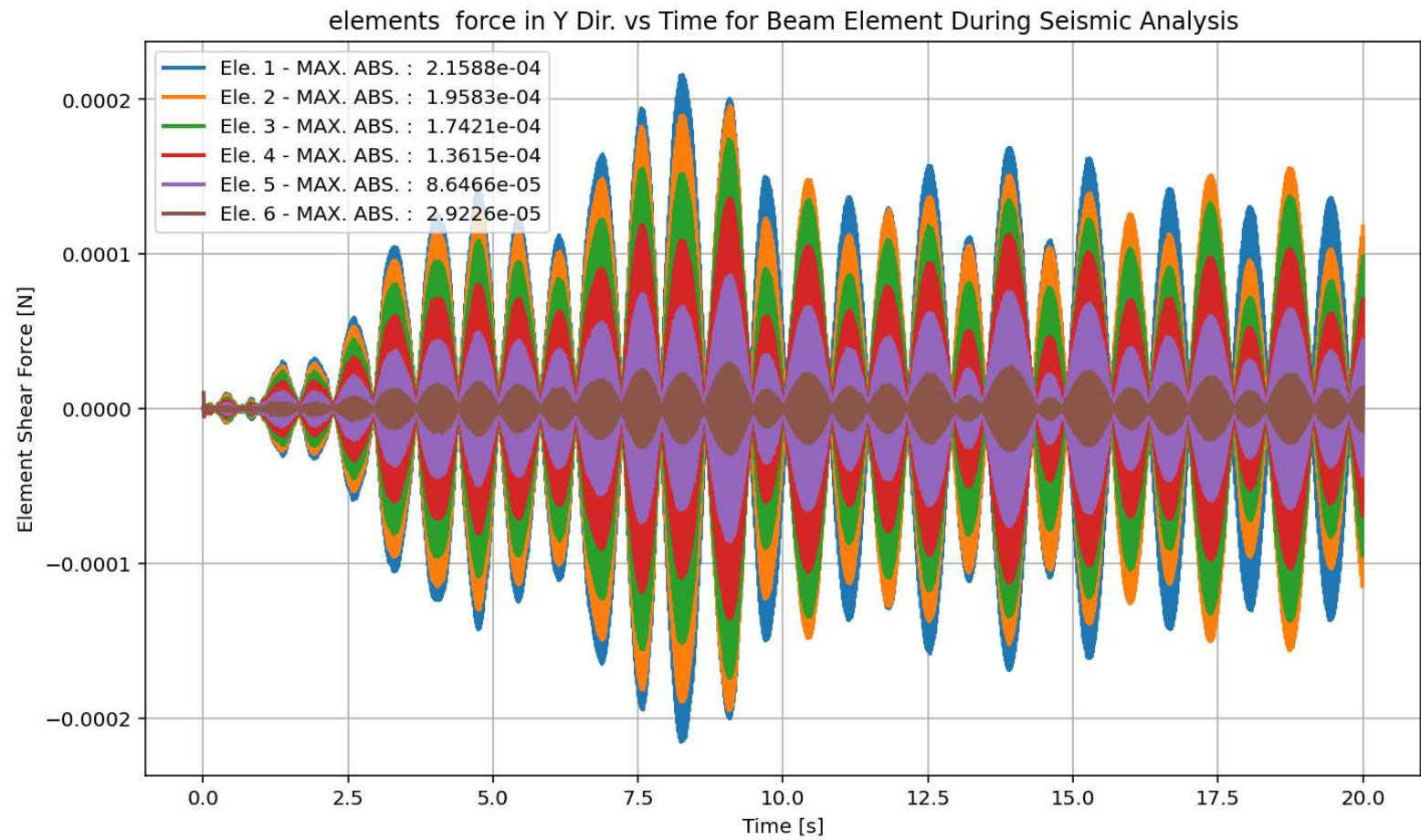
1020 |
1021 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
1022     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
1023     duration = 20.0           # [s] Analysis duration
1024     dt = 0.01                # [s] Time step
1025     GMfact = 9810             # [mm/s2] standard acceleration of gravity or stand
1026     SSF_X = 50.0             # Seismic Acceleration Scale Factor in X Direction
1027     SSF_Y = 50.0             # Seismic Acceleration Scale Factor in Y Direction
1028     iv0_X = 0.0005           # [mm/s] Initial velocity applied to the node in
1029     iv0_Y = 0.0005           # [mm/s] Initial velocity applied to the node in
1030     st_iv0 = 0.0             # [s] Initial velocity applied starting time
1031     SEI = 'X'                 # Seismic Direction
1032     DR = 0.03                # Damping ratio
1033
1034 # Define time series for input motion (Acceleration time history)
1035 if SEI == 'X':
1036     SEISMIC_TAG_01 = 100
1037     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Accelerati
1038     # Define load patterns
1039     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fact $
1040     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01, '-v
1041 if SEI == 'Y':
1042     SEISMIC_TAG_02 = 200
1043     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Accelerati
1044     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02, '-v
1045 if SEI == 'XY':
1046     SEISMIC_TAG_01 = 100
1047     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Accelerati
1048     # Define load patterns
1049     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fact $
1050     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01, '-v
1051     SEISMIC_TAG_02 = 200
1052     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Accelerati
1053     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02, '-v
    
```

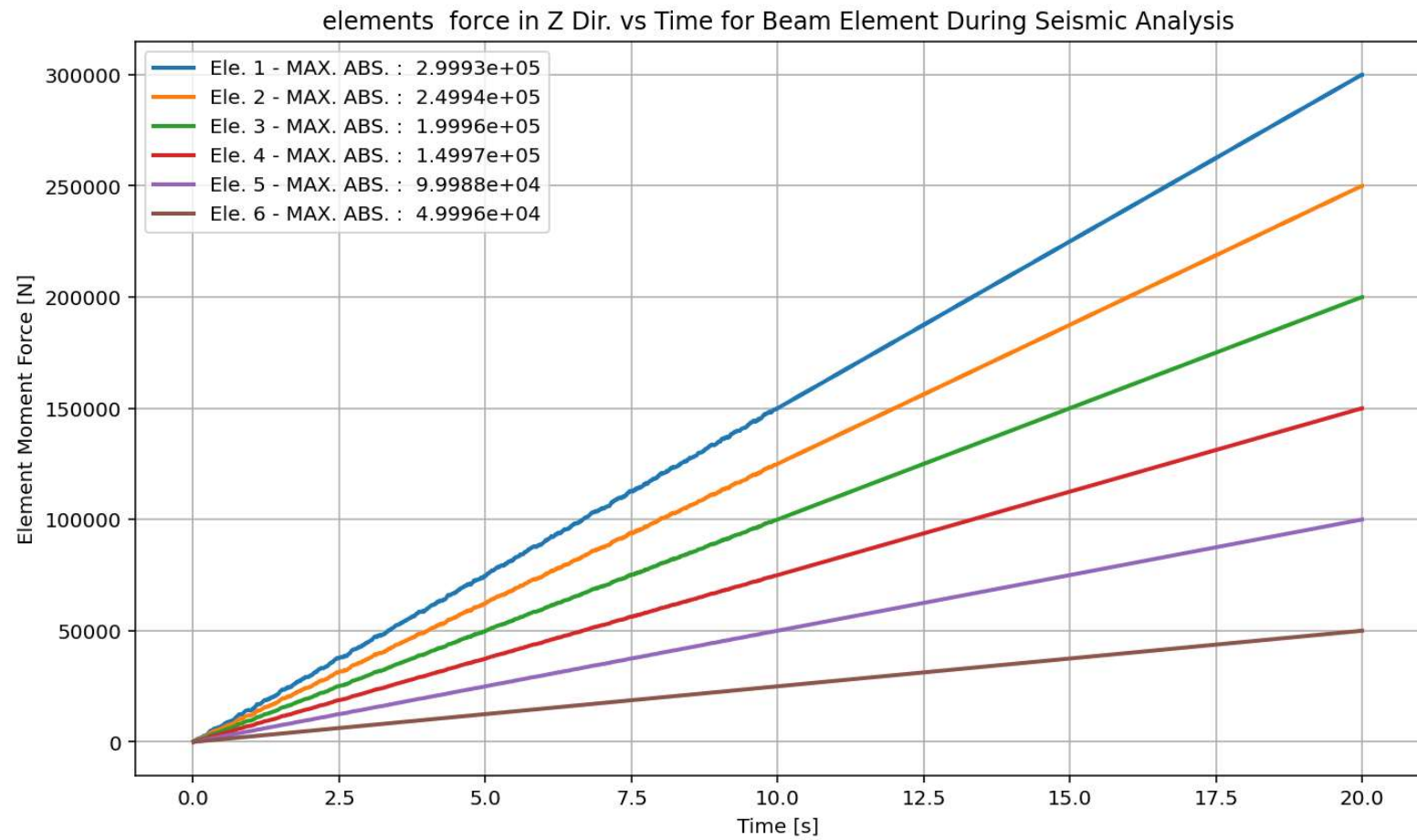




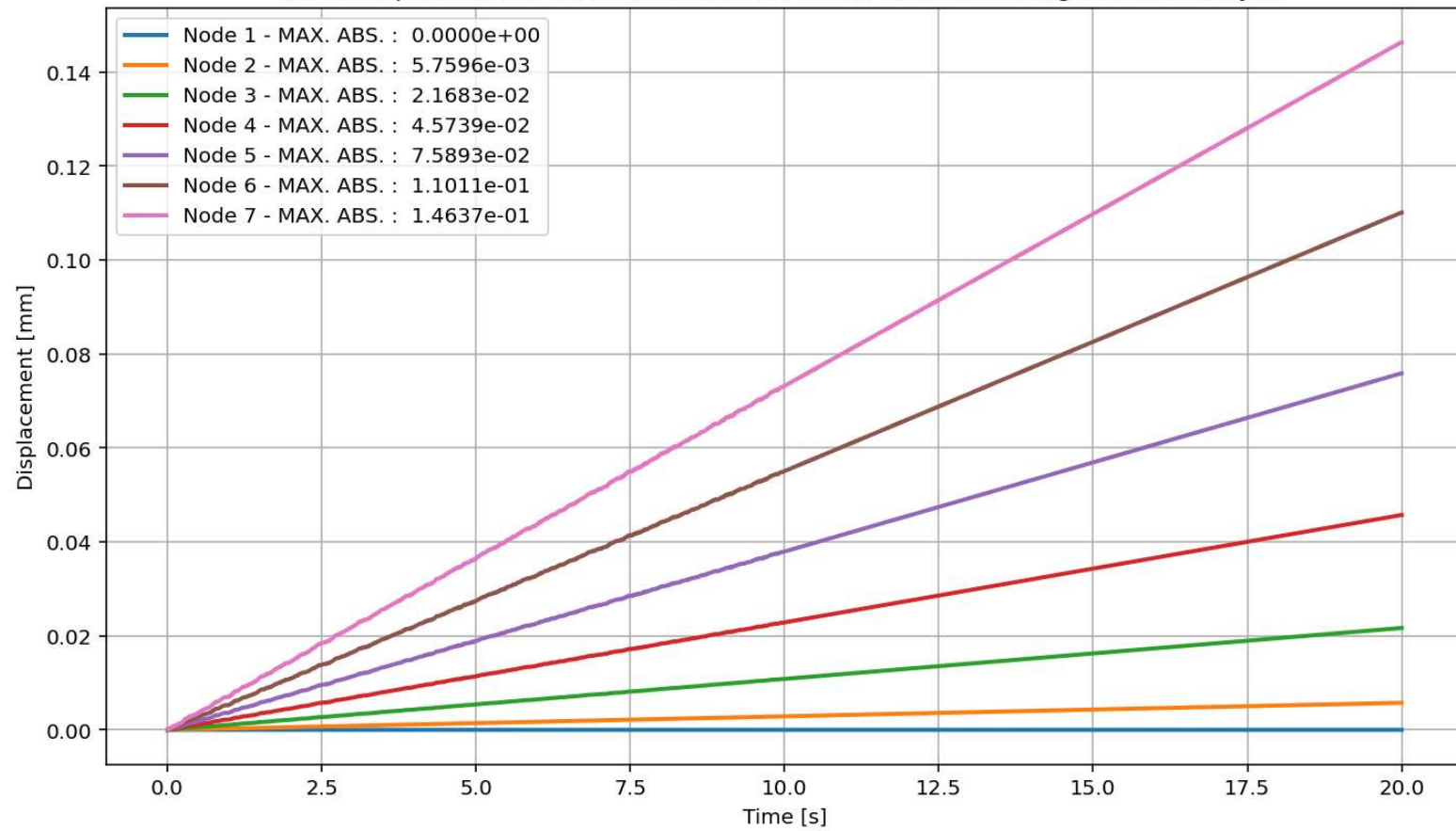




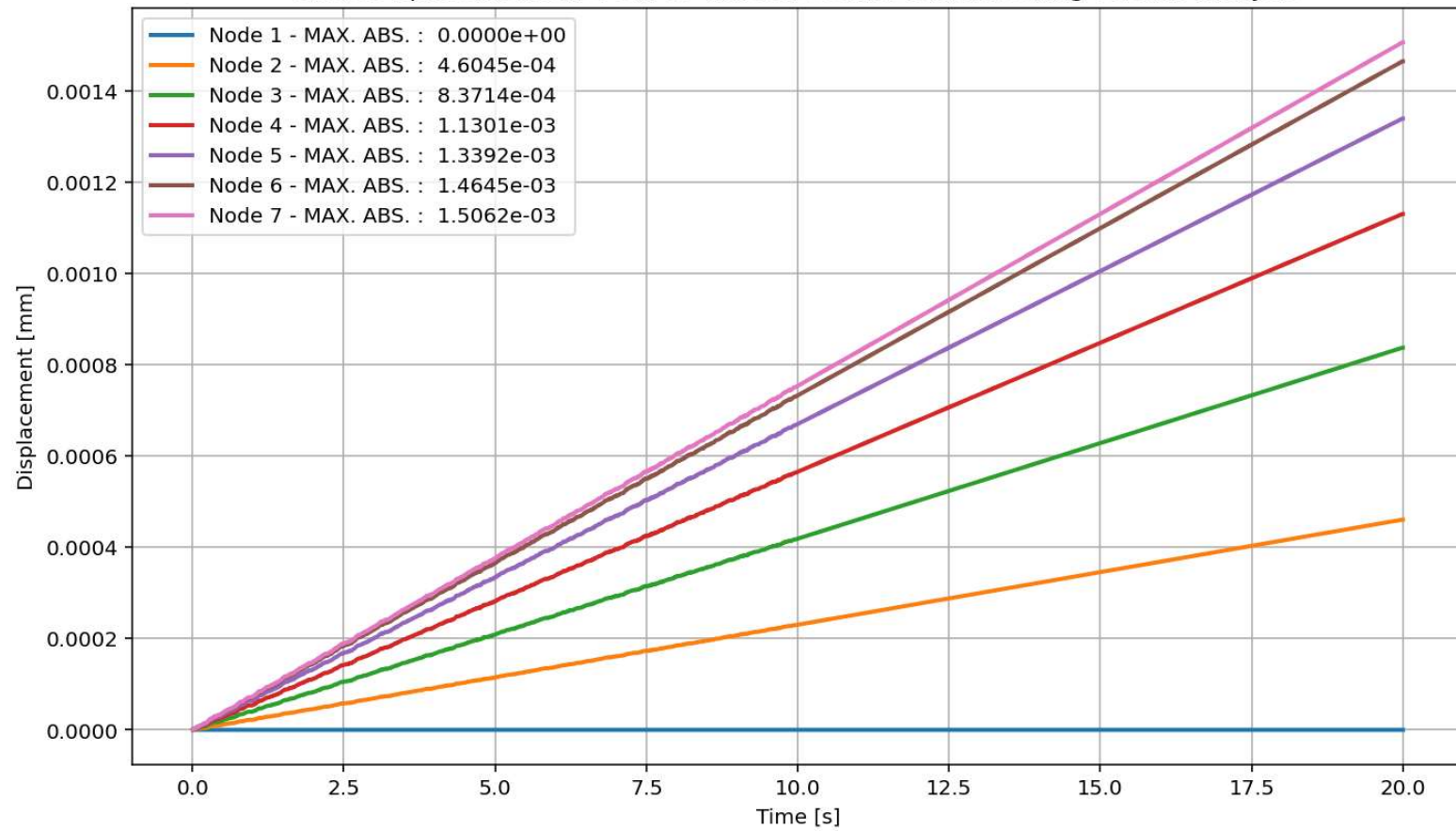


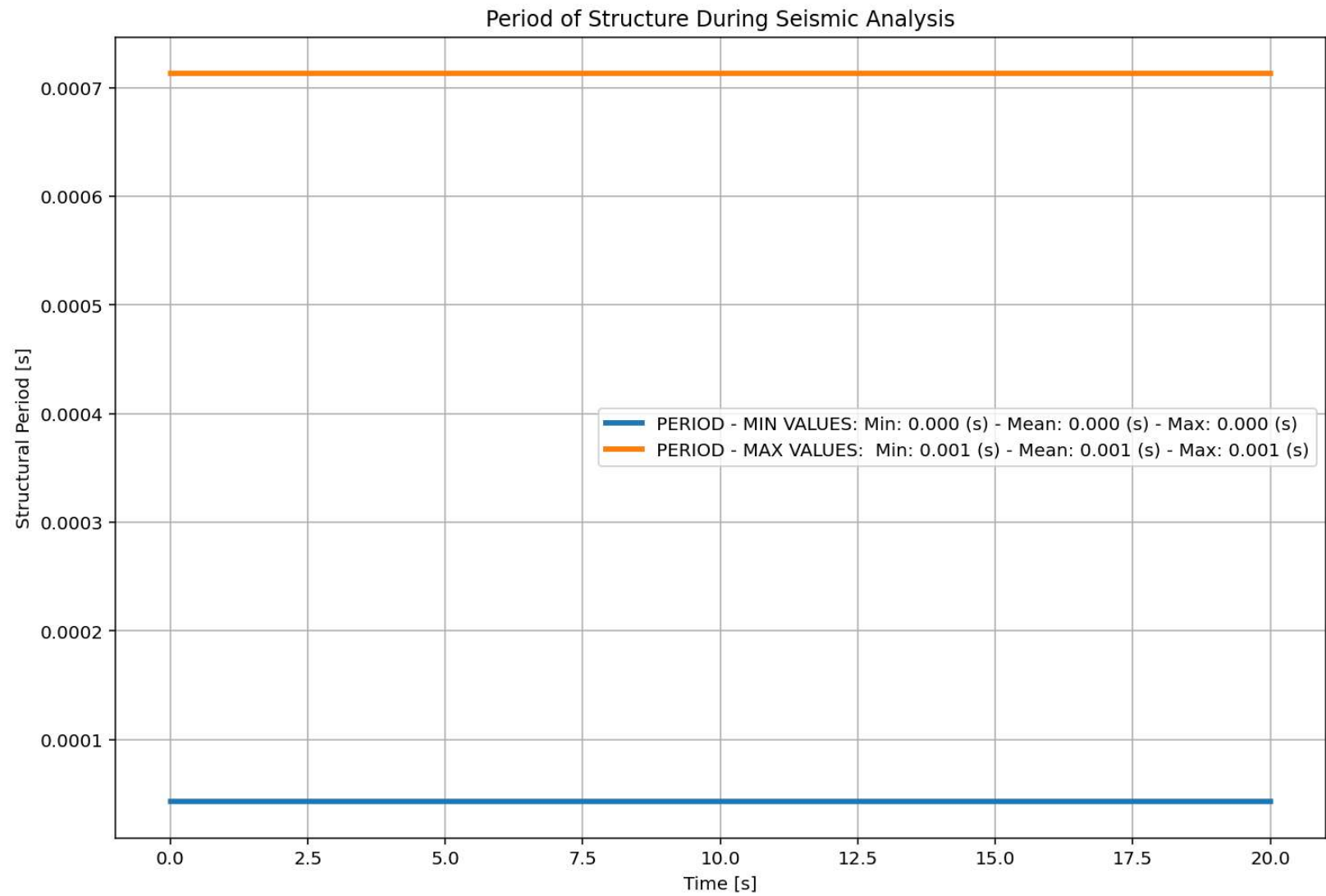


Node Displacements in X Dir. vs Time for Beam Element During Seismic Analysis

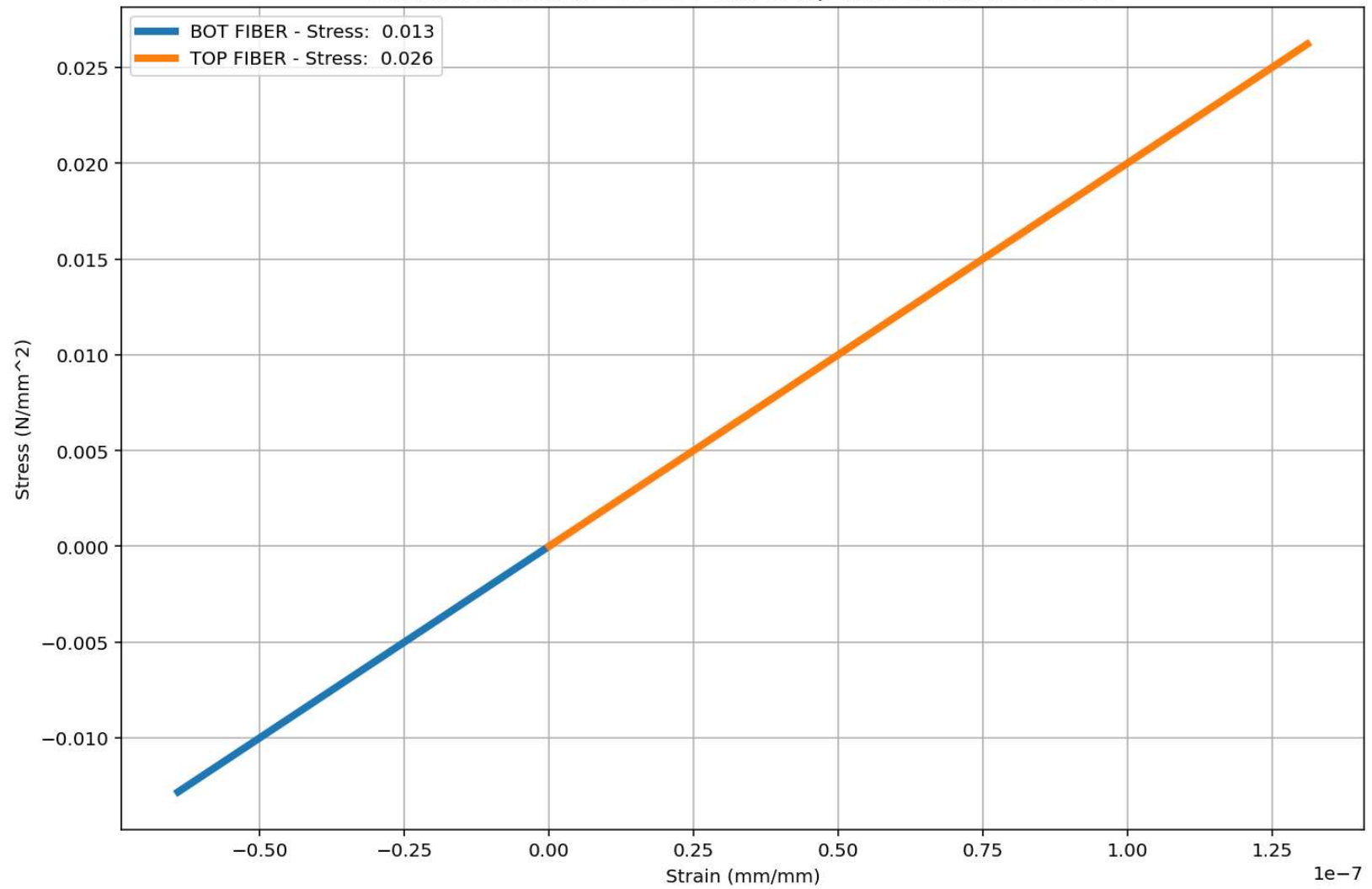


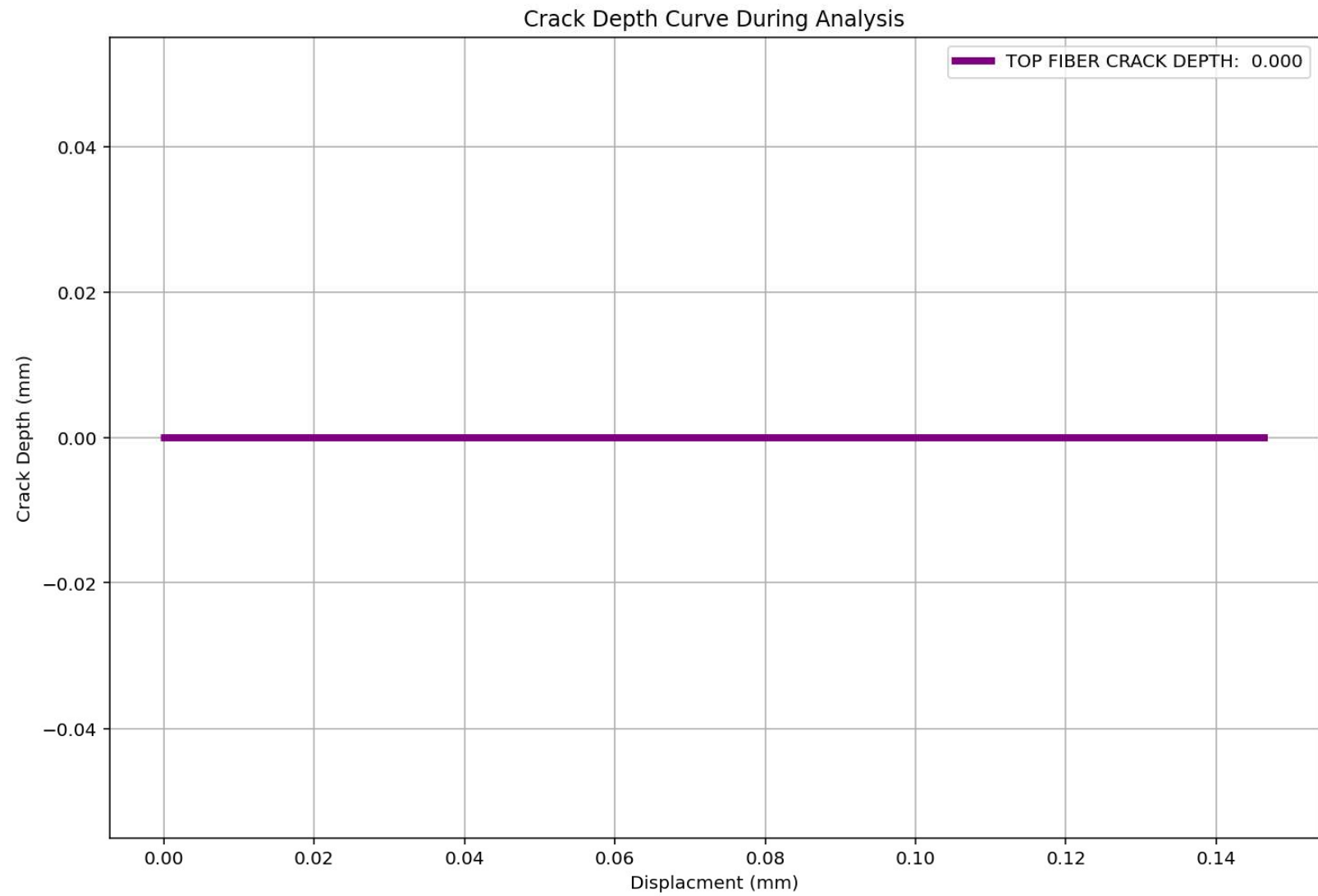
Node Displacements in Y Dir. vs Time for Beam Element During Seismic Analysis

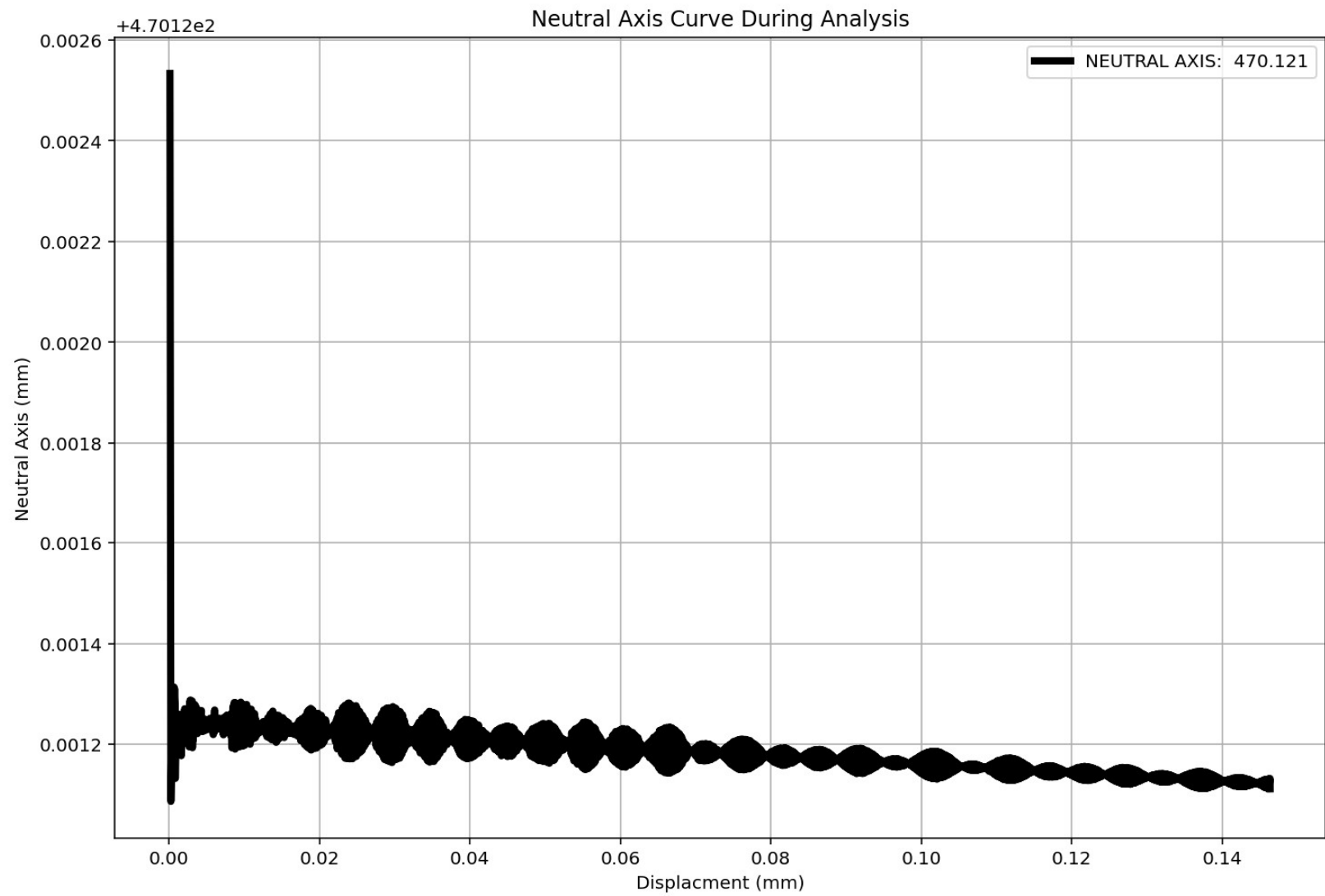




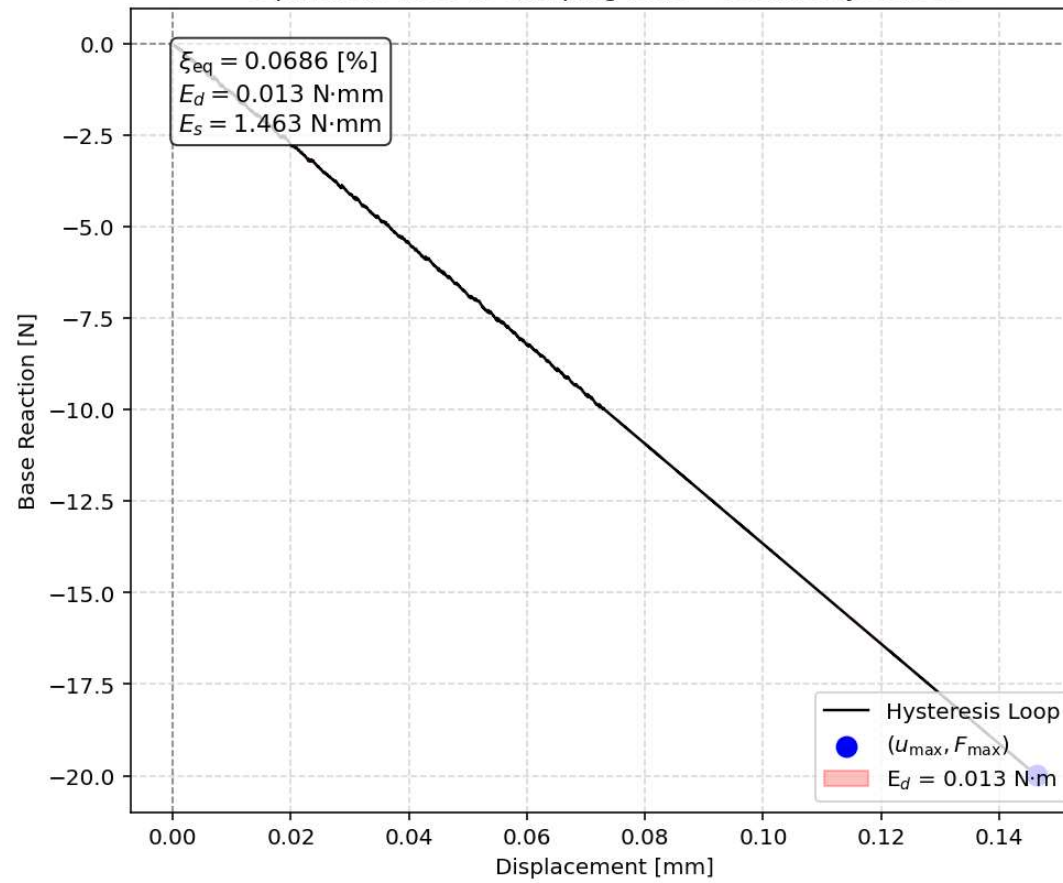
Behavior of element - Stress-strain of Top and Bottom Fibers Curve

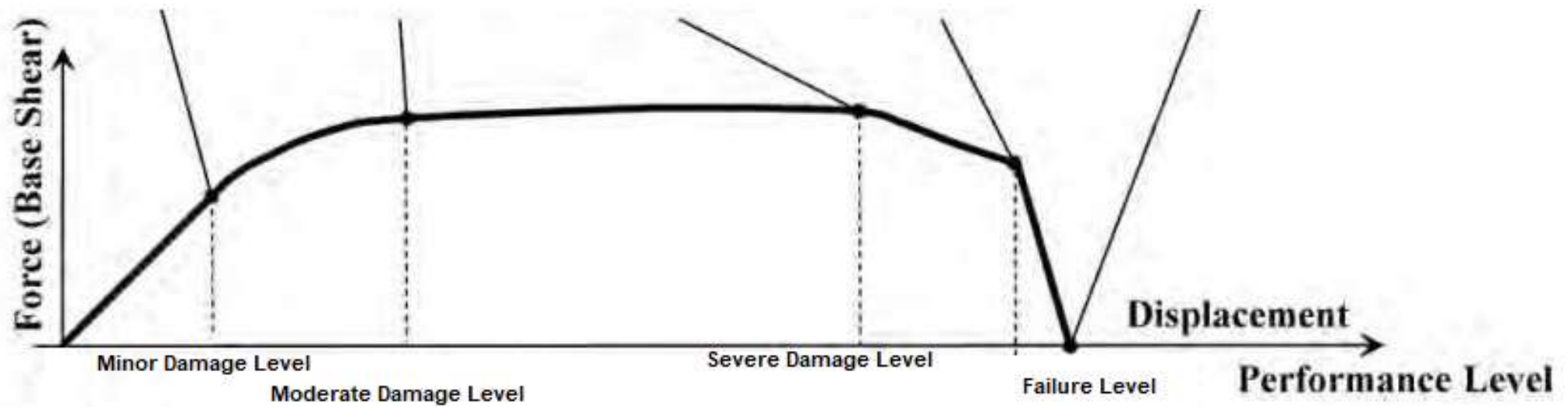
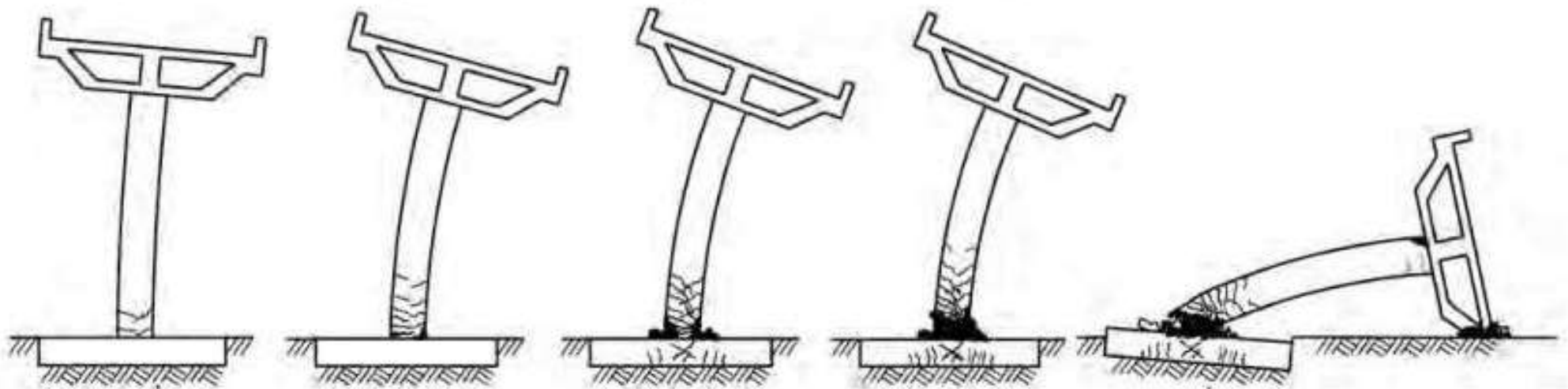


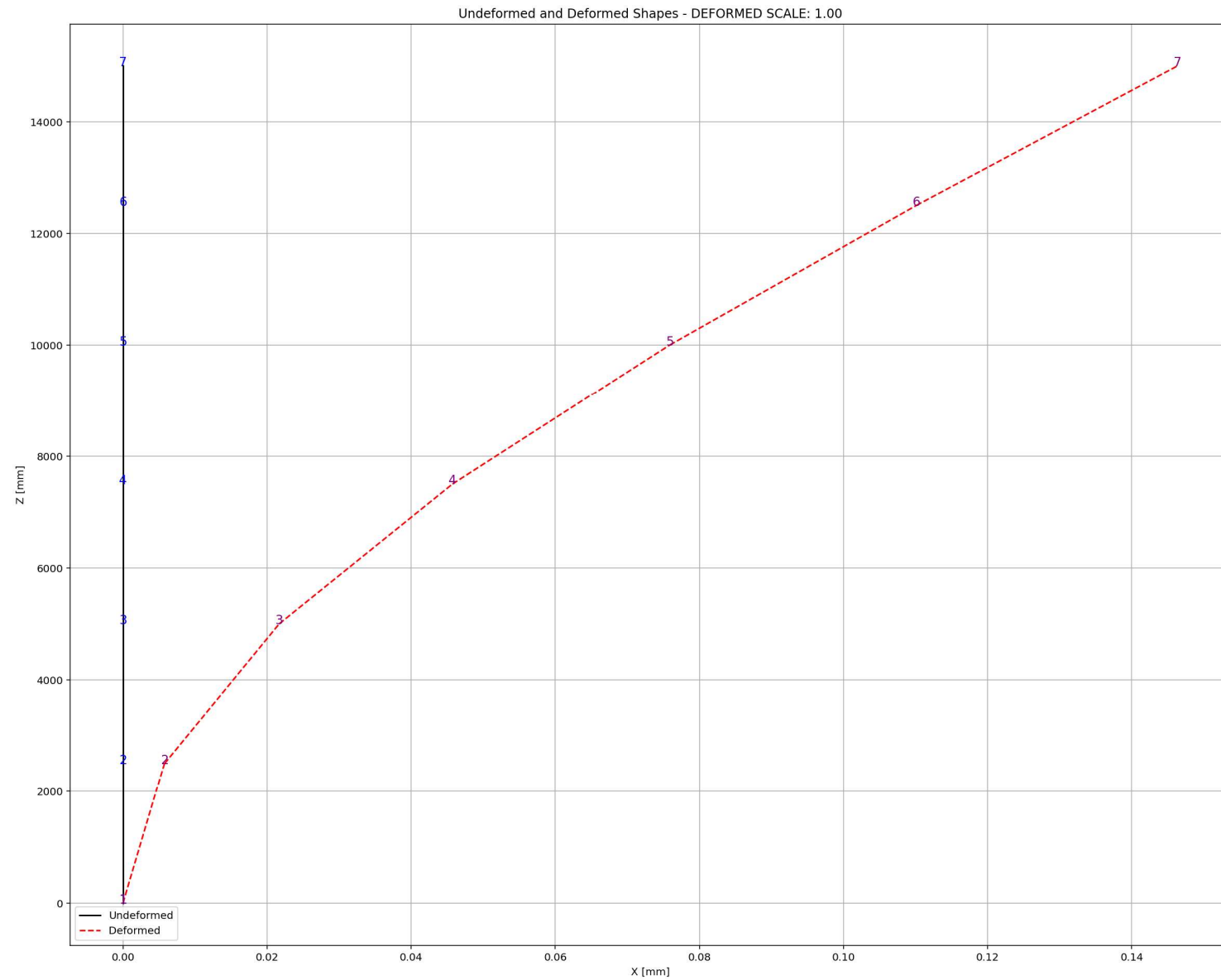




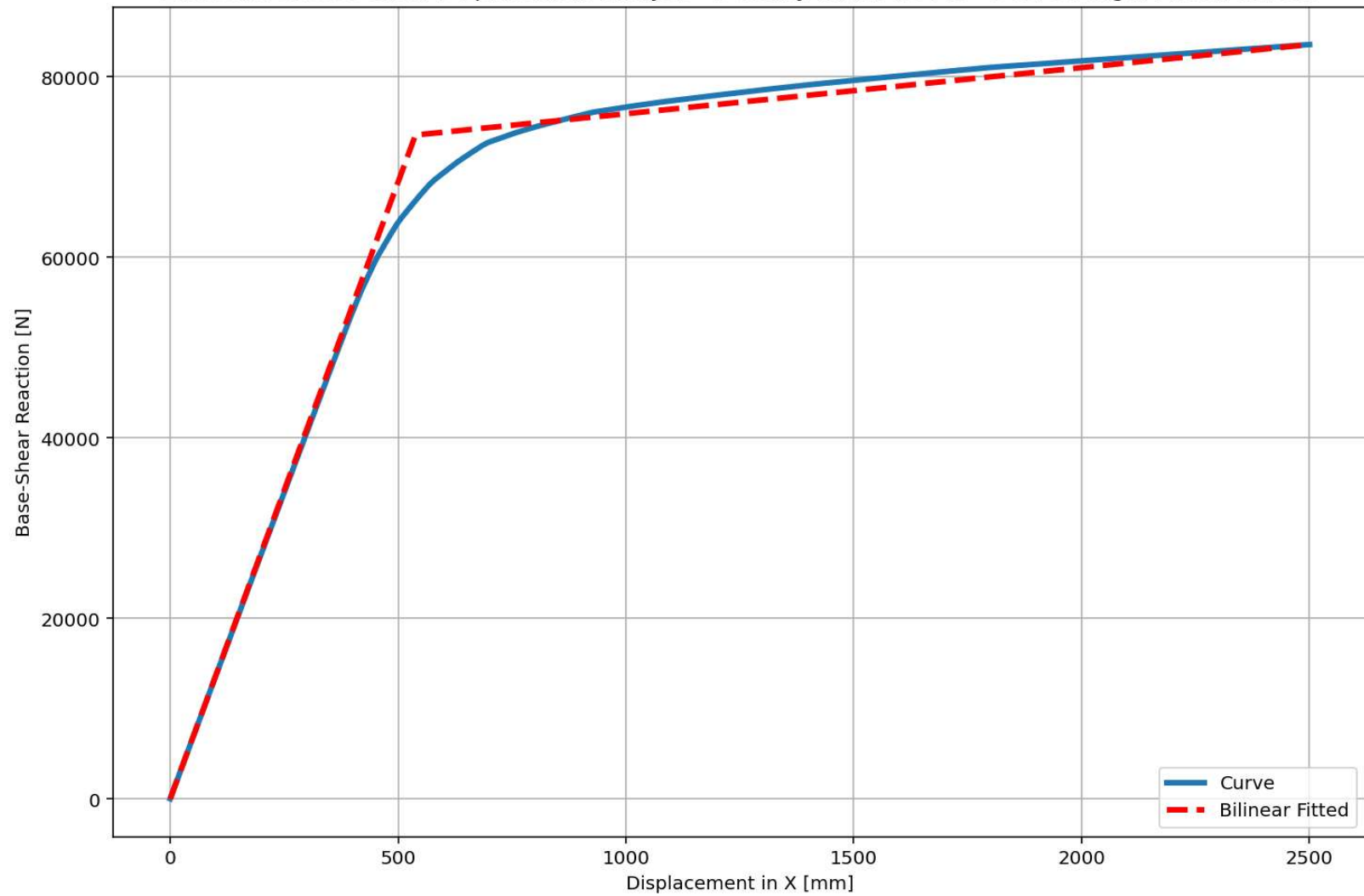
Equivalent viscous damping ratio - Seismic Hysteresis







Last Data of Base Shear-Displacement Analysis - Ductility Ratio: 4.6448 - Over Strength Factor: 1.1363



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0. 0.]

[538.23218671 73528.95054126]

[2500. 83552.50874925]]

+=====+

+-----+

Structure Elastic Stiffness : 136.61

Structure Plastic Stiffness : 33.42

Structure Tangent Stiffness : 5.11

Structure Ductility Ratio : 4.64

Structure Over Strength Factor: 1.14

+-----+

Over Strength Coefficient (Ω_0): 1.1363

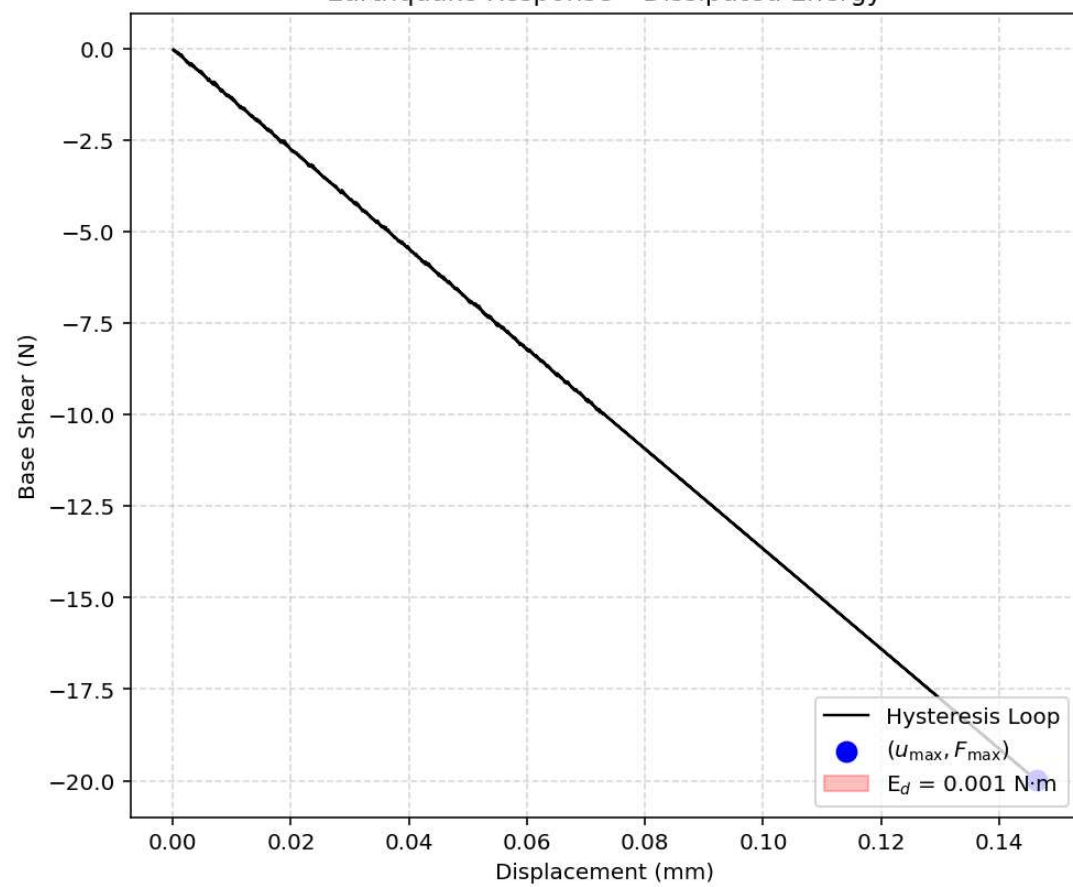
Displacement Ductility Ratio (μ): 4.6448

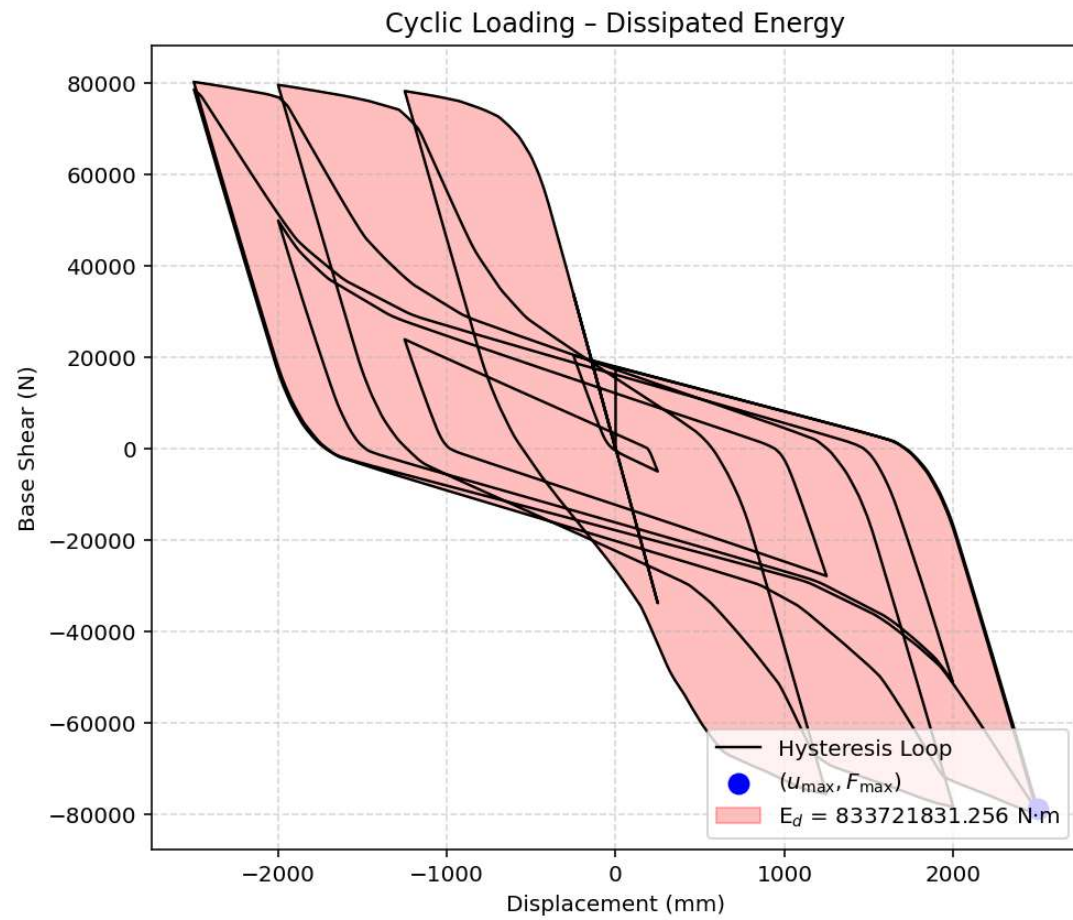
Ductility Coefficient (R_μ): 4.6448

Structural Behavior Coefficient (R): 5.2780

Structural Ductility Damage Index in X Direction: -27.4286 (%)

Earthquake Response - Dissipated Energy





Dissipated Energy from Earthquake= 0.00 N·mm

Dissipated Energy from Cyclic Displacement= 833721831.26 N·mm

DISSIPATED ENERGY CAPACITY INDEX: 0.000 [%]

ZONE 1: VERY MINOR DAMAGE

